
DATA ANALYST

Série 2 — Volume II

Cahier d'exercices et d'applications

Exercices corrigés · Applications guidées · Projet
Compagnon du volume II : Préparation des données

À utiliser avec le livre : chaque chapitre renvoie ici
pour s'entraîner.

Table des matières

Mode d'emploi	1
Comment utiliser ce cahier	1
Numérotation et niveaux	1
Données et environnement	2
Lancer le code	3
Vérifier son installation	3
Mini-diagnostic de départ	4
Corrigés du mini-diagnostic	5
Chapitre 1 : Nettoyage des données — exercices et applications	9
Applications	9
Application 1.1 — Profiler un fichier avant tout calcul (sections 1.1.1 et 1.1.2)	9
Application 1.2 — Tester le mécanisme d'absence (section 1.1.3)	10
Application 1.3 — Détecter les aberrantes : un compromis précision/rappel (sections 1.2.2 et 1.2.3)	11
Application 1.4 — Dédoublonner les commandes du site avec un journal (section 1.3.2)	12
Application 1.5 — Détecter et corriger le changement d'unité (section 1.4.2)	13
Application 1.6 — Lire les douze fichiers de caisse et les réconcilier (sections 1.3.3 et 1.4.6)	14
Application 1.7 — Nettoyer le fichier clients du CRM (sections 1.4.4, 1.4.5 et 1.5) .	16
Application 1.8 — Comparer les imputations à la vérité, avec vos propres trous (section 1.6)	17
Exercices	18
Exercice 1.1 ★ — Deux lectures d'un même fichier (section 1.1.1)	18
Exercice 1.2 ★★ — Déduire avant d'imputer (section 1.1.2)	18
Exercice 1.3 ★★ — Le mécanisme de la dépense (section 1.1.3)	18
Exercice 1.4 ★ — Les seuils à la main (section 1.2.2)	18
Exercice 1.5 ★★ — Plafonner à 95 %, 99 %, 99,9 % (section 1.2.4)	18
Exercice 1.6 ★ — Doublons d'un petit tableau (section 1.3.1)	18
Exercice 1.7 ★★ — Le plus récent ou le plus complet ? (section 1.3.4)	19
Exercice 1.8 ★★ — Corriger « à l'œil » (section 1.4.2)	19
Exercice 1.9 ★★ — Un treizième fichier (section 1.4.6)	19
Exercice 1.10 ★★ — Normaliser des téléphones (section 1.5.3)	19
Exercice 1.11 ★★ — Médiane de groupe contre moyenne de groupe (section 1.6.2) .	19
Exercice 1.12 ★★ — Un MNAR que vous fabriquez, et un δ à choisir (section 1.6.6) .	19
Corrigés	20
Corrigé 1.1	20
Corrigé 1.2	20

Corrigé 1.3	20
Corrigé 1.4	21
Corrigé 1.5	21
Corrigé 1.6	21
Corrigé 1.7	22
Corrigé 1.8	22
Corrigé 1.9	23
Corrigé 1.10	23
Corrigé 1.11	24
Corrigé 1.12	24
Chapitre 2 : Transformation et fusion des données — exercices et applications	26
Applications	26
Application 2.1 — La marge par canal et par trimestre (section 2.1)	26
Application 2.2 — Délais, clients dormants et fuite d'information (sections 2.1.5 et 2.1.7)	27
Application 2.3 — Contrôler une jointure (sections 2.2.2 et 2.2.5)	28
Application 2.4 — Lire un fichier de janvier et un fichier de décembre (section 2.2.3)	29
Application 2.5 — Nettoyer l'export du site (section 2.2.4)	30
Application 2.6 — La table des ventes et son contrôle (sections 2.2.6 à 2.2.8)	30
Application 2.7 — Agréger et contrôler (section 2.3)	31
Application 2.8 — Le tableur de stocks (section 2.4)	32
Application 2.9 — Dédoublonner le CRM sans téléphone (section 2.5)	33
Application 2.10 — Rapprocher le catalogue du fournisseur (section 2.5.7)	34
Exercices	34
Exercice 2.1 ★ — La marge à la main (section 2.1.2)	34
Exercice 2.2 ★ — Les semaines ISO (section 2.1.3)	34
Exercice 2.3 ★★ — Les bornes d'une classe (section 2.1.4)	35
Exercice 2.4 ★★ — Zéro ou vide ? (section 2.1.7)	35
Exercice 2.5 ★ — Le type de jointure (section 2.2.1)	35
Exercice 2.6 ★★ — Prévoir le nombre de lignes (section 2.2.2)	35
Exercice 2.7 ★★ — Le mojibake et le codage (section 2.2.3)	35
Exercice 2.8 ★★ — Chercher une rupture d'unité (section 2.2.4)	35
Exercice 2.9 ★ — Le panier moyen par canal (section 2.3.2)	35
Exercice 2.10 ★★ — La moyenne mobile (section 2.3.6)	35
Exercice 2.11 ★★ — L'aller-retour pivot (section 2.4.1)	36
Exercice 2.12 ★★ — Inconnu n'est pas zéro (section 2.4.2)	36
Exercice 2.13 ★★ — Levenshtein à la main (section 2.5.3)	36
Exercice 2.14 ★★ — Choisir un seuil par les coûts (section 2.5.6)	36
Pistes pour les « À vous » des applications	36
Piste de l'application 2.1	36
Piste de l'application 2.2	36
Piste de l'application 2.3	37
Piste de l'application 2.4	37
Piste de l'application 2.5	37
Piste de l'application 2.6	38
Piste de l'application 2.7	38
Piste de l'application 2.8	39
Piste de l'application 2.9	39
Piste de l'application 2.10	39
Corrigés des exercices	40
Corrigé 2.1	40

Corrigé 2.2	40
Corrigé 2.3	40
Corrigé 2.4	41
Corrigé 2.5	41
Corrigé 2.6	41
Corrigé 2.7	41
Corrigé 2.8	42
Corrigé 2.9	42
Corrigé 2.10	42
Corrigé 2.11	43
Corrigé 2.12	43
Corrigé 2.13	43
Corrigé 2.14	44
Chapitre 3 : Qualité des données et réconciliation — exercices et applications	45
Applications	45
Application 3.1 — Mesurer la qualité du CRM (section 3.1)	46
Application 3.2 — Écrire les contrôles du CRM et les rejouer en SQL (section 3.2) .	47
Application 3.3 — Plage ou dépendance ? (section 3.2.3)	48
Application 3.4 — Détecter une rupture dans une série (section 3.2.5)	48
Application 3.5 — Réconcilier le site avec la base (section 3.3.3)	49
Application 3.6 — Réconcilier la caisse avec la base (section 3.3.4)	50
Application 3.7 — Réconcilier le catalogue du fournisseur (section 3.3.5)	51
Application 3.8 — Tolérances, règles et rapport d'exceptions (section 3.4)	52
Application 3.9 — pandera et Great Expectations sur l'export du site (section 3.5) .	52
Exercices	53
Exercice 3.1 ★ — La complétude du CRM (section 3.1.2)	54
Exercice 3.2 ★ — Valider un téléphone (section 3.1.3)	54
Exercice 3.3 ★★ — Choisir une clé d'unicité (section 3.1.3 et 3.1.8)	54
Exercice 3.4 ★★ — Le tableau de bord avant et après correction (section 3.1.7) . . .	54
Exercice 3.5 ★ — La moyenne qui cache (section 3.1.8)	54
Exercice 3.6 ★ — Trop strict ou normalisable ? (section 3.2.2)	54
Exercice 3.7 ★★ — Un contrôle de dépendance (section 3.2.3)	54
Exercice 3.8 ★★ — La caisse a-t-elle une rupture ? (section 3.2.5)	54
Exercice 3.9 ★★ — Des contraintes SQL (section 3.2.7)	55
Exercice 3.10 ★★ — La cascade de la caisse en décembre (section 3.3.4)	55
Exercice 3.11 ★★★ — La cascade du site en septembre (section 3.3.3)	55
Exercice 3.12 ★★★ — Tolérance et exceptions par mois (section 3.4)	55
Corrigés	55
Corrigé 3.1	55
Corrigé 3.2	55
Corrigé 3.3	56
Corrigé 3.4	56
Corrigé 3.5	57
Corrigé 3.6	57
Corrigé 3.7	57
Corrigé 3.8	57
Corrigé 3.9	58
Corrigé 3.10	58
Corrigé 3.11	58
Corrigé 3.12	59

Chapitre 4 : Documentation et dictionnaires de données — exercices et applications	60
Applications	60
Application 4.1 — La fiche de l'export du site (section 4.1.2)	60
Application 4.2 — Le journal du nettoyage de l'export du site (sections 4.1.3 et 4.1.4)	62
Application 4.3 — Le dictionnaire de clients et de produits (sections 4.2.2 et 4.2.3)	63
Application 4.4 — Le test du dictionnaire (section 4.2.5)	64
Application 4.5 — Un glossaire pour la boutique (section 4.2.6)	65
Application 4.6 — Le lignage d'un rapport et l'analyse d'impact (sections 4.3.2 et 4.3.3)	66
Application 4.7 — Empreintes et manifeste (sections 4.3.4 et 4.3.6)	67
Application 4.8 — Rejouer un chiffre depuis le brut (section 4.3.8)	68
Exercices	69
Exercice 4.1 ★ — Qu'est-ce qui manque ? (section 4.1.1)	69
Exercice 4.2 ★ — La fiche du catalogue fournisseur (section 4.1.2)	69
Exercice 4.3 ★★ — L'équation de conservation (section 4.1.3)	69
Exercice 4.4 ★★ — Nommer ses fichiers (section 4.1.6)	69
Exercice 4.5 ★ — Des commentaires qui disent pourquoi (section 4.1.6)	69
Exercice 4.6 ★★ — Ce que le squelette ne sait pas dire (section 4.2.2)	70
Exercice 4.7 ★★ — Le dictionnaire de cinq colonnes (section 4.2.3)	70
Exercice 4.8 ★★ — Un test sur le statut (section 4.2.5)	70
Exercice 4.9 ★ — Une définition unique de « commande annulée » (section 4.2.6)	70
Exercice 4.10 ★★ — Le lignage d'une campagne (section 4.3.2)	70
Exercice 4.11 ★★ — Un journal d'audit qui ne s'efface pas (section 4.3.5)	70
Exercice 4.12 ★★ — Une documentation qui se trompe (section 4.3.8)	70
Corrigés	70
Corrigé 4.1	71
Corrigé 4.2	71
Corrigé 4.3	71
Corrigé 4.4	71
Corrigé 4.5	72
Corrigé 4.6	72
Corrigé 4.7	72
Corrigé 4.8	73
Corrigé 4.9	73
Corrigé 4.10	73
Corrigé 4.11	74
Corrigé 4.12	74
Pistes pour les « À vous » des applications	75
Chapitre 5 : Confidentialité et anonymisation des données — exercices et applications	76
Applications	76
Application 5.1 — Classer les colonnes et normaliser le consentement (section 5.1 du livre)	76
Application 5.2 — Attaque par dictionnaire et hachage à clé (section 5.2 du livre)	77
Application 5.3 — Le coût du bruit et de la synthèse (section 5.2.4 du livre)	78
Application 5.4 — Unicité et k-anonymat sur vos propres combinaisons (sections 5.3.1 à 5.3.3 du livre)	79
Application 5.5 — Recoupement par date et montant (section 5.3.5 du livre)	80
Application 5.6 — l-diversité et bruit de Laplace (sections 5.3.6 et 5.3.7 du livre)	80
Application 5.7 — Le fichier d'envoi, les petits effectifs et le contrôle (section 5.4 du livre)	81
Exercices	82

Exercice 5.1 ★ — Quatre colonnes à classer (section 5.1.2 du livre)	82
Exercice 5.2 ★ — Un consentement contradictoire (section 5.1.4 du livre)	82
Exercice 5.3 ★★ — Combien de lignes manque-t-on ? (section 5.1.5 du livre)	82
Exercice 5.4 ★ — Le dictionnaire d'un numéro de téléphone (section 5.2.2 du livre)	82
Exercice 5.5 ★★ — Un sel par ligne (section 5.2.3 du livre)	82
Exercice 5.6 ★★ — Rééchantillonner des lignes n'est pas anonymiser (section 5.2.4 du livre)	82
Exercice 5.7 ★ — Le k-anonymat à la main (section 5.3.2 du livre)	82
Exercice 5.8 ★★ — Choisir la largeur de la tranche (section 5.3.3 du livre)	83
Exercice 5.9 ★★★ — Rendre un fichier de commandes inattaquable (section 5.3.5 du livre)	83
Exercice 5.10 ★★ — Mesurer la diversité (section 5.3.6 du livre)	83
Exercice 5.11 ★★ — Masquer sans trahir (section 5.4.2 du livre)	83
Exercice 5.12 ★★★ — La note de transmission (section 5.4.5 du livre)	83
Corrigés	83
Corrigé 5.1	83
Corrigé 5.2	84
Corrigé 5.3	84
Corrigé 5.4	84
Corrigé 5.5	85
Corrigé 5.6	85
Corrigé 5.7	85
Corrigé 5.8	86
Corrigé 5.9	86
Corrigé 5.10	87
Corrigé 5.11	87
Corrigé 5.12	88
Pistes des applications	88
Projet du volume et auto-évaluation — exercices et applications	90
Projet du volume	90
P.1 Le cahier des charges	90
P.2 Étape 1 : l'inventaire des sources	91
P.3 Étape 2 : lire la caisse, douze fichiers, trois formats	92
P.4 Étape 3 : nettoyer la caisse	93
P.5 Étape 4 : nettoyer l'export du site	94
P.6 Étape 5 : réconcilier avec la base	95
P.7 Étape 6 : assembler la table unique	95
P.8 Étape 7 : les contrôles	96
P.9 Étape 8 : le journal et le dictionnaire	97
P.10 Étape 9 : juger, puis ouvrir la correction	98
P.11 Les limites de l'étude	99
P.12 Variante : dédoublonner le CRM	99
Auto-évaluation	100
Nettoyage des données (chapitre 1)	100
Transformation et fusion (chapitre 2)	100
Qualité et réconciliation (chapitre 3)	101
Documentation (chapitre 4)	101
Confidentialité (chapitre 5)	101
Corrigés des questions	102
Votre grille d'auto-évaluation	105

Mode d'emploi

« On nettoie en regardant, on valide en recoupant. »

Ce cahier est le **compagnon du livre** du volume II (*Préparation des données*). Le livre explique les idées ; le cahier les fait travailler. Il contient, chapitre par chapitre, des **applications guidées**, des **exercices** et leurs **corrigés**, puis le **projet du volume** (nettoyer et réconcilier deux sources désordonnées en un jeu de données fiable) et l'auto-évaluation.

Comment utiliser ce cahier

1. **Lisez d'abord la section du livre.** Chaque section qui a un prolongement ici se termine par une ligne de ce genre : > ■ **Pour s'entraîner.** Cahier, chapitre 1 : application 1.1, exercices 1.1 à 1.4.
2. **Regardez les données avant de les corriger.** Ouvrez le fichier, comptez les lignes, listez les valeurs distinctes d'une colonne, regardez les premières et les dernières lignes. Un tiers des erreurs se voit à cette étape.
3. **Essayez avant de regarder le corrigé.** Les énoncés sont regroupés dans la partie *Exercices*, les corrigés dans la partie *Corrigés* du même chapitre.
4. **Écrivez ce que vous décidez.** Pour chaque règle de nettoyage, notez ce qu'elle fait, pourquoi, et **combien de lignes** elle touche. C'est l'habitude centrale du volume.
5. **Vérifiez par un second chemin.** Un total obtenu par pandas se recoupe par SQL ou par une autre source ; un écart est un signal, pas un détail.
6. **N'utilisez pas les fichiers *verite_** pour décider.** Ce sont des corrigés : on les ouvre après avoir fait son travail, pour mesurer ce qu'on a bien et mal fait.

Numérotation et niveaux

Chaque chapitre du cahier correspond au chapitre du livre de même numéro.

Élément	Numérotation	Exemple
Application	Application N.k	Application 2.1 : première application du chapitre 2
Exercice	Exercice N.k	Exercice 2.3 : troisième exercice du chapitre 2
Corrigé	Corrigé N.k	Corrigé 2.3 : correction de l'exercice 2.3

La difficulté des exercices est indiquée par des étoiles :

Niveau	Signification
★	application directe d'une idée ou d'un geste vu dans la section
★★	demande de combiner deux idées, ou un petit raisonnement
★★★	demande de la réflexion, un choix de règle ou une petite expérience

Un exercice cite la section du livre qu'il exerce, par exemple « (section 1.2) ».

Données et environnement

Les données sont celles du livre, dans le dossier `donnees/`. Elles sont toutes **simulées**, avec des graines fixes (script `build/donnees_a2.py`) : vos résultats seront identiques à ceux du livre. Le catalogue complet figure dans la section « Carte du volume, données et environnement » du livre ; en voici l'essentiel.

Fichier	Contenu	Chapitres
<code>crm_clients.csv</code>	le fichier clients : doublons, formats mêlés, lignes de test	1, 2, 3, 5, projet
<code>site_commandes.csv</code> , <code>site_lignes.csv</code>	export de la plateforme web (Site, 2025)	1, 2, 3, projet
<code>caisse/caisse_2025-01.csv ...</code> <code>-12.csv</code>	exports mensuels de la caisse, trois réglages différents	1, 2, 3, projet
<code>catalogue_fournisseur.csv</code>	catalogue d'un fournisseur	2, 3
<code>stocks_tableur.xlsx</code>	stocks saisis dans un tableur	2
<code>profil_clients.csv</code>	profils avec des valeurs manquantes	1
<code>montants_saisis.csv</code>	montants avec anomalies de saisie	1, 3
<code>clients.csv</code> , <code>produits.csv</code> , <code>commandes.csv</code> , <code>lignes_commande.csv</code>	la base propre du volume I	tous
<code>verite_*.csv</code> , <code>profil_clients_verite.csv</code>	corrigés de chaque source	à la fin

Les chiffres sont fictifs. La boutique, ses clients et leurs noms sont inventés. Ne tirez de ces données aucune conclusion sur le monde réel : elles servent à apprendre une méthode.

Chaque chapitre du cahier est **autonome** : il commence par ses imports et recharge ses données. Les applications sont dimensionnées pour s'exécuter en **quelques secondes** sur un ordinateur ordinaire, sans réseau.

Lancer le code

Placez-vous dans le dossier du volume (celui qui contient `donnees/`), car les chemins sont relatifs (`donnees/crm_clients.csv`). Vous pouvez travailler dans un notebook Jupyter, dans un éditeur avec la commande `python`, dans RStudio pour R, ou dans un outil de bases de données pour SQL.

Les chemins. Si Python répond `FileNotFoundError`, c'est presque toujours que vous n'êtes pas dans le bon dossier : vérifiez avec `import os; print(os.getcwd())`.

L'encodage. Les fichiers de caisse de janvier à juin sont en `cp1252` (Windows), les suivants en `UTF-8`. Si vous lisez un fichier avec le mauvais encodage, vous obtenez une erreur (`UnicodeDecodeError`) ou des caractères étranges (`NÂ°`, `Ã©`) : c'est un exercice du chapitre 1, pas une panne de votre ordinateur.

Excel. Les exercices qui demandent un tableur se font dans Excel ou dans LibreOffice Calc (gratuit), qui lit les mêmes fichiers ; quand un menu ou une fonction diffère, la documentation de votre version fait foi.

Vérifier son installation

Quatre courts blocs vérifient que les bibliothèques sont installées et que les données se chargent, en Python, en SQL puis en R.

1. Les bibliothèques Python (le volume utilise en plus celles du volume I) :

```
import sys
from importlib import import_module
from importlib.metadata import version
print("python", sys.version.split()[0])
for p in ["numpy", "pandas", "openpyxl", "rapidfuzz", "jellyfish", "unidecode", "ftfy", "pandera"]:
    import_module(p)
    print(f"{p:12s}", version(p))
```

```
python 3.13.3
numpy      2.5.3
pandas     3.0.6
openpyxl   3.1.5
rapidfuzz  3.14.6
jellyfish  1.2.1
unidecode  1.4.0
ftfy       6.3.1
pandera    0.34.1
```

2. Les données. Nous chargeons le CRM (tout en texte, pour ne rien déformer) et comptons les fichiers de caisse :

```
import glob
import pandas as pd

crm = pd.read_csv("donnees/crm_clients.csv", dtype=str)
print("CRM :", len(crm), "lignes,", crm.shape[1], "colonnes")
print("fichiers de caisse :", len(glob.glob("donnees/caisse/caisse_2025-*.csv")))
```

CRM : 7140 lignes, 11 colonnes
fichiers de caisse : 12

3. Une requête SQL. Nous copions le CRM dans une base SQLite en mémoire, puis interrogeons-la :

```
import sqlite3
con = sqlite3.connect(":memory:")
crm.to_sql("crm", con, index=False)
```

```
SELECT source_saisie, COUNT(*) AS lignes
FROM crm
GROUP BY source_saisie
ORDER BY lignes DESC;
```

source_saisie	lignes
caisse	3507
site	2965
import	668

4. Le même comptage en R, pour vérifier que R et le tidyverse sont là :

```
suppressPackageStartupMessages({library(readr); library(dplyr)})
crm <- read_csv(file.path(Sys.getenv("DONNEES"), "crm_clients.csv"), col_types = cols(.default =
  ↪ col_character()))
print(count(crm, source_saisie, sort = TRUE))
```

```
# A tibble: 3 × 2
  source_saisie      n
  <chr>          <int>
1 caisse         3507
2 site           2965
3 import          668
```

Vous devez lire une version pour chaque bibliothèque, **7 140 lignes** de CRM pour **11 colonnes**, **12 fichiers de caisse**, et **le même tableau** par source de saisie en SQL et en R. Si tout concorde, votre installation est prête.

Mini-diagnostic de départ

Huit questions pour vérifier que les réflexes de base de la préparation des données sont en place. Répondez par écrit, puis comparez avec les corrigés plus bas ; chaque corrigé indique le chapitre à lire en cas d'hésitation. Il n'y a pas de note : l'objectif est de savoir **où revenir** avant de commencer.

- Voici un extrait de six lignes d'un fichier clients (tout est inventé). Listez **tous** les problèmes de qualité que vous y voyez.

id	prenom	nom	email	ville	code_postal	naissance
1	Mirela	Dorvane	mirela.dorvane@exemple.org	Ville A	01234	12/03/1985
2	MIRELA	DORVANE	MIRELA.DORVANE@EXEMPLE.ORG	ville a	1234	1985-03-12
3	Tavin	Kelmar	tavin.kelmar@courrier.test	Ville B	05100	31/02/1990
4	Tavin	Kelmar	NaN	Vile B		NaN 03/04/1990
5	Test	TEST	test@example.com	Ville A	01000	01/01/2000
6	Solena	Barvert	solena.barvert@mail.example	ج المدينة	12345	5 mai 1971

- Dans `profil_clients.csv`, la colonne `depense_2025` contient des valeurs manquantes. Combien y en a-t-il, combien de clients ont une dépense égale à **zéro**, et que peut-on déduire de la colonne `nb_commandes_2025` pour les manquants ?

3. Dans le CRM, combien de lignes sont **exactement** identiques (toutes colonnes sauf `id_crm`), parmi les lignes qui ne sont pas des lignes de test ? Si vous normalisez l'adresse e-mail (espaces, majuscules), combien de lignes partagent leur adresse avec une autre ?
4. On joint les 120 produits de la boutique au catalogue du fournisseur en comparant le nom en minuscules. Combien de lignes attendez-vous ? Combien obtenez-vous, et pourquoi ?
5. Un fichier contient la date 03/04/2025. S'agit-il du 3 avril ou du 4 mars ? Comment trancher, et que montrent les dates de naissance du CRM écrites `xx/xx/aaaa` ?
6. Dans `montants_saisis.csv`, combien de lignes ont un montant qui diffère de « quantité × prix unitaire » ? Que faut-il savoir avant de les déclarer fausses ?
7. Le fichier `caisse/caisse_2025-01.csv` provoque une `UnicodeDecodeError` quand on le lit avec les réglages par défaut de Python (UTF-8). Que se passe-t-il, et comment lire correctement le fichier de janvier puis celui de juillet ?
8. Dans l'export du site, le total d'une commande vaut en médiane 82,93 avant le 15 septembre et 7 808 après. Que s'est-il passé et comment le vérifier ?

Corrigés du mini-diagnostic

1. Les problèmes visibles : (a) les lignes 1 et 2 sont la **même personne** (casse du nom et de l'adresse, ville « ville a », code postal amputé de son zéro, date écrite dans deux formats), donc un **doublon flou** ; (b) les lignes 3 et 4 sont probablement la même personne aussi (même nom, même ville à une faute près), avec une **adresse manquante**, un **code postal manquant** et deux dates **différentes** ; (c) la date 31/02/1990 est **impossible** ; (d) 03/04/1990 est **ambiguë** (3 avril ou 4 mars ?) ; (e) la ligne 5 est une **ligne de test** à retirer ; (f) la ligne 6 écrit la ville en **arabe** et la date en **texte** (« 5 mai 1971 »), deux formats à unifier ; (g) la ville « Vile B » est une **faute de frappe**. Aucune de ces corrections n'est difficile ; la difficulté est de les **repérer toutes** et de **décider** sans les propager aux autres lignes. À relire : sections 1.3 et 1.4.

```
dates = pd.to_datetime(pd.Series(["31/02/1990", "03/04/1990", "12/03/1985"]), format="%d/%m/%Y",
↳ errors="coerce")
print(dates.dt.strftime("%Y-%m-%d").fillna("impossible").tolist())
```

```
['impossible', '1990-04-03', '1985-03-12']
```

2. Les valeurs manquantes se comptent avec `isna`, les vrais zéros avec une comparaison :

```
prof = pd.read_csv("donnees/profil_clients.csv")
manque = prof["depense_2025"].isna()
print("manquants :", int(manque.sum()), "| zéros :", int((prof["depense_2025"] == 0).sum()))
print("manquants avec 0 commande :", int((manque & (prof["nb_commandes_2025"] == 0)).sum()), "| avec
↳ au moins 1 commande :", int((manque & (prof["nb_commandes_2025"] > 0)).sum()))
print("clients à dépense nulle ayant commandé :", int(((prof["depense_2025"] == 0) &
↳ (prof["nb_commandes_2025"] > 0)).sum()))
```

```
manquants : 297 | zéros : 2024
manquants avec 0 commande : 101 | avec au moins 1 commande : 196
clients à dépense nulle ayant commandé : 0
```

Il y a **297 manquants** et **2 024 zéros**. Un manquant n'est pas un zéro : le zéro est une **information** (« n'a rien dépensé »), le manquant est une **absence** d'information. La colonne `nb_commandes_2025` permet de **reconstituer** une partie des manquants : 101 des 297 clients n'ont passé aucune commande, leur dépense est donc **forcément nulle** ; pour les 196 autres (au moins une commande), elle est strictement positive mais **inconnue**. Aucun client qui a commandé n'a une dépense nulle : une dépense de zéro avec des commandes serait une incohérence. À relire : section 1.1.

3. Parmi les lignes réelles, il n'y a **aucune** ligne exactement identique : une recherche de doublons exacts ne trouve rien, alors que le fichier en contient bel et bien. Avec une adresse normalisée, c'est différent :

```
crm = pd.read_csv("donnees/crm_clients.csv", dtype=str)
reel = crm[crm["nom"] != "TEST"]
print("lignes identiques (hors id_crm, lignes de test exclues) :",
      ↪ int(reel.drop(columns="id_crm").duplicated().sum()))
mail = reel["email"].str.strip().str.lower().dropna()
partage = mail.value_counts()
print("adresses partagées par plusieurs lignes :", int((partage > 1).sum()), "| lignes concernées :",
      ↪ int(partage[partage > 1].sum()))
```

```
lignes identiques (hors id_crm, lignes de test exclues) : 0
adresses partagées par plusieurs lignes : 656 | lignes concernées : 1333
```

Après normalisation, **656 adresses** sont partagées par **1 333 lignes**. Le doublon exact ne trouve rien parce que les doublons sont **flous** (casse, accents, fautes) : il faut **normaliser** avant de comparer, puis comparer des textes qui se ressemblent (chapitre 2, section 2.5). Et l'adresse seule ne suffit pas : certaines lignes en double n'ont pas d'adresse. À relire : section 1.3.

4. On attend **120 lignes** (une par produit, en joignant « un à un »). On en obtient davantage :

```
produits = pd.read_csv("donnees/produits.csv")
cat = pd.read_csv("donnees/catalogue_fournisseur.csv")
produits["cle"] = produits["nom_produit"].str.lower()
cat["cle"] = cat["designation"].str.strip().str.lower()
j = produits.merge(cat, on="cle", how="left")
print(len(produits), "produits ->", len(j), "lignes après jointure |",
      ↪ int(j["code_fournisseur"].isna().sum()), "produits sans correspondance")
```

```
120 produits -> 160 lignes après jointure | 12 produits sans correspondance
```

On obtient **160 lignes** : la jointure **multiplie** les lignes parce que le **nom n'est pas une clé**. La boutique compte 120 produits mais seulement 60 noms distincts (chaque nom est porté par deux produits à prix différents) : un nom qui apparaît deux fois à gauche et deux fois à droite produit quatre lignes. Douze produits restent sans correspondance (absents du catalogue, ou désignation trop réécrite). La parade : vérifier l'unicité de la clé **avant** de joindre, ajouter un second critère (le prix), ou rapprocher autrement. À relire : sections 2.2 et 2.5.

5. Rien dans 03/04/2025 ne permet de trancher : c'est le **3 avril** en ordre jour-mois, le **4 mars** en ordre mois-jour. On tranche en regardant les **autres valeurs de la même colonne** : si l'on trouve 25/04/2025, le premier nombre est un jour ; si l'on trouve 04/25/2025, c'est un mois. À défaut, on demande à la source, ou on lit la documentation du logiciel. Dans le CRM :

```
x = crm.loc[crm["date_naissance"].str.fullmatch(r"\d{2}/\d{2}/\d{4}"), "date_naissance"]
a, b = x.str[:2].astype(int), x.str[3:5].astype(int)
print("dates xx/xx/aaaa :", len(x), "| premier nombre > 12 (jour sûr) :", int((a > 12).sum()), "|
      ↪ second nombre > 12 (mois en premier) :", int((b > 12).sum()), "| ambiguës :", int(((a <= 12) & (b
      ↪ <= 12)).sum()))
```

```
dates xx/xx/aaaa : 4674 | premier nombre > 12 (jour sûr) : 2083 | second nombre > 12 (mois en premier)
      ↪ : 634 | ambiguës : 1969
```

La colonne mélange **les deux ordres** : 2 083 dates ont un premier nombre supérieur à 12 (ordre jour-mois), 634 ont un second nombre supérieur à 12 (ordre mois-jour), et **1 969 sont ambiguës**. On n'applique donc pas un seul format à toute la colonne ; on se sert de la source de saisie, et l'on accepte de laisser certaines dates **non résolues** plutôt que de les inventer. À relire : section 1.4.

6. On compare le montant à la quantité multipliée par le prix :

```

m = pd.read_csv("donnees/montants_saisis.csv")
l = pd.read_csv("donnees/lignes_commande.csv", usecols=["id_ligne",
↳ "remise_pct"]).set_index("id_ligne")
m["remise"] = m["id_ligne"].map(l["remise_pct"])
sans = (m["montant"] - m["quantite"] * m["prix_unitaire"]).abs() >= 0.01
avec = (m["montant"] - (m["quantite"] * m["prix_unitaire"] * (1 - m["remise"] / 100)).round(2)).abs()
↳ >= 0.01
print("écarts sans tenir compte de la remise :", int(sans.sum()), "| en tenant compte de la remise :",
↳ int(avec.sum()))
print("lignes remises :", int((m["remise"] > 0).sum()), "| valeurs 9999 :", int((m["montant"] ==
↳ 9999).sum()), "| négatives :", int((m["montant"] < 0).sum()), "| nulles :", int((m["montant"] ==
↳ 0).sum()))

```

```

écarts sans tenir compte de la remise : 1103 | en tenant compte de la remise : 85
lignes remises : 1030 | valeurs 9999 : 9 | négatives : 22 | nulles : 10

```

Sans précaution, **1 103** lignes diffèrent ; mais **1 030** lignes ont une remise, qui explique la plupart des écarts. En tenant compte de la remise, il n'en reste que **85**, qui sont les vraies anomalies (décimale décalée, signe inversé, zéro, valeur 9999). Avant de déclarer une valeur fausse, il faut donc connaître la **règle** qui relie les colonnes. On recoupe à l'intérieur du fichier. À relire : section 1.2.

7. Python lit par défaut en UTF-8, or le fichier de janvier est encodé en cp1252 : un octet comme é n'y a pas la même forme et la lecture échoue. En forçant cp1252, on lit correctement. Le fichier de juillet est en UTF-8 avec **marque BOM** : lu avec le mauvais encodage, il donne des symboles étranges au début de la première ligne ; lu en utf-8-sig, il est propre.

```

brut = open("donnees/caisse/caisse_2025-01.csv", "rb").read()
try:
    brut.decode("utf-8")
except UnicodeDecodeError as e:
    print(type(e).__name__, "-", str(e)[:48])
print(repr(brut.decode("cp1252").splitlines()[3][:30]))
juillet = open("donnees/caisse/caisse_2025-07.csv", "rb").read()
print(repr(juillet.decode("cp1252").splitlines()[0][:34]), "|",
↳ repr(juillet.decode("utf-8-sig").splitlines()[0][:34]))

```

```

UnicodeDecodeError - 'utf-8' codec can't decode byte 0xe9 in position
'N° ticket;Date;Heure;Article;C'
'ï»¿Export caisse - Boutique;;;;;;;;; | 'Export caisse - Boutique;;;;;;;;;'

```

La règle : **ne jamais deviner**. On essaie UTF-8, on regarde le résultat, et l'on traite chaque fichier selon son encodage (section 1.5). Un bon réflexe est de **noter l'encodage** de chaque source dans la documentation (chapitre 4). À relire : sections 1.4 et 1.5.

8. À partir du 15 septembre, la plateforme exprime le total **en centimes** (une commande de 78 € est écrite 7800). La médiane est multipliée par près de cent. On le vérifie en regardant la **distribution par période** et en la comparant à une source indépendante :

```

site = pd.read_csv("donnees/site_commandes.csv", dtype=str)
site["montant"] = site["total"].str.replace("€", "").str.replace(" ", "").str.replace(",",
↳ ".").astype(float)
site["date"] = pd.to_datetime(site["created_at"].str.replace("T", " ").str.replace("Z", ""))
print("médiane avant le 15 septembre :", float(site.loc[site["date"] < "2025-09-15",
↳ "montant"].median()), "| après :", float(site.loc[site["date"] >= "2025-09-15",
↳ "montant"].median()))

```

```

médiane avant le 15 septembre : 82.93 | après : 7808.0

```

Le contrôle qui l'aurait révélé : **comparer un total à un ordre de grandeur connu** (le chiffre d'affaires d'un mois ne peut pas être cent fois celui du précédent), ou au total d'une autre source

(la base de commandes). La correction (diviser par 100 après la date de bascule) est une **décision** à noter dans le journal. À relire : sections 1.4 et 3.2.

À retenir. Si vous avez repéré la plupart des défauts des questions 1 à 3, votre œil de nettoyeur est en place ; les questions 4 à 6 testent des réflexes de **vérification** (unicité d'une clé, ambiguïté d'un format, règle interne qui relie les colonnes) ; les questions 7 et 8 testent la **méthode** (ne jamais deviner un encodage, contrôler un ordre de grandeur). Le chapitre 1 détaille les cinq premiers, les chapitres 2 et 3 les trois autres.

Chapitre 1 : Nettoyage des données

— exercices et applications

Ce chapitre du cahier prolonge le chapitre 1 du livre. Les **applications** sont de petites études guidées sur les fichiers désordonnés de la boutique, à refaire pas à pas ; les **exercices** (★ facile, ★★ moyen, ★★★ plus long) vous demandent de produire vous-même un résultat, avec un corrigé détaillé en fin de chapitre. Les fichiers `verite_*` ne servent qu'à **juger** un nettoyage : ouvrez-les à la fin, comme un corrigé. Tout le code se rejoue d'un trait, dans l'ordre.

```
import os, io, re, sys, glob, tempfile
import numpy as np, pandas as pd
sys.path.insert(0, "build")
import outils_ch01 as C
D = os.environ["DONNEES"]
profil = pd.read_csv(os.path.join(D, "profil_clients.csv"))
verite = pd.read_csv(os.path.join(D, "profil_clients_verite.csv"))
```

Applications

Application 1.1 — Profiler un fichier avant tout calcul (sections 1.1.1 et 1.1.2)

Objectif. Le premier geste d'une analyste est un **profil** du fichier : combien de lignes, quels types, combien de manquants, quelles valeurs spéciales. On le fait **avant** de calculer quoi que ce soit.

Étape 1 — Un profil par colonne. Une petite fonction qui résume chaque colonne : type, manquants, valeurs distinctes, minimum et maximum.

```
def profiler(t):
    p = pd.DataFrame({"type": t.dtypes.astype(str), "manquants": t.isna().sum(), "part_%":
    ↪ (t.isna().mean() * 100).round(1), "distinctes": t.nunique()})
    p["min"] = [t[c].min() if pd.api.types.is_numeric_dtype(t[c]) else "" for c in t.columns]
    p["max"] = [t[c].max() if pd.api.types.is_numeric_dtype(t[c]) else "" for c in t.columns]
    return p
print(profiler(profil).to_string())
```

	type	manquants	part_%	distinctes	min	max
id_client	int64	0	0.0	6000	1	6000
age	int64	0	0.0	68	18	85
canal_acquisition	str	0	0.0	3		
revenu_annuel	float64	1039	17.3	555	6900.0	104400.0
nb_commandes_2025	int64	0	0.0	24	0	25
depense_2025	float64	297	5.0	2838	0.0	3382.33
satisfaction_moy	float64	562	9.4	312	1.0	5.0
minutes_site	float64	0	0.0	533	0.2	86.4

Lecture. Trois colonnes ont des manquants (revenu, dépense, satisfaction). Les colonnes sans trou ont tout de même des **valeurs à regarder** : le minimum de age, le maximum de minutes_site, les zéros éventuels.

Étape 2 — Les valeurs spéciales d'une colonne de montants. Le fichier montants_saisis.csv contient des montants saisis à la main. On cherche les valeurs qui **ressemblent** à des codes plutôt qu'à des montants : zéros, négatifs, 9999.

```
saisis = pd.read_csv(os.path.join(D, "montants_saisis.csv"))
print("montants nuls :", int((saisis["montant"] == 0).sum()), "| négatifs :", int((saisis["montant"]
↪ < 0).sum()), "| égaux à 9999 :", int((saisis["montant"] == 9999).sum()))
print("montants au-dessus de 1 000 € :", int((saisis["montant"] > 1000).sum()))
```

```
montants nuls : 10 | négatifs : 22 | égaux à 9999 : 9
montants au-dessus de 1 000 € : 28
```

Lecture. Les zéros, les négatifs et les « 9999 » ne sont pas des montants plausibles pour une boutique dont la ligne moyenne vaut une quarantaine d'euros : ce sont des **suspects** à examiner (section 1.2), non des manquants. On les **liste**, on ne les remplace pas encore.

Étape 3 — Ce qui se déduit. Les clients sans commande ont forcément dépensé 0 €.

```
sans_commande = profil["nb_commandes_2025"] == 0
deduite = profil["depense_2025"].where(~sans_commande, 0.0)
print("dépenses manquantes avant :", int(profil["depense_2025"].isna().sum()), "| après déduction :",
↪ int(deduite.isna().sum()))
```

```
dépenses manquantes avant : 297 | après déduction : 196
```

À vous. Appliquez profiler au fichier crm_clients.csv lu **sans** dtype=str, puis avec dtype=str, et comparez les types obtenus (exercice 1.1).

Application 1.2 — Tester le mécanisme d'absence (section 1.1.3)

Objectif. Pour chaque colonne avec des trous, déterminer si l'absence dépend de **ce que l'on connaît** (MAR) ou semble aléatoire.

Étape 1 — Un test du khi-deux par colonne et par variable.

```
from scipy.stats import chi2_contingency
profil["classe_age"] = pd.cut(profil["age"], [0, 29, 44, 59, 200], labels=["moins de 30", "30-44",
↪ "45-59", "60 et plus"])
profil["classe_commandes"] = pd.cut(profil["nb_commandes_2025"], [-1, 0, 3, 10, 1000], labels=["0",
↪ "1 à 3", "4 à 10", "plus de 10"])
for col in ["depense_2025", "revenu_annuel", "satisfaction_moy"]:
    ps = {v: chi2_contingency(pd.crosstab(profil[v], profil[col].isna()))[1] for v in ["classe_age",
↪ "canal_acquisition", "classe_commandes"]}
    print(f"{col:17s}", {k: float(f"{p:.2g}") for k, p in ps.items()})
```

```
depense_2025      {'classe_age': 0.72, 'canal_acquisition': 0.53, 'classe_commandes': 0.54}
revenu_annuel     {'classe_age': 7.2e-49, 'canal_acquisition': 1.4e-09, 'classe_commandes': 0.13}
satisfaction_moy  {'classe_age': 0.76, 'canal_acquisition': 0.36, 'classe_commandes': 0.28}
```

Lecture. Le revenu dépend de l'âge et du canal (probabilités critiques minuscules). La **dépense** ne dépend ni de l'âge, ni du canal, ni du nombre de commandes (0,72 ; 0,53 ; 0,54) : l'absence est compatible avec le hasard, y compris chez les clients sans commande (101 sur 2 125, soit 4,8 %, contre 5,1 % ailleurs), et c'est pourquoi 101 des 297 trous sont **déductibles** sans rien changer au mécanisme. La **satisfaction** ne dépend de rien de visible.

Étape 2 — Prédire l'absence. Une autre façon de tester MAR : essayer de **prédire** l'absence à partir des colonnes connues. Une aire sous la courbe ROC (la probabilité qu'un client dont la valeur manque reçoive un score plus élevé qu'un client dont elle est connue) proche de 0,5 signifie « on ne fait pas mieux que le hasard ».

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
X = pd.get_dummies(profil[["age", "canal_acquisition", "nb_commandes_2025", "minutes_site"]],
↳ drop_first=True).astype(float)
for col in ["revenu_annuel", "depense_2025", "satisfaction_moy"]:
    y = profil[col].isna().astype(int)
    p = LogisticRegression(max_iter=2000).fit((X - X.mean()) / X.std(), y).predict_proba((X -
↳ X.mean()) / X.std())[:, 1]
    print(f"{col:17s} aire sous la courbe ROC pour prédire l'absence : {roc_auc_score(y, p):.3f}")
```

```
revenu_annuel      aire sous la courbe ROC pour prédire l'absence : 0.618
depense_2025       aire sous la courbe ROC pour prédire l'absence : 0.524
satisfaction_moy   aire sous la courbe ROC pour prédire l'absence : 0.521
```

Lecture. Une valeur de 0,62 pour le revenu, nettement supérieure à 0,5, confirme que l'absence est **prévisible** par l'âge et le canal (MAR). Les valeurs proches de 0,5 pour la dépense (0,52) et pour la satisfaction (0,52) ne prouvent **pas** que le mécanisme est aléatoire : pour la satisfaction, il dépend de la valeur manquante elle-même (MNAR), invisible ici.

Étape 3 — La vérité (réservée au cahier).

```
vraie = pd.cut(verite["satisfaction_moy"], [0, 2.5, 3.5, 4.5, 5.01], labels=["≤ 2,5", "2,5-3,5",
↳ "3,5-4,5", "> 4,5"])
print((profil["satisfaction_moy"].isna().groupby(vraie, observed=True).mean() *
↳ 100).round(1).to_string())
```

```
satisfaction_moy
≤ 2,5      37.6
2,5-3,5    7.5
3,5-4,5    8.9
> 4,5     7.1
```

À vous. Refaites le test pour `depense_2025` en ne gardant que les clients **ayant commandé** : l'absence y est-elle aléatoire ? (exercice 1.3)

Application 1.3 — Détecter les aberrantes : un compromis précision/rappel (sections 1.2.2 et 1.2.3)

Objectif. Voir comment le seuil d'une méthode statistique déplace l'équilibre entre « trop de fausses alertes » et « anomalies ratées », et comparer à la règle métier.

Étape 1 — Préparer.

```
saisis = saisis.merge(pd.read_csv(os.path.join(D, "verite_montants.csv")), on="id_ligne")
saisis["vraie_anomalie"] = saisis["anomalie"].notna()
m = saisis["montant"]
mad = (m - m.median()).abs().median()
score = (0.6745 * (m - m.median()) / mad).abs()
print("anomalies injectées :", int(saisis["vraie_anomalie"].sum()), "sur", len(saisis), "| MAD :",
↳ round(mad, 2))
```

```
anomalies injectées : 85 sur 6000 | MAD : 17.47
```

Étape 2 — Balayer le seuil du score z robuste.

```
for seuil in (2, 3.5, 5, 10, 20, 50):
    signal = score > seuil
    vrais = int((signal & saisis["vraie_anomalie"]).sum())
    print(f"seuil {seuil:>4} : signalées {int(signal.sum()):4d} | précision {vrais / signal.sum() *
        ↳ 100:5.1f} % | rappel {vrais / saisis['vraie_anomalie'].sum() * 100:5.1f} %")
```

```
seuil    2 : signalées 695 | précision  9.6 % | rappel  78.8 %
seuil   3.5 : signalées 324 | précision 17.0 % | rappel  64.7 %
seuil    5 : signalées 150 | précision 34.0 % | rappel  60.0 %
seuil   10 : signalées  68 | précision 67.6 % | rappel  54.1 %
seuil   20 : signalées  33 | précision 100.0 % | rappel  38.8 %
seuil   50 : signalées  26 | précision 100.0 % | rappel  30.6 %
```

Lecture. Plus le seuil monte, plus la **précision** augmente (on ne signale que des cas flagrants) et plus le **rappel** chute (on rate les anomalies discrètes). Aucun seuil ne donne à la fois une précision et un rappel proches de 100 %, parce que la distribution des montants légitimes a elle-même une longue queue.

Étape 3 — La règle métier.

```
rapport = saisis["montant"] / (saisis["quantite"] * saisis["prix_unitaire"])
suspect = ~rapport.between(0.795, 1.005)
vrais = int((suspect & saisis["vraie_anomalie"]).sum())
print("règle métier : signalées", int(suspect.sum()), "| précision", round(vrais / suspect.sum() *
    ↳ 100, 1), "% | rappel", round(vrais / saisis["vraie_anomalie"].sum() * 100, 1), "%")
```

```
règle métier : signalées 85 | précision 100.0 % | rappel 100.0 %
```

Étape 4 — L'effet sur le total.

```
corrige = np.where(suspect, saisis["quantite"] * saisis["prix_unitaire"], saisis["montant"])
vrai_total = saisis["montant_vrai"].sum()
print("total saisi :", round(saisis["montant"].sum()), "| corrigé par la règle :",
    ↳ round(corrige.sum()), "| vrai :", round(vrai_total), "| écart du total corrigé :",
    ↳ round((corrige.sum() / vrai_total - 1) * 100, 2), "%")
```

```
total saisi : 447950 | corrigé par la règle : 255711 | vrai : 255631 | écart du total corrigé : 0.03 %
```

À vous. Ajoutez une deuxième règle (montant strictement positif et quantité entre 1 et 10) et vérifiez ce qu'elle signale en plus ou en moins de la première.

Application 1.4 — Dédoublonner les commandes du site avec un journal (section 1.3.2)

Objectif. Construire une fonction qui **nettoie** l'export du site et garde la trace de ce qu'elle retire.

Étape 1 — Les trois raisons de retirer une ligne.

```
site = pd.read_csv(os.path.join(D, "site_commandes.csv"), dtype=str)
def nettoyer_site(t):
    journal = []
    sans_doublon = t.drop_duplicates("order_ref")
    journal.append(("doublon d'export", t[t.duplicated("order_ref")]))
    test = sans_doublon["customer_email"].str.strip().str.lower().eq("test@example.com")
    journal.append(("commande de test", sans_doublon[test]))
    reste = sans_doublon[~test]
    annulee = reste["status"].str.lower().eq("cancelled")
    journal.append(("commande annulée", reste[annulee]))
    return reste[~annulee], journal
propre, journal = nettoyer_site(site)
print("lignes avant :", len(site), "| après :", len(propre))
```

lignes avant : 6259 | après : 5897

Étape 2 — Le journal.

```
for raison, lignes in journal:
    print(f"{raison:18s} {len(lignes):4d} lignes")
print("somme :", sum(len(l) for _, l in journal), "= lignes retirées :", len(site) - len(propres))
```

```
doublon d'export    121 lignes
commande de test    60 lignes
commande annulée    181 lignes
somme : 362 = lignes retirées : 362
```

Lecture. La somme des lignes du journal est **égale** au nombre de lignes retirées : rien n'a disparu sans explication. C'est la propriété à exiger de tout nettoyage.

Étape 3 — Contrôles de bon sens. Deux lignes de même `order_ref` ont-elles toujours le même contenu ? Y a-t-il des commandes sans e-mail ?

```
doublons = site[site["order_ref"].duplicated(keep=False)]
print("références en double dont le contenu diffère :",
      ↪ int((doublons.groupby("order_ref").apply(lambda g: len(g.drop_duplicates()) > 1)).sum()))
print("commandes sans e-mail :", int(site["customer_email"].isna().sum()), "| statuts :",
      ↪ sorted(site["status"].str.lower().unique()))
```

```
références en double dont le contenu diffère : 0
commandes sans e-mail : 0 | statuts : ['cancelled', 'paid']
```

À vous. Ajoutez à la fonction une quatrième raison : une commande dont le total vaut 0 ou est négatif.

Application 1.5 — Détecter et corriger le changement d'unité (section 1.4.2)

Objectif. Trouver la date d'un changement d'unité **sans regarder le fichier à la main**, le corriger et le vérifier.

Étape 1 — Convertir les montants et dater les commandes.

```
def nombre(texte):
    return float(texte.replace("€", "").replace(" ", "").replace(",", ".").strip())
site["total_n"] = site["total"].map(nombre)
site["dt"] = pd.to_datetime(site["created_at"].str.replace("Z", "", regex=False), format="mixed")
quotidien = site.set_index("dt")["total_n"].resample("D").median()
print(quotidien.describe().round(0).to_string())
```

```
count    365.0
mean     2453.0
std       3793.0
min        25.0
25%       75.0
50%       98.0
75%      6314.0
max     12742.0
```

Étape 2 — Chercher la rupture : le rapport à la médiane des quatorze jours précédents.

```
precedent = quotidien.rolling(14, min_periods=7).median().shift(1)
rapport = quotidien / precedent
jour_rupture = rapport[rapport > 20].index[0]
print("premier jour où le montant médian dépasse 20 fois celui des 14 jours précédents :",
      ↪ jour_rupture.date())
```

```
print("rapport ce jour-là :", round(rapport[jour_rupture], 1), "| rapports de plus de 20 :",
      ↪ int((rapport > 20).sum()))
```

```
premier jour où le montant médian dépasse 20 fois celui des 14 jours précédents : 2025-09-15
rapport ce jour-là : 149.7 | rapports de plus de 20 : 7
```

Lecture. La rupture est détectée **par un calcul**, et non à l'œil : le 15 septembre, le montant médian est près de 150 fois celui des quatorze jours précédents (un facteur de l'ordre de 100, bruité par la variabilité d'une médiane quotidienne), ce qui désigne le passage de l'euro au centime. Sept jours dépassent le seuil de 20 : les premiers jours après la rupture, tant que la médiane glissante est encore dominée par les anciens montants en euros.

Étape 3 — Corriger et vérifier.

```
site["total_corrige"] = np.where(site["dt"] >= jour_rupture, site["total_n"] / 100, site["total_n"])
verite_site = pd.read_csv(os.path.join(D, "verite_site.csv")).drop_duplicates("order_ref")
x = site.merge(verite_site[verite_site["default"] != "test"], on="order_ref")
print("écart maximal avec la vérité :", round((x["total_corrige"] - x["total_vrai"]).abs().max(), 4),
      ↪ "€ sur", len(x), "commandes")
```

```
écart maximal avec la vérité : 0.0 € sur 6199 commandes
```

Étape 4 — Le chiffre d'affaires mensuel du site, avant et après.

```
site["mois"] = site["dt"].dt.strftime("%Y-%m")
propre = site.drop_duplicates("order_ref")
propre = propre[~propre["customer_email"].str.strip().str.lower().eq("test@example.com") &
               ↪ (propre["status"].str.lower() != "cancelled")]
print(propre.groupby("mois")[["total_n", "total_corrige"]].sum().round(0).astype(int).to_string())
```

mois	total_n	total_corrige
2025-01	40221	40221
2025-02	32746	32746
2025-03	40821	40821
2025-04	41828	41828
2025-05	47471	47471
2025-06	48555	48555
2025-07	52340	52340
2025-08	36278	36278
2025-09	3012754	54524
2025-10	5499972	55000
2025-11	6288428	62884
2025-12	8749607	87496

À vous. Que se passe-t-il si l'on corrige « à l'œil », en divisant par cent toutes les valeurs supérieures à 1 000 € ? (exercice 1.8)

Application 1.6 — Lire les douze fichiers de caisse et les réconcilier (sections 1.3.3 et 1.4.6)

Objectif. Lire les douze exports mensuels avec la fonction `lire_caisse`, puis **expliquer** mois par mois l'écart avec le total affiché.

Étape 1 — Lire et vérifier.

```
fichiers = sorted(glob.glob(os.path.join(D, "caisse", "*.csv")))
cais = pd.concat([C.lire_caisse(f)[0].assign(mois=os.path.basename(f)[7:14]) for f in fichiers],
                 ↪ ignore_index=True)
totaux = {os.path.basename(f)[7:14]: C.lire_caisse(f)[1] for f in fichiers}
print("lignes lues :", len(cais), "| montants vides :", int(cais["montant"].isna().sum()), "|
      ↪ colonnes :", list(cais.columns))
```

```
lignes lues : 12678 | montants vides : 399 | colonnes : ['ticket', 'date', 'heure', 'article',
↳ 'categorie', 'quantite', 'prix_unitaire', 'montant', 'id_commande', 'mois', 'remise_pct']
```

Étape 2 — Rapprocher chaque ligne d'une ligne de la base (avec un rang).

```
produits = pd.read_csv(os.path.join(D, "produits.csv"))[["id_produit", "nom_produit"]]
base = pd.read_csv(os.path.join(D, "lignes_commande.csv")).merge(pd.read_csv(os.path.join(D,
↳ "commandes.csv"))[["id_commande", "canal", "date_commande"]], on="id_commande").merge(produits,
↳ on="id_produit")
base = base[(base["canal"] == "Boutique") & (base["date_commande"] >= "2025-01-01")].copy()
base["article"], cais["article"] = base["nom_produit"].str.lower(), cais["article"].str.lower()
cle = ["id_commande", "article", "quantite", "prix_unitaire"]
for t in cais, base:
    t["rang"] = t.groupby(cle).cumcount()
r = cais.merge(base[cle + ["rang", "montant"]].rename(columns={"montant": "montant_base"}), on=cle +
↳ ["rang"], how="left", indicator=True)
print(r["_merge"].value_counts().to_string())
```

```
_merge
both          12611
left_only      67
right_only      0
```

Étape 3 — L'écart de chaque mois, expliqué.

```
r["double"] = r["_merge"] == "left_only"
r["vide_retrouve"] = np.where(r["montant"].isna() & ~r["double"], r["montant_base"], 0.0)
g = r.groupby("mois").apply(lambda t: pd.Series({"somme_lue": t["montant"].sum(), "doubles":
↳ t.loc[t["double"], "montant"].sum(), "vides_retrouvés": t["vide_retrouve"].sum()}))
g["reconstitué"] = g["somme_lue"] - g["doubles"] + g["vides_retrouvés"]
g["total_affiché"] = pd.Series(totaux)
g["écart_restant"] = (g["total_affiché"] - g["reconstitué"]).round(2)
print(g.round(2).to_string())
```

mois	somme_lue	doubles	vides_retrouvés	reconstitué	total_affiché	écart_restant
2025-01	37826.31	245.22	1301.32	38882.41	38882.41	0.0
2025-02	32273.46	0.00	805.95	33079.41	33079.41	0.0
2025-03	37863.67	135.26	1310.49	39038.90	39038.90	0.0
2025-04	45426.14	671.69	1078.12	45832.57	45832.57	0.0
2025-05	43381.31	291.72	1816.33	44905.92	44905.92	0.0
2025-06	42277.42	236.32	1077.06	43118.16	43118.16	0.0
2025-07	41035.68	144.25	704.39	41595.82	41595.82	0.0
2025-08	42350.83	451.37	974.04	42873.50	42873.50	0.0
2025-09	45600.67	321.44	1075.02	46354.25	46354.25	0.0
2025-10	49839.95	82.00	1563.53	51321.48	51321.48	0.0
2025-11	59096.64	243.54	1733.52	60586.62	60586.62	0.0
2025-12	70924.34	303.48	2764.01	73384.87	73384.87	0.0

Lecture. L'écart restant est nul pour chacun des douze mois : non seulement le total global, mais chaque fichier se reconstitue au centime. Une réconciliation fine, mois par mois, localise une erreur si elle survient (un mois dont l'écart restant ne serait pas nul).

À vous. Quel mois compte le plus de lignes en double ? Le plus de montants vides ?

Application 1.7 — Nettoyer le fichier clients du CRM (sections 1.4.4, 1.4.5 et 1.5)

Objectif. Assembler les corrections de la section 1.4 et de la section 1.5 en un **seul traitement**, qui produit un tableau propre et une **table de contrôle**.

Étape 1 — Lire en texte, retirer les lignes de test.

```
crm = pd.read_csv(os.path.join(D, "crm_clients.csv"), dtype=str)
test = crm["email"].str.strip().str.lower().eq("test@example.com")
crm = crm[~test].copy()
print("lignes :", len(test), "| lignes de test retirées :", int(test.sum()), "| restantes :",
      ↪ len(crm))
```

```
lignes : 7140 | lignes de test retirées : 140 | restantes : 7000
```

Étape 2 — Normaliser les colonnes.

```
def ville_propre(s):
    s = s.strip()
    return "Ville " + chr(65 + C.AR.index(s.split()[-1])) if s.startswith("المدينة") else
    ↪ C.normaliser_ville(s)
crm["ville_propre"] = crm["ville"].map(ville_propre)
crm["cp"] = crm["code_postal"].str.zfill(5)
crm["email_propre"] = crm["email"].str.strip().str.lower()
crm["tel"] = crm["telephone"].map(lambda s: (lambda ch: ch if re.fullmatch(r"0\d{9}", ch) else
    ↪ None)("0" + re.sub(r"\D", "", s)[2:]) if s.startswith("+") else re.sub(r"\D", "", s)))
crm["naissance"] = crm["date_naissance"].map(lambda s: C.date_mixte(s,
    ↪ americain=bool(re.fullmatch(r"\d\d/\d\d/\d{4}", s)) and int(s[3:5]) > 12))
crm["consentement"] = crm["consentement_marketing"].str.strip().str.lower().map({"oui": True, "o":
    ↪ True, "1": True, "true": True, "non": False})
print(crm[["ville_propre", "cp", "tel", "naissance", "consentement"]].isna().sum().to_string())
```

```
ville_propre      0
cp                424
tel               0
naissance         44
consentement     2161
```

Étape 3 — La table de contrôle.

```
regles = {"ville reconnue": crm["ville_propre"].str.fullmatch(r"Ville [A-T]"),
          "code postal de 5 chiffres (si renseigné)": crm["cp"].isna() |
          ↪ crm["cp"].str.fullmatch(r"\d{5}"),
          "e-mail valide (si renseigné)": crm["email_propre"].isna() |
          ↪ crm["email_propre"].str.fullmatch(r"(?!.*\.\.)([w.-]+@[w-]+\.[w.]+)",
          "naissance lisible": crm["naissance"].notna(),
          "naissance entre 1920 et 2010": crm["naissance"].isna() |
          ↪ crm["naissance"].between("1920-01-01", "2010-12-31")}
controle = pd.DataFrame({"infractions": {k: int(~v).sum() for k, v in regles.items()}})
controle["part_%"] = (controle["infractions"] / len(crm) * 100).round(2)
print(controle.to_string())
```

	infractions	part_%
ville reconnue	0	0.00
code postal de 5 chiffres (si renseigné)	0	0.00
e-mail valide (si renseigné)	95	1.36
naissance lisible	44	0.63
naissance entre 1920 et 2010	24	0.34

Étape 4 — Dédoublonner par e-mail, en gardant la ligne la plus complète.

```
avec_email = crm[crm["email_propre"].notna()].copy()
avec_email["manquants"] = avec_email[["tel", "cp", "naissance"]].isna().sum(axis=1)
fusion = avec_email.sort_values(["email_propre", "manquants"]).groupby("email_propre",
    ↪ as_index=False).first()
```

```
print("lignes avec e-mail :", len(avec_email), "| après fusion :", len(fusion), "| lignes sans e-mail  
↳ (gardées telles quelles) :", int(crm["email_propre"].isna().sum()))
```

```
lignes avec e-mail : 6769 | après fusion : 6092 | lignes sans e-mail (gardées telles quelles) : 231
```

À vous. Quel est le nombre de clients après fusion, en comptant les lignes sans e-mail comme autant de clients distincts ? Comparez avec les 6 000 clients de la vérité : que vous apprend l'écart ?

Application 1.8 — Comparer les imputations à la vérité, avec vos propres trous (section 1.6)

Objectif. Pour choisir une méthode d'imputation, on la teste sur des trous que l'on a soi-même faits dans des données complètes, en variant le mécanisme.

Étape 1 — Fabriquer trois mécanismes d'absence sur le revenu (complet dans la vérité).

```
rng = np.random.default_rng(1)
rev = verite["revenu_annuel"]
p_mar = np.where(verite["age"] < 30, 0.35, 0.12) # dépend de l'âge, connu
p_mnar = 0.08 + 0.35 * (rev > rev.quantile(0.75)) # dépend du revenu lui-même
masques = {"MCAR": rng.random(len(rev)) < 0.17, "MAR": rng.random(len(rev)) < p_mar, "MNAR":
↳ rng.random(len(rev)) < p_mnar}
print({k: round(float(v.mean()) * 100, 1) for k, v in masques.items()}, "% de manquants")
```

```
{'MCAR': 17.3, 'MAR': 15.7, 'MNAR': 17.6} % de manquants
```

Étape 2 — Trois méthodes d'imputation.

```
from sklearn.linear_model import LinearRegression
X = pd.get_dummies(verite[["age", "canal_acquisition", "nb_commandes_2025", "minutes_site"]],
↳ drop_first=True).astype(float)
classe = pd.cut(verite["age"], [0, 29, 44, 59, 200])
def imputer(masque):
    obs = rev.where(~masque)
    return {"moyenne": pd.Series(obs.mean(), index=rev.index),
            "moyenne par âge": obs.groupby(classe, observed=True).transform("mean"),
            "régression": pd.Series(LinearRegression().fit(X[~masque], rev[~masque]).predict(X),
↳ index=rev.index)}
```

Étape 3 — Biais de la moyenne et erreur, par mécanisme et par méthode.

```
lignes = []
for meca, masque in masques.items():
    for nom, imp in imputer(masque).items():
        complet = rev.where(~masque, imp)
        lignes.append((meca, nom, round(complet.mean() - rev.mean(), round(np.sqrt(((imp[masque] -
↳ rev[masque]) ** 2).mean()))))
print(pd.DataFrame(lignes, columns=["mécanisme", "méthode", "biais_moyenne_€",
↳ "erreur_typique_€"]).to_string(index=False))
```

mécanisme	méthode	biais_moyenne_€	erreur_typique_€
MCAR	moyenne	4	11397
MCAR	moyenne par âge	28	10843
MCAR	régression	48	10839
MAR	moyenne	251	10869
MAR	moyenne par âge	-33	9934
MAR	régression	-19	9851
MNAR	moyenne	-1749	15767
MNAR	moyenne par âge	-1574	14586
MNAR	régression	-1558	14460

Lecture. Sous **MCAR**, les trois méthodes retrouvent la moyenne. Sous **MAR**, la moyenne simple est biaisée alors que les méthodes qui utilisent l'âge corrigent le biais. Sous **MNAR** (les revenus élevés manquent plus souvent), **aucune** méthode ne retrouve la moyenne : toutes sous-estiment le revenu, parce que l'information sur ce qui manque a disparu avec les valeurs.

À vous. Répétez avec dix graines différentes et regardez la variabilité du biais : la conclusion dépend-elle du hasard d'une graine ?

Exercices

Exercice 1.1 ★ — Deux lectures d'un même fichier (section 1.1.1)

Lisez `crm_clients.csv` une première fois **sans** option, une seconde fois avec `dtype=str`. Pour la colonne `code_postal`, comparez le type obtenu et le nombre de valeurs manquantes. Que devient un code qui commence par 0 ?

Exercice 1.2 ★★ — Déduire avant d'imputer (section 1.1.2)

Dans `profil_clients.csv`, vérifiez la règle « dépense nulle si et seulement si aucune commande » sur les clients dont la dépense est **connue**. Combien de clients la violent ? Que concluez-vous sur la déduction des 101 dépenses manquantes ?

Exercice 1.3 ★★ — Le mécanisme de la dépense (section 1.1.3)

Parmi les clients **ayant passé au moins une commande**, la dépense manque-t-elle au hasard ? Testez sa dépendance à la classe d'âge et au canal d'acquisition avec un khi-deux, et calculez la part de manquants.

Exercice 1.4 ★ — Les seuils à la main (section 1.2.2)

Pour les neuf montants 20, 25, 30, 35, 40, 45, 50, 60 et 260 : calculez à la main les quartiles (méthode de la médiane des moitiés ou interpolation linéaire de pandas), l'écart interquartile et les seuils de la règle $1,5 \times \text{EIQ}$, puis le score z classique de 260 et son score z robuste. Que signale chaque méthode ?

Exercice 1.5 ★★ — Plafonner à 95 %, 99 %, 99,9 % (section 1.2.4)

Sur la dépense annuelle **vraie** (`profil_clients_verite.csv`), plafonnez les valeurs aux centiles 95, 99 et 99,9. Pour chaque plafond, donnez la moyenne, l'écart-type et la part du chiffre d'affaires qui est « coupée ».

Exercice 1.6 ★ — Doublons d'un petit tableau (section 1.3.1)

On donne le tableau de six lignes ci-dessous. Combien de doublons exacts ? Combien si l'on ne regarde que `ticket` et `article` ? Que fait `keep=False` ?

```
petit = pd.DataFrame({"ticket": ["T1", "T1", "T2", "T2", "T3", "T3"], "article": ["Vase", "Vase",
↪ "Plaid", "Bol", "Bol", "Bol"],
                      "quantite": [1, 1, 2, 1, 1, 2], "montant": [20.0, 20.0, 60.0, 15.0, 15.0, 30.0]})
```

Exercice 1.7 ★★ — Le plus récent ou le plus complet ? (section 1.3.4)

Sur le CRM (sans les lignes de test), regroupez les lignes par e-mail normalisé. Comparez deux règles de fusion : garder, pour chaque e-mail, la ligne à la date_inscription la plus **récente**, ou la ligne la plus **complète** (le moins de colonnes vides). Dans chaque cas, combien de codes postaux manquent après fusion ?

Exercice 1.8 ★★ — Corriger « à l'œil » (section 1.4.2)

Dans l'export du site, corrigez le changement d'unité en divisant par 100 **toutes** les valeurs supérieures à 1 000 € (et rien d'autre). Comparez le résultat à la vérité : combien de commandes sont mal corrigées, dans quel sens ? Pourquoi une correction **datée** est-elle meilleure ?

Exercice 1.9 ★★★ — Un treizième fichier (section 1.4.6)

Le logiciel de caisse envoie, en janvier 2026, un fichier au format encore différent : séparateur tabulation, dates AAAA-MM-JJ, colonnes dans un autre ordre. Voici son contenu (écrit dans un fichier temporaire par le code ci-dessous). C. lire_caisse ne sait pas le lire. Écrivez une version **plus générale** de la fonction, qui détecte le séparateur parmi ; , et tabulation, et le format de date, puis vérifiez le total affiché.

```
contenu = "Export caisse - Boutique\t\t\t\t\t\nPériode : janvier
↳ 2026\t\t\t\t\t\n\t\t\t\t\t\nTicket\tDate\tHeure\tArticle\tCatégorie\tQté\tPrix
↳ unitaire\tMontant\nT90001\t2026-01-02\t10:05\tPlaid
↳ nordique\tMaison\t1\t22.56\t22.56\nT90001\t2026-01-02\t10:05\tVase
↳ mat\tDécoration\t2\t14.90\t29.80\n\t\t\t\t\t\tTotal\t52.36\n"
dossier = tempfile.mkdtemp(dir=os.environ.get("TMPDIR"))
fichier13 = os.path.join(dossier, "caisse_2026-01.csv")
open(fichier13, "w", encoding="utf-8").write(contenu)
```

Exercice 1.10 ★★ — Normaliser des téléphones (section 1.5.3)

Appliquez une normalisation en dix chiffres commençant par 0 aux numéros ci-dessous (formats mélangés, dont un invalide). Quels sont ceux que l'on **rejette** et pourquoi ?

```
numeros = ["01 23 45 67 89", "01.23.45.67.89", "0123456789", "(0)1 23456789", "+99 1 23 45 67 89", "12
↳ 34 56", "01 23 45 67 8A"]
```

Exercice 1.11 ★★ — Médiane de groupe contre moyenne de groupe (section 1.6.2)

Sur le revenu manquant de profil_clients.csv, imputez par la **médiane** de chaque groupe (classe d'âge × canal) puis par la **moyenne** de chaque groupe. Comparez l'erreur individuelle, le biais de la moyenne et l'écart-type final à la vérité. Laquelle choisir, et pourquoi la différence est-elle faible ?

Exercice 1.12 ★★★ — Un MNAR que vous fabriquez, et un δ à choisir (section 1.6.6)

À partir de la satisfaction **vraie**, supprimez avec la probabilité 0,5 les valeurs inférieures à 3 (et 0,05 les autres). Calculez la moyenne observée et le biais. Cherchez, par un balayage, le décalage δ qui, soustrait aux valeurs imputées par la moyenne, retrouve la vraie moyenne. Dans la vraie vie, comment choisiriez-vous δ sans connaître la vérité ?

Corrigés

Corrigé 1.1

```
brut = pd.read_csv(os.path.join(D, "crm_clients.csv"))
texte = pd.read_csv(os.path.join(D, "crm_clients.csv"), dtype=str)
zero = texte["code_postal"].str.startswith("0", na=False)
print("sans option :", brut["code_postal"].dtype, "| manquants",
      ↳ int(brut["code_postal"].isna().sum()), "| avec dtype=str :", texte["code_postal"].dtype, "|
      ↳ manquants", int(texte["code_postal"].isna().sum()))
print("codes commençant par 0 lus sans option :", brut.loc[zero, "code_postal"].head(3).tolist(), "|
      ↳ en texte :", texte.loc[zero, "code_postal"].head(3).tolist())
```

```
sans option : float64 | manquants 424 | avec dtype=str : str | manquants 424
codes commençant par 0 lus sans option : [5723.0, 9321.0, 5723.0] | en texte : ['05723', '09321',
      ↳ '05723']
```

Sans option, pandas lit la colonne en **nombre décimal** (float64, à cause des manquants) : le code 05723 devient 5723.0, et le zéro initial est **perdu définitivement** (il faudrait le réinventer avec `zfill(5)`, en supposant que tous les codes ont cinq chiffres). Avec `dtype=str`, le code reste 05723. Le nombre de manquants est le même, mais la colonne n'a plus le même sens. **Règle** : les identifiants se lisent en texte.

Corrigé 1.2

```
connus = profil.dropna(subset=["depense_2025"])
print("clients avec dépense connue :", len(connus))
print("dépense nulle ET commandes > 0 :", int(((connus["depense_2025"] == 0) &
      ↳ (connus["nb_commandes_2025"] > 0)).sum()))
print("dépense > 0 ET aucune commande :", int(((connus["depense_2025"] > 0) &
      ↳ (connus["nb_commandes_2025"] == 0)).sum()))
```

```
clients avec dépense connue : 5703
dépense nulle ET commandes > 0 : 0
dépense > 0 ET aucune commande : 0
```

La règle n'a **aucune exception** parmi les clients dont la dépense est connue (les deux comptes de violations sont nuls). On peut donc appliquer la déduction aux 101 dépenses manquantes des clients sans commande, avec la confiance que donne une règle vérifiée sur plus de 5 700 cas. C'est plus sûr que n'importe quelle imputation statistique.

Corrigé 1.3

```
avec = profil[profil["nb_commandes_2025"] > 0].copy()
avec["classe_age"] = pd.cut(avec["age"], [0, 29, 44, 59, 200])
print("clients avec commandes :", len(avec), "| dépense manquante :",
      ↳ int(avec["depense_2025"].isna().sum()), f"({avec['depense_2025'].isna().mean() * 100:.1f} %)")
for v in ["classe_age", "canal_acquisition"]:
    print(v, "p =", round(chi2_contingency(pd.crosstab(avec[v], avec["depense_2025"].isna()))[1], 3))
```

```
clients avec commandes : 3875 | dépense manquante : 196 (5.1 %)
classe_age p = 0.974
canal_acquisition p = 0.579
```

Parmi les 3 875 clients qui ont commandé, la dépense manque pour 5,1 % d'entre eux, sans dépendance visible à l'âge ni au canal (probabilités critiques de 0,97 et 0,58) : c'est compatible avec

un mécanisme **MCAR**. C'est cohérent avec l'application 1.2 : ni l'âge, ni le canal, ni le nombre de commandes n'expliquent l'absence de la dépense.

Corrigé 1.4

```
v = pd.Series([20, 25, 30, 35, 40, 45, 50, 60, 260])
q1, q3 = v.quantile([0.25, 0.75])
print("Q1 =", q1, "| Q3 =", q3, "| EIQ =", q3 - q1, "| seuils :", q1 - 1.5 * (q3 - q1), "et", q3 + 1.5
      ↪ * (q3 - q1))
mad = (v - v.median()).abs().median()
print("z classique de 260 :", round((260 - v.mean()) / v.std(), 2), "| z robuste de 260 :",
      ↪ round(0.6745 * (260 - v.median()) / mad, 2), "| MAD =", mad)
```

```
Q1 = 30.0 | Q3 = 50.0 | EIQ = 20.0 | seuils : 0.0 et 80.0
z classique de 260 : 2.63 | z robuste de 260 : 14.84 | MAD = 10.0
```

À la main : les valeurs triées occupent les rangs 0 à 8 ; avec l'interpolation linéaire, Q_1 est au rang 2 (30) et Q_3 au rang 6 (50), donc l'EIQ vaut 20 et les seuils sont $30 - 30 = 0$ et $50 + 30 = 80$: **260 est signalée**. Le **score z classique** de 260 vaut environ 2,6 (la moyenne, 62,8, et l'écart-type, 75, sont tous deux tirés par 260 elle-même) : sous le seuil de 3, **260 n'est pas signalée**, c'est le masquage. Le **score z robuste** vaut 14,8 (la MAD vaut 10) : **260 est signalée**. Deux méthodes sur trois la repèrent ; sur un petit échantillon, le score z classique est le moins fiable.

Corrigé 1.5

```
dep = verite["depense_2025"]
print(f"vérité : moyenne {dep.mean():.1f}, écart-type {dep.std():.1f}")
for q in (0.95, 0.99, 0.999):
    plafond = dep.quantile(q)
    coupe = (dep - dep.clip(upper=plafond)).sum() / dep.sum()
    print(f"plafond au centile {q * 100:g} ({plafond:7.1f} €) : moyenne
          ↪ {dep.clip(upper=plafond).mean():6.1f} | écart-type {dep.clip(upper=plafond).std():6.1f} |
          ↪ part du CA coupée {coupe * 100:4.1f} %")
```

```
vérité : moyenne 220.8, écart-type 318.0
plafond au centile 95 ( 842.9 €) : moyenne 202.0 | écart-type 252.1 | part du CA coupée 8.5 %
plafond au centile 99 (1452.0 €) : moyenne 217.3 | écart-type 300.0 | part du CA coupée 1.6 %
plafond au centile 99.9 (2230.8 €) : moyenne 220.3 | écart-type 314.4 | part du CA coupée 0.2 %
```

Plus le plafond est bas, plus la moyenne et surtout l'écart-type diminuent, et plus la part du chiffre d'affaires « coupée » augmente : plafonner au 95^e centile (843 €) retire 8,5 % du chiffre d'affaires et ramène l'écart-type de 318 € à 252 €, plafonner au 99^e (1 452 €) en retire 1,6 %, plafonner au 99,9^e seulement 0,2 %. Le plafond est un **curseur** entre « une moyenne stable » et « la vérité des gros clients » : on le choisit en fonction de la question (une moyenne de pilotage tolère le plafond, un calcul de chiffre d'affaires non).

Corrigé 1.6

```
print("doublons exacts :", int(petit.duplicated().sum()), "| sur (ticket, article) :",
      ↪ int(petit.duplicated(["ticket", "article"]).sum()))
print(petit[petit.duplicated(["ticket", "article"], keep=False)].index.tolist(), "| keep=False marque
      ↪ toutes les copies, première comprise")
```

```
doublons exacts : 1 | sur (ticket, article) : 2
[0, 1, 4, 5] | keep=False marque toutes les copies, première comprise
```

Il y a **1 doublon exact** (la ligne T1, Vase, 1, 20 € répétée). Sur (ticket, article) seulement, il y en a **2** : T1/Vase, mais aussi T3/Bol (la quantité diffère : 1 et 2). Ce n'est **pas** un doublon, mais deux lignes d'un même ticket pour un même article (deux achats, ou une erreur de saisie) : la clé était trop **large**. `keep=False` marque **toutes** les copies, première comprise, ce qui sert à les afficher et à les examiner (alors que `keep="first"` ne marque que les suivantes).

Corrigé 1.7

```
c = pd.read_csv(os.path.join(D, "crm_clients.csv"), dtype=str)
c = c[c["email"].str.strip().str.lower() != "test@example.com"].dropna(subset=["email"]).copy()
c["email_cle"] = c["email"].str.strip().str.lower()
c["ins"] = pd.to_datetime(c["date_inscription"], format="%d/%m/%Y")
c["manquants"] = c[["telephone", "code_postal", "date_naissance"]].isna().sum(axis=1)
recent = c.sort_values(["email_cle", "ins"], kind="stable").drop_duplicates("email_cle", keep="last")
complete = c.sort_values(["email_cle", "manquants"], kind="stable").drop_duplicates("email_cle",
↳ keep="first")
fusion = c.sort_values(["email_cle", "manquants"], kind="stable").groupby("email_cle").first()
print("lignes :", len(c), "| fiches :", len(complete), "| égalité des dates d'inscription dans les
↳ groupes de doublons :", bool((c.groupby("email_cle")["ins"].nunique() <= 1).all()))
print("codes postaux manquants : ligne la plus récente", int(recent["code_postal"].isna().sum()), "|
↳ ligne la plus complète", int(complete["code_postal"].isna().sum()), "| fusion colonne par
↳ colonne", int(fusion["code_postal"].isna().sum()))
```

```
lignes : 6769 | fiches : 6092 | égalité des dates d'inscription dans les groupes de doublons : False
codes postaux manquants : ligne la plus récente 368 | ligne la plus complète 328 | fusion colonne par
↳ colonne 328
```

Choisir **la ligne la plus récente** laisse 368 codes postaux manquants ; choisir **la ligne la plus complète** en laisse 328, et la **fusion colonne par colonne** (`groupby(...).first()`) prend, pour chaque colonne, la première valeur **non vide**) aussi : ici, la ligne la plus complète contient déjà tout ce que les autres copies savent, de sorte que la fusion ne gagne rien de plus. Le critère « le plus récent » n'a presque aucun pouvoir de discrimination : les copies d'un même client recopient la **même date d'inscription** (les seuls groupes aux dates différentes sont les trois e-mails partagés par deux clients distincts), et la ligne retenue dépend alors de l'ordre du fichier. Le critère se choisit d'après ce qui **distingue** réellement les copies.

Corrigé 1.8

```
x = site.merge(pd.read_csv(os.path.join(D,
↳ "verite_site.csv")).drop_duplicates("order_ref").query("default != 'test'"), on="order_ref")
x["a_l_oeil"] = np.where(x["total_n"] > 1000, x["total_n"] / 100, x["total_n"])
ecart = x["a_l_oeil"] - x["total_vrai"]
mal = ecart.abs() > 0.01
print("commandes mal corrigées :", int(mal.sum()), "sur", len(x), "| trop petites :", int((ecart <
↳ -0.01).sum()), "| trop grandes :", int((ecart > 0.01).sum()))
print("total en centimes le plus bas parmi les commandes mal corrigées :", int(x.loc[mal,
↳ "total_n"].min()), "| plus haut :", int(x.loc[mal, "total_n"].max()), "| date minimale :",
↳ x.loc[mal, "dt"].min().date())
```

```
commandes mal corrigées : 49 sur 6199 | trop petites : 0 | trop grandes : 49
total en centimes le plus bas parmi les commandes mal corrigées : 239 | plus haut : 917 | date
↳ minimale : 2025-09-17
```

La correction « à l'œil » rate 49 commandes, **toutes cent fois trop grandes** : ce sont des commandes **postérieures au 15 septembre** dont le total, **en centimes**, est inférieur à 1 000 (donc moins de 10 €) : le seuil sur la valeur ne les divise pas. Elles sont peu nombreuses parce que les petites commandes sont rares, mais elles faussent la moyenne et surtout les valeurs extrêmes. Le seuil

confond **le niveau d'une valeur et son unité**. La correction **datée** s'appuie sur la seule information fiable : la date où le système a changé, que l'on a détectée (application 1.5) et vérifiée.

Corrigé 1.9

```
def lire_caisse2(fichier):
    brut = open(fichier, "rb").read()
    try:
        lignes, enc = brut.decode("utf-8-sig").splitlines(), "utf-8"
    except UnicodeDecodeError:
        lignes, enc = brut.decode("cp1252").splitlines(), "cp1252"
    est_entete = lambda l: re.match(r'^"? (N° ticket|Ticket)', l) is not None
    i0 = next(i for i, l in enumerate(lignes) if est_entete(l))
    sep = max([";", ",", "\t"], key=lambda s: lignes[i0].count(s))
    total = float(lignes[-1].split(sep)[-1].strip('"').replace(",", "."))
    t = pd.read_csv(io.StringIO("\n".join([lignes[i0]] + [l for l in lignes[i0 + 1:-1] if not
    ↪ est_entete(l)])), sep=sep, dtype=str)
    t.columns = [c.replace("N° ticket", "ticket").replace("Ticket", "ticket").replace("Quantité",
    ↪ "Qté") for c in t.columns]
    t = t.rename(columns={"Date": "date", "Heure": "heure", "Article": "article", "Catégorie":
    ↪ "categorie", "Qté": "quantite", "Prix unitaire": "prix_unitaire", "Montant": "montant"})
    for c in ("prix_unitaire", "montant"):
        t[c] = pd.to_numeric(t[c].str.replace(",", "."), errors="coerce")
    fmt = "%Y-%m-%d" if re.fullmatch(r"\d{4}-\d\d-\d\d", t["date"].iloc[0]) else ("%d/%m/%y" if
    ↪ len(t["date"].iloc[0]) == 8 else "%d/%m/%Y")
    t["date"] = pd.to_datetime(t["date"], format=fmt)
    t["quantite"] = t["quantite"].astype(int)
    return t, total, f"{enc}, séparateur {sep!r}"
```

```
t13, total13, desc13 = lire_caisse2(fichier13)
print(desc13, "| lignes :", len(t13), "| somme lue :", round(t13["montant"].sum(), 2), "| total
↪ affiché :", total13, "| dates :", t13["date"].dt.strftime("%d/%m/%Y").tolist())
print("l'ancienne fonction sur le même fichier :", end=" ")
try:
    C.lire_caisse(fichier13)
except Exception as e:
    print(type(e).__name__)
```

```
utf-8, séparateur '\t' | lignes : 2 | somme lue : 52.36 | total affiché : 52.36 | dates :
↪ ['02/01/2026', '02/01/2026']
l'ancienne fonction sur le même fichier : ValueError
```

Deux changements suffisent : le séparateur est choisi par **le caractère le plus fréquent** dans l'entête (parmi trois candidats), et le format de date est **déduit de la forme** de la première date. La somme lue (52,36 €) retrouve le total affiché. La généralisation a un coût : chaque nouvelle variante de format demande de reprendre la fonction. Raison de plus pour obtenir de l'équipe qui produit l'export un **format stable et documenté** (CSV en UTF-8, point-virgule, dates ISO).

Corrigé 1.10

```
def tel(s):
    chiffres = re.sub(r"\D", "", s)
    if s.startswith("+"):
        chiffres = "0" + chiffres[2:]
    return chiffres if re.fullmatch(r"0\d{9}", chiffres) else None
for n in numeros:
    print(f"{n:22s} -> {tel(n)}")
```

```

01 23 45 67 89      -> 0123456789
01.23.45.67.89      -> 0123456789
0123456789          -> 0123456789
(0)1 23456789       -> 0123456789
+99 1 23 45 67 89  -> 0123456789
12 34 56            -> None
01 23 45 67 8A     -> None

```

Les cinq premiers numéros se ramènent à 0123456789. « +99 1 23 45 67 89 » est accepté **parce que** l'indicatif de deux chiffres est remplacé par 0 : c'est une hypothèse (l'indicatif n'est pas contrôlé) à documenter. « 12 34 56 » est **rejeté** (six chiffres : trop court) et « 01 23 45 67 8A » aussi (le A disparaît avec \D, il ne reste que neuf chiffres). On préfère rejeter une valeur invalide plutôt que d'en inventer une.

Corrigé 1.11

```

m = profil["revenu_annuel"].isna()
profil["classe_age2"] = pd.cut(profil["age"], [0, 29, 44, 59, 200])
for nom in ("median", "mean"):
    imp = profil.groupby(["classe_age2", "canal_acquisition"],
        ↳ observed=True)["revenu_annuel"].transform(nom)
    complet = profil["revenu_annuel"].where(~m, imp)
    print(f"nom:{6s} par groupe : erreur typique {np.sqrt(((imp[m] - verite.loc[m, 'revenu_annuel'])
        ↳ ** 2).mean()):8.0f} | biais de la moyenne {complet.mean() -
        ↳ verite['revenu_annuel'].mean():6.0f} | écart-type {complet.std():8.0f}")

```

median par groupe : erreur typique	10352	biais de la moyenne	-320	écart-type	10400
mean par groupe : erreur typique	10180	biais de la moyenne	-24	écart-type	10365

La **moyenne** de groupe fait légèrement mieux ici : erreur individuelle de 10 180 € contre 10 352 €, et surtout **biais de la moyenne de -24 € contre -320 €**. La raison est simple : la distribution des revenus est asymétrique (la médiane d'un groupe est inférieure à sa moyenne), donc imputer la médiane **tire la moyenne vers le bas**. La différence d'erreur individuelle est faible parce que **le groupe explique peu le revenu** : peu importe la valeur centrale, l'incertitude individuelle domine. On choisit la médiane si l'on veut être robuste aux extrêmes, la moyenne si l'on veut **préserver la moyenne**.

Corrigé 1.12

```

rng = np.random.default_rng(3)
sat = verite["satisfaction_moy"]
manque = rng.random(len(sat)) < np.where(sat < 3, 0.5, 0.05)
obs = sat.where(~manque)
print("part de manquants :", round(manque.mean() * 100, 1), "% | moyenne vraie :", round(sat.mean(),
    ↳ 3), "| moyenne observée :", round(obs.mean(), 3), "| biais :", round(obs.mean() - sat.mean(), 3))
for delta in (0, 0.1, 0.2, 0.3, 0.4, 0.5):
    print(f"delta {delta:.1f} : moyenne après imputation {obs.where(~manque, obs.mean() -
        ↳ delta).mean():.3f}")
print("décalage réel entre répondants et non-répondants :", round(obs.mean() - sat[manque].mean(), 3))

```

part de manquants :	12.1 %	moyenne vraie :	3.729	moyenne observée :	3.808	biais :	0.079
delta 0.0 :	moyenne après imputation 3.808						
delta 0.1 :	moyenne après imputation 3.796						
delta 0.2 :	moyenne après imputation 3.784						
delta 0.3 :	moyenne après imputation 3.772						
delta 0.4 :	moyenne après imputation 3.760						
delta 0.5 :	moyenne après imputation 3.748						
décalage réel entre répondants et non-répondants :	0.653						

L'absence supprime surtout les clients peu satisfaits : la moyenne observée (3,81) est supérieure à la vérité (3,73) : le biais est positif, de 0,08 point. Le balayage montre que la moyenne retrouvée dépend **linéairement** de δ , et que le δ qui rétablit la vraie moyenne est celui que l'on lit dans la dernière ligne, l'écart réel entre répondants et non-répondants (0,65 point) : avec 12 % de manquants, il faut soustraire 0,65 aux valeurs imputées pour compenser un biais de 0,08. Cet écart ne s'obtient **qu'avec la vérité**. Dans la vraie vie, on choisit δ **à partir d'une source externe** (une enquête de relance auprès d'un échantillon de non-répondants, qui mesure l'écart réel) ou on présente **plusieurs valeurs** de δ en disant comment la conclusion change : c'est une analyse de sensibilité, pas une correction.

Chapitre 2 : Transformation et fusion des données — exercices et applications

Ce chapitre du cahier prolonge le chapitre 2 du livre. Les **applications** sont de petites études guidées sur les données de la boutique : chacune annonce un objectif, avance par étapes courtes (au plus vingt-cinq lignes par bloc) et se termine par un « À vous » corrigé en fin de chapitre. Les **exercices** sont notés ★ (direct), ★★ (demande un peu de méthode), ★★★ (demande de combiner plusieurs sections). Les corrigés suivent : cherchez avant de les ouvrir. Les lectures de la caisse et du site, les jointures et le rapprochement du fichier clients **reposent sur le module `build/outils_ch02.py`**, qui regroupe les fonctions que le livre construit pas à pas : le cahier peut ainsi se concentrer sur les questions plutôt que sur la recopie du code de lecture.

Une seule cellule charge les bibliothèques et les tables de base (lignes de commande, commandes, produits). Les autres cellules en dépendent ; relancez-la si vous repartez de zéro.

```
import sys, os
import numpy as np, pandas as pd
sys.path.insert(0, "build")
import outils_ch02 as O          # les fonctions de lecture du livre, regroupées (voir
↳ build/outils_ch02.py)
lig = pd.read_csv("donnees/lignes_commande.csv")
cmd = pd.read_csv("donnees/commandes.csv", parse_dates=["date_commande"])
prod = pd.read_csv("donnees/produits.csv")
print(len(lig), "lignes |", len(cmd), "commandes |", len(prod), "produits")
```

83905 lignes | 36395 commandes | 120 produits

Applications

Application 2.1 — La marge par canal et par trimestre (section 2.1)

Objectif. Calculer la marge brute hors taxe de chaque ligne, puis son taux par canal et par trimestre en 2025, en respectant la règle « un ratio d'agrégat est un rapport de sommes ».

Étape 1 : relier les trois tables. On joint les lignes aux produits (coût d'achat) et aux commandes (canal, date), en déclarant la cardinalité de chaque jointure.

```
x = lig.merge(prod[["id_produit", "categorie", "cout_achat"]], on="id_produit", validate="m:1")
x = x.merge(cmd[["id_commande", "canal", "date_commande"]], on="id_commande", validate="m:1")
```

```
x["ca_ht"] = x["montant"] / (1 + 0.TVA)
x["marge_ht"] = x["ca_ht"] - x["quantite"] * x["cout_achat"]
x["annee"], x["trimestre"] = x["date_commande"].dt.year, x["date_commande"].dt.quarter
print(len(lig), "->", len(x), "lignes | TVA fictive :", 0.TVA)
```

```
83905 -> 83905 lignes | TVA fictive : 0.2
```

Étape 2 : le tableau. Taux de marge par canal (en lignes) et par trimestre (en colonnes), pour 2025.

```
tab_marge = x[x["annee"] == 2025].groupby(["canal", "trimestre"]).agg(ca_ht=("ca_ht", "sum"),
↳ marge=("marge_ht", "sum"))
tab_marge["taux"] = tab_marge["marge"] / tab_marge["ca_ht"]
print(tab_marge["taux"].unstack("trimestre").round(3).to_string())
```

trimestre	1	2	3	4
canal				
Boutique	0.370	0.384	0.382	0.381
Réseaux	0.369	0.389	0.378	0.383
Site	0.366	0.383	0.377	0.384

Étape 3 : le contrôle. La somme des marges des groupes doit retrouver la marge totale de 2025.

```
total_2025 = x.loc[x["annee"] == 2025, "marge_ht"].sum()
print("somme des groupes :", round(tab_marge["marge"].sum(), 2), "| marge totale :",
↳ round(total_2025, 2), "| égalité :", bool(np.isclose(tab_marge["marge"].sum(), total_2025)))
```

```
somme des groupes : 419016.93 | marge totale : 419016.93 | égalité : True
```

À vous. Quel couple (canal, trimestre) a le taux de marge le plus bas ? Calculez aussi, pour ce couple, la moyenne **des taux des lignes** : est-elle égale au taux de marge ? Pourquoi ? (*Piste en fin de chapitre.*)

Application 2.2 — Délais, clients dormants et fuite d'information (sections 2.1.5 et 2.1.7)

Objectif. Mesurer le délai entre les commandes d'un client, ranger les clients dans des classes, puis constater qu'une variable qui regarde le futur **triche**.

Étape 1 : un délai par commande. On fabrique la table des paniers (une ligne par commande), on la trie par client et par date, et l'on décale d'une ligne **au sein de chaque client**.

```
paniers = lig.groupby("id_commande", as_index=False)["montant"].sum().rename(columns={"montant":
↳ "panier"})
paniers = paniers.merge(cmd[["id_commande", "id_client", "date_commande"]], on="id_commande",
↳ validate="1:1")
paniers = paniers.sort_values(["id_client", "date_commande", "id_commande"]).reset_index(drop=True)
g = paniers.groupby("id_client")["date_commande"]
paniers["delai_jours"] = (paniers["date_commande"] - g.shift()).dt.days
print(paniers["delai_jours"].describe().round(1).to_string())
```

count	31589.0
mean	86.4
std	111.1
min	0.0
25%	17.0
50%	47.0
75%	110.0
max	1014.0

Étape 2 : des classes de clients. Le délai **moyen** de chaque client, découpé avec des bornes que l'on écrit : moins de 30 jours, 30 à 90, 90 à 180, 180 et plus. Les clients à une seule commande n'ont **pas** de délai : ils restent à part.

```
moyen = paniers.groupby("id_client")["delai_jours"].mean()
classes = pd.cut(moyen, [0, 30, 90, 180, np.inf], right=False, labels=["< 30 j", "30-90 j", "90-180 j", "180 j et +"])
print(classes.value_counts(dropna=False).sort_index().to_string())
print("clients sans délai (une seule commande) :", int(moyen.isna().sum()))
```

```
delai_jours
< 30 j      180
30-90 j     1404
90-180 j    1319
180 j et +  1070
NaN         833
clients sans délai (une seule commande) : 833
```

Étape 3 : la fuite. Au 31 mars 2025, on veut prévoir qui commandera **encore en 2025**. Deux versions du nombre de commandes : avant la coupure, et au total.

```
from sklearn.metrics import roc_auc_score
def auc_fuite(coupure):
    c = pd.Timestamp(coupure)
    avant = paniers[paniers["date_commande"] <= c].groupby("id_client").size().rename("avant")
    total = paniers.groupby("id_client").size().rename("total")
    futur = paniers[(paniers["date_commande"] > c) & (paniers["date_commande"].dt.year == 2025)].groupby("id_client").size().rename("futur")
    t = pd.concat([avant, total, futur], axis=1).dropna(subset=["avant"]).fillna(0)
    y = (t["futur"] > 0).astype(int)
    return round(roc_auc_score(y, t["avant"], 3), round(roc_auc_score(y, t["total"], 3), len(t)))
print("coupure au 31 mars : AUC 'avant', AUC 'total', clients ->", auc_fuite("2025-03-31"))
```

```
coupure au 31 mars : AUC 'avant', AUC 'total', clients -> (0.787, 0.902, 4225)
```

À vous. Relancez `auc_fuite("2025-09-30")`. L'écart entre les deux AUC augmente-t-il ou diminue-t-il ? Pourquoi ? (*Piste en fin de chapitre.*)

Application 2.3 — Contrôler une jointure (sections 2.2.2 et 2.2.5)

Objectif. Écrire la fonction de contrôle d'une jointure, l'appliquer à deux jointures dont l'une est **saine** et l'autre **n-n**, et réparer la seconde.

Étape 1 : la fonction.

```
def controler(gauche, droite, cle, how="left"):
    r = gauche.merge(droite, on=cle, how=how, indicator=True)
    return {"avant": len(gauche), "après": len(r), "clé dupliquée à droite":
        int(droite.duplicated(cle).sum()),
        "sans correspondance": int((r["_merge"] == "left_only").sum())}
print("lignes x produits, sur l'identifiant :", controler(lig, prod[["id_produit", "cout_achat"]],
    "id_produit"))
```

```
lignes x produits, sur l'identifiant : {'avant': 83905, 'après': 83905, 'clé dupliquée à droite': 0,
    'sans correspondance': 0}
```

Étape 2 : la jointure fautive. Les lignes de la caisse, jointes aux produits sur le **nom** normalisé.

```
caisse, ctrl = 0.charger_caisse()
produits = 0.charger_produits()
caisse["nom_norm"] = caisse["article"].map(0.norm_texte)
print("caisse x produits, sur le nom :", controler(caisse, produits[["nom_norm", "cout_achat"]],
    "nom_norm"))
```

```
caisse x produits, sur le nom : {'avant': 12678, 'après': 25356, 'clé dupliquée à droite': 60, 'sans  
↪ correspondance': 0}
```

Étape 3 : la réparation. On ajoute le prix de 2025 à la clé, avec une tolérance d'un demi-centime, et l'on compte les lignes uniques, ambiguës et sans candidat.

```
c = caisse.reset_index().merge(produits[["nom_norm", "id_produit", "prix_2025"]], on="nom_norm")
c = c[(c["prix_unitaire"] - c["prix_2025"]).abs() < 0.011]
n = c.groupby("index").size()
print("uniques :", int((n == 1).sum()), "| ambiguës :", int((n > 1).sum()), "| sans candidat :",
↪ len(caisse) - len(n))
```

```
uniques : 12452 | ambiguës : 226 | sans candidat : 0
```

À vous. Pourquoi la clé (nom, prix) ne départage-t-elle pas toutes les lignes ? Que faut-il faire des lignes restantes ? (*Piste en fin de chapitre.*)

Application 2.4 — Lire un fichier de janvier et un fichier de décembre (section 2.2.3)

Objectif. Constater la dérive de schéma entre deux fichiers de la caisse, les lire avec la même fonction et appliquer la somme de contrôle.

Étape 1 : regarder les octets. Le codage et la première ligne d'en-tête diffèrent.

```
for f in ["caisse_2025-01.csv", "caisse_2025-12.csv"]:
    brut = open(os.path.join("donnees/caisse", f), "rb").read()
    entete = brut.decode("utf-8-sig" if brut.startswith(b"\xef\xbb\xbf") else
↪ "cp1252").splitlines()[3]
    print(f, "| BOM UTF-8 :", brut.startswith(b"\xef\xbb\xbf"), "| en-tête :", entete[:70])
```

```
caisse_2025-01.csv | BOM UTF-8 : False | en-tête : N° ticket;Date;Heure;Article;Catégorie;Qté;Prix
↪ unitaire;Montant
caisse_2025-12.csv | BOM UTF-8 : True | en-tête : Ticket,Date,Heure,Article,Catégorie,Qté,Prix
↪ unitaire,Remise (%),Monta
```

Étape 2 : lire les deux avec la même fonction. 0.lire_caisse_fichier détecte le codage, le séparateur, la décimale et le format de date.

```
d1, t1 = 0.lire_caisse_fichier("donnees/caisse/caisse_2025-01.csv")
d12, t12 = 0.lire_caisse_fichier("donnees/caisse/caisse_2025-12.csv")
for nom, d, t in [("janvier", d1, t1), ("décembre", d12, t12)]:
    print(f"nom:{9s} lignes : {len(d):5d} | total affiché : {t:10.2f} | somme lue :
↪ {d['montant'].sum():10.2f} | vides : {int(d['montant'].isna().sum())}")
print("remise renseignée en janvier :", bool(d1["remise_pct"].notna().any()), "| en décembre :",
↪ bool(d12["remise_pct"].notna().any()))
```

```
janvier lignes : 955 | total affiché : 38882.41 | somme lue : 37826.31 | vides : 36
décembre lignes : 1694 | total affiché : 73384.87 | somme lue : 70924.34 | vides : 63
remise renseignée en janvier : False | en décembre : True
```

Étape 3 : reconstituer. Quantité × prix × (1 − remise), la remise étant supposée nulle quand elle n'est pas écrite. Quel écart reste-t-il ?

```
for nom, d, t in [("janvier", d1, t1), ("décembre", d12, t12)]:
    rec = d["montant"].fillna((d["quantite"] * d["prix_unitaire"] * (1 - d["remise_pct"].fillna(0) /
↪ 100)).round(2))
    print(nom, "écart au total affiché après reconstitution :", round(rec.sum() - t, 2))
```

```
janvier écart au total affiché après reconstitution : 351.63
décembre écart au total affiché après reconstitution : 303.48
```

À vous. Pourquoi l'écart de décembre est-il plus petit que celui de janvier, proportionnellement ? Qu'est-ce qui reste, dans chaque cas ? (*Piste en fin de chapitre.*)

Application 2.5 — Nettoyer l'export du site (section 2.2.4)

Objectif. Appliquer les filtres du livre à l'export du site, comparer le chiffre d'affaires mensuel avec la base et expliquer l'écart.

Étape 1 : lire et marquer. `0.lire_site` ne supprime rien : il ajoute des colonnes qui disent si la ligne est un test, un doublon d'export, une commande annulée.

```
site, lignes_site = 0.lire_site()
print("lignes :", len(site), "| tests :", int(site["est_test"].sum()), "| doublons :",
      ↪ int(site["est_double"].sum()), "| annulées :", int((site["statut"] == "cancelled").sum()))
valides = site[~site["est_test"] & ~site["est_double"]]
print("commandes valides (annulées comprises) :", len(valides))
```

```
lignes : 6259 | tests : 60 | doublons : 121 | annulées : 186
commandes valides (annulées comprises) : 6078
```

Étape 2 : le chiffre d'affaires mensuel. Avec et sans les commandes annulées.

```
valides = valides.assign(mois=valides["date_heure"].dt.month)
ca_avec = valides.groupby("mois")["total_num"].sum()
ca_sans = valides[valides["statut"] != "cancelled"].groupby("mois")["total_num"].sum()
print(pd.DataFrame({"avec annulées": ca_avec, "sans annulées": ca_sans}).round(0).T.to_string())
```

mois		1	2	3	4	5	6	7	8	9	10
↪ 11	12										
avec annulées		41358.0	34084.0	42308.0	43392.0	48614.0	50044.0	53634.0	37311.0	55538.0	
↪		56535.0	64851.0	90048.0							
sans annulées		40221.0	32746.0	40821.0	41828.0	47471.0	48555.0	52340.0	36278.0	54524.0	
↪		55000.0	62884.0	87496.0							

Étape 3 : la comparaison avec la base.

```
mt = lig.groupby("id_commande")["montant"].sum()
b = cmd[(cmd["canal"] == "Site") & (cmd["date_commande"].dt.year == 2025)].assign(ca=lambda d:
      ↪ d["id_commande"].map(mt))
base_mois = b.groupby(b["date_commande"].dt.month)["ca"].sum()
print("écart max entre la base et « avec annulées » :", round((base_mois - ca_avec).abs().max(), 2),
      ↪ "| écart max avec « sans annulées » :", round((base_mois - ca_sans).abs().max(), 2))
```

```
écart max entre la base et « avec annulées » : 0.0 | écart max avec « sans annulées » : 2552.4
```

À vous. Que valent exactement, chaque mois, les écarts entre la base et « sans annulées » ? À quoi correspondent-ils ? (*Piste en fin de chapitre.*)

Application 2.6 — La table des ventes et son contrôle (sections 2.2.6 à 2.2.8)

Objectif. Construire la table des ventes (caisse et site) avec `0.ventes_2025`, la contrôler mois par mois contre la base et chercher ce qui manque.

Étape 1 : la table.

```
ventes, ctrl_caisse = 0.ventes_2025()
ventes["mois"] = ventes["date"].dt.month
print(len(ventes), "lignes |", ventes.groupby("canal")["montant"].sum().round(2).to_dict())
```

```
26188 lignes | {'Boutique': 564472.59, 'Site': 600164.13}
```

Étape 2 : le contrôle par canal et par mois.

```
v25 = lig.merge(cmd[["id_commande", "canal", "date_commande"]], on="id_commande", validate="m:1")
v25 = v25[v25["date_commande"].dt.year == 2025].assign(mois=lambda d: d["date_commande"].dt.month)
comp = pd.DataFrame({"table": ventes.groupby(["canal", "mois"])["montant"].sum(), "base":
    ↪ v25.groupby(["canal", "mois"])["montant"].sum()}).dropna()
comp["ecart_pct"] = (comp["table"] / comp["base"] - 1) * 100
print(comp["ecart_pct"].unstack("canal").round(2).to_string())
```

canal	Boutique	Site
mois		
1	0.90	-2.75
2	0.00	-3.93
3	0.37	-3.51
4	1.48	-3.60
5	0.95	-2.35
6	0.67	-2.98
7	0.47	-2.41
8	1.05	-2.77
9	0.72	-1.83
10	0.16	-2.72
11	0.40	-3.03
12	0.41	-2.83

Étape 3 : anti-jointures dans les deux sens.

```
base_bs = set(v25.loc[v25["canal"].isin(["Boutique", "Site"]), "id_commande"])
ids = set(ventes["id_commande"])
print("commandes de la base absentes de la table :", len(base_bs - ids), "| de la table absentes de la
    ↪ base :", len(ids - base_bs))
```

```
commandes de la base absentes de la table : 181 | de la table absentes de la base : 0
```

À vous. Quels mois ont l'écart le plus grand pour chaque canal, et pourquoi ? Que sont les commandes de la base absentes de la table ? Pourquoi, pour la Boutique, aucune commande ne manque-t-elle alors que les totaux diffèrent ? (*Piste en fin de chapitre.*)

Application 2.7 — Agréger et contrôler (section 2.3)

Objectif. Passer de la ligne à la commande, calculer des parts avec transform, puis voir quels contrôles détectent une erreur.

Étape 1 : la table des commandes.

```
cmd_v = ventes.groupby(["canal", "id_commande"], as_index=False).agg(ca=("montant", "sum"),
    ↪ lignes=("montant", "size"), mois=("mois", "first"))
print(len(ventes), "lignes ->", len(cmd_v), "commandes | CA :", round(ventes["montant"].sum(), 2),
    ↪ "=", round(cmd_v["ca"].sum(), 2))
print("panier moyen par canal :", cmd_v.groupby("canal")["ca"].mean().round(2).to_dict())
```

```
26188 lignes -> 11339 commandes | CA : 1164636.72 = 1164636.72
panier moyen par canal : {'Boutique': 103.73, 'Site': 101.77}
```

Étape 2 : la part de chaque catégorie dans son mois.

```
cat_mois = ventes.groupby(["mois", "categorie"], as_index=False)["montant"].sum()
cat_mois["part"] = cat_mois["montant"] / cat_mois.groupby("mois")["montant"].transform("sum")
print(cat_mois[cat_mois["mois"] == 12].sort_values("part",
↳ ascending=False).round(3).head(3).to_string(index=False))
print("somme des parts par mois (min, max) :", round(cat_mois.groupby("mois")["part"].sum().min(),
↳ 6), round(cat_mois.groupby("mois")["part"].sum().max(), 6))
```

```
mois categorie montant part
12 Décoration 52982.84 0.329
12 Maison 39492.40 0.245
12 Cuisine 29751.27 0.185
somme des parts par mois (min, max) : 1.0 1.0
```

Étape 3 : quel contrôle voit quoi ? On fabrique deux erreurs : 100 lignes dupliquées, et 100 montants multipliés par dix.

```
total_ref = ventes["montant"].sum()
fautes = {"100 lignes dupliquées": pd.concat([ventes, ventes.head(100)]), "100 montants x10":
↳ ventes.assign(montant=np.where(ventes.index < 100, ventes["montant"] * 10, ventes["montant"]))}
for nom, t in fautes.items():
    interne = np.isclose(t.groupby("canal")["montant"].sum().sum(),
↳ t.groupby("mois")["montant"].sum().sum())
    print(f"{nom:24s} | contrôle interne (canal = mois) : {bool(interne)} | contrôle externe (= total
↳ de référence) : {bool(np.isclose(t['montant'].sum(), total_ref))}")
```

```
100 lignes dupliquées | contrôle interne (canal = mois) : True | contrôle externe (= total de
↳ référence) : False
100 montants x10 | contrôle interne (canal = mois) : True | contrôle externe (= total de
↳ référence) : False
```

À vous. Quel type d'erreur le contrôle **externe** voit-il, et pas l'interne ? Que ferait-il falloir pour détecter une erreur qui laisse le total inchangé (deux lignes échangées) ? (Piste en fin de chapitre.)

Application 2.8 — Le tableur de stocks (section 2.4)

Objectif. Lire le tableur désordonné, le passer au format long, compter les ruptures par catégorie et vérifier un aller-retour large → long → large.

Étape 1 : lire et filtrer.

```
import openpyxl
ws = openpyxl.load_workbook("donnees/stocks_tableur.xlsx")["Stock 2025"]
brut = pd.DataFrame(list(ws.iter_rows(min_row=5, values_only=True))).iloc[:, :14]
brut.columns = ["ref", "designation"] + list(range(1, 13))
est_produit = brut["ref"].astype(str).str.fullmatch(r"P\d{3}")
print(len(brut), "lignes lues |", int(est_produit.sum()), "lignes de produits")
```

```
137 lignes lues | 120 lignes de produits
```

Étape 2 : convertir et dépivoter. « rupture » vaut 0, « ND » et « — » sont des inconnus.

```
def vers_nombre(v):
    if isinstance(v, (int, float)): return v
    s = str(v).strip()
    return 0 if s == "rupture" else (int(s) if s.isdigit() else np.nan)
stock_large = brut[est_produit].copy()
stock_large[list(range(1, 13))] = stock_large[list(range(1, 13))].apply(lambda col:
↳ col.map(vers_nombre))
stock = stock_large.melt(id_vars=["ref", "designation"], var_name="mois", value_name="stock")
stock["id_produit"] = stock["ref"].str[1:].astype(int)
```

```
stock = stock.merge(prod[["id_produit", "categorie"]], on="id_produit", validate="m:1")
print(len(stock), "lignes |", int(stock["stock"].isna().sum()), "inconnus |", int((stock["stock"] ==
↳ 0).sum()), "ruptures")
```

1440 lignes | 71 inconnus | 120 ruptures

Étape 3 : analyse et aller-retour.

```
rupt = stock.assign(rupture=stock["stock"] == 0).pivot_table(index="categorie", columns="mois",
↳ values="rupture", aggfunc="sum")
print("ruptures par catégorie sur l'année :", rupt.sum(axis=1).astype(int).to_dict())
large2 = stock.pivot(index="id_produit", columns="mois", values="stock")
ref = stock_large.assign(id_produit=stock_large["ref"].str[1:].astype(int)).set_index("id_produit")[
↳ list(range(1, 13))].astype(float)
print("aller-retour identique :", bool(large2.astype(float).equals(ref.astype(float))))
```

```
ruptures par catégorie sur l'année : {'Bien-être': 13, 'Cuisine': 25, 'Décoration': 27, 'Jardin': 24,
↳ 'Maison': 22, 'Papeterie': 9}
aller-retour identique : True
```

À vous. Quelle catégorie compte le plus de ruptures, et quelle part de ses cellules cela représente-t-il ? Que se passerait-il sur la moyenne du stock si l'on remplaçait « ND » et « — » par zéro ? (*Piste en fin de chapitre.*)

Application 2.9 — Dédoublonner le CRM sans téléphone (section 2.5)

Objectif. Comparer trois clés de blocage, puis régler les seuils d'un score de ressemblance et mesurer précision et rappel.

Étape 1 : charger le CRM normalisé et les paires vraies.

```
crm, vraies = 0.charger_crm()
print(len(crm), "lignes |", len(vraies), "paires vraies |", crm["ville_norm"].nunique(), "villes
↳ normalisées")
```

7000 lignes | 1050 paires vraies | 20 villes normalisées

Étape 2 : trois blocages. Le rappel du blocage borne celui de tout ce qui suit.

```
crm["debut_nom"] = crm["nom_tri"].str[:3]
for cles in (["ville_norm"], ["ville_norm", "annee_naiss"], ["annee_naiss", "debut_nom"]):
    b = 0.paires(crm, cles, taille_max=10_000) # on accepte aussi les gros groupes, pour voir le
↳ vrai nombre de paires
    print(f"{' + '.join(cles):26s}", 0.juger(b, vraies))
```

```
ville_norm          {'paires': 1979607, 'précision': 0.001, 'rappel': 1.0}
ville_norm + annee_naiss {'paires': 42538, 'précision': 0.024, 'rappel': 0.984}
annee_naiss + debut_nom  {'paires': 10690, 'précision': 0.085, 'rappel': 0.864}
```

Étape 3 : le score sur le blocage retenu, et trois seuils.

```
bloc = 0.paires(crm, ["ville_norm", "annee_naiss"])
S = 0.scorer_paires(crm, bloc, vraies)
for seuil in (70, 80, 90):
    sel = S["score"] >= seuil
    print(f"seuil {seuil} | paires : {int(sel.sum()):5d} | précision : {S.loc[sel, 'vrai'].mean():.3f}
↳ | rappel : {S.loc[sel, 'vrai'].sum() / len(vraies):.3f}")
```

```
seuil 70 | paires : 1111 | précision : 0.924 | rappel : 0.978
seuil 80 | paires : 1003 | précision : 0.991 | rappel : 0.947
seuil 90 | paires : 921 | précision : 0.999 | rappel : 0.876
```

À vous. Quel blocage a le meilleur compromis entre nombre de paires et rappel ? Pourquoi la clé « début du nom » est-elle fragile ici ? (*Piste en fin de chapitre.*)

Application 2.10 — Rapprocher le catalogue du fournisseur (section 2.5.7)

Objectif. Voir l'effet de la tolérance de prix sur le rapprochement du catalogue.

Étape 1 : le rapprochement de référence. 0.rapprocher_catalogue reprend la méthode du livre (nom ressemblant, pénalisé par l'écart de prix).

```
R = 0.rapprocher_catalogue()
vp = pd.read_csv("donnees/verite_produits.csv").rename(columns={"id_produit": "id_vrai"})
J = R.merge(vp, on="code_fournisseur", validate="1:1")
print("lignes :", len(J), "| acceptées :", int(J["accepte"].sum()), "| nouveautés (sans équivalent)
↳ :", int((J["id_vrai"] == -1).sum()))
```

lignes : 118 | acceptées : 108 | nouveautés (sans équivalent) : 10

Étape 2 : varier la tolérance de prix.

```
for tol in (0.01, 0.035, 0.10, 0.50):
    A = 0.rapprocher_catalogue(seuil_prix=tol).merge(vp, on="code_fournisseur")
    acc = A[A["accepte"]]
    print(f"tolérance {tol:.3f} | acceptées : {len(acc):3d} | nouveautés acceptées à tort :
↳ {int((acc['id_vrai'] == -1).sum())} | bons produits manqués : {int((~A['accepte'] &
↳ (A['id_vrai'] != -1).sum())}")
```

tolérance 0.010 | acceptées : 34 | nouveautés acceptées à tort : 0 | bons produits manqués : 74
 tolérance 0.035 | acceptées : 108 | nouveautés acceptées à tort : 0 | bons produits manqués : 0
 tolérance 0.100 | acceptées : 108 | nouveautés acceptées à tort : 0 | bons produits manqués : 0
 tolérance 0.500 | acceptées : 108 | nouveautés acceptées à tort : 0 | bons produits manqués : 0

À vous. Quelle tolérance choisiriez-vous pour la boutique, et à quelle condition accepterait-on une tolérance plus large ? (*Piste en fin de chapitre.*)

Exercices

Exercice 2.1 ★ — La marge à la main (section 2.1.2)

Trois lignes de vente toutes taxes comprises, TVA à 20 % : (1) 59,90 € pour 2 articles, coût d'achat unitaire 18,40 € ; (2) 24,00 € pour 1 article, coût 14,50 € ; (3) 120,00 € pour 4 articles avec 20 % de remise **déjà déduits** du montant, coût unitaire 18,00 €. Calculez à la main la marge hors taxe de chaque ligne et son taux, puis le taux de marge **global** des trois lignes. Est-ce la moyenne des trois taux ?

Exercice 2.2 ★ — Les semaines ISO (section 2.1.3)

Donnez l'année ISO et la semaine ISO des dates suivantes : 30 décembre 2024, 1er janvier 2025, 29 décembre 2025, 1er janvier 2026. Vérifiez avec pandas.

Exercice 2.11 ★★ — L'aller-retour pivot (section 2.4.1)

À partir de la table des ventes, fabriquez le tableau « chiffre d'affaires par catégorie (lignes) et par mois (colonnes) » avec `pivot_table`, repassez-le au format long avec `melt`, puis rebâissez-le avec `pivot`. Vérifiez l'égalité des deux tableaux larges et celle des totaux. Que se passe-t-il si vous utilisez `pivot` directement sur la table des ventes ?

Exercice 2.12 ★★ — Inconnu n'est pas zéro (section 2.4.2)

Dans le tableur de stocks, comptez les cellules « ND » et « — ». Calculez le stock moyen d'octobre (a) en ignorant les inconnus, (b) en les remplaçant par zéro. De combien la seconde moyenne sous-estime-t-elle la première ? Le raisonnement change-t-il pour « rupture » ?

Exercice 2.13 ★★ — Levenshtein à la main (section 2.5.3)

Calculez à la main la distance de Levenshtein entre bougie et bougeoir, en remplissant la table. Vérifiez avec `rapidfuzz`. Calculez ensuite la similarité ($100 \times (1 - \text{distance} / \text{longueur maximale})$) et comparez-la au `token_set_ratio` de « bougie nordique » et « nordique bougie ».

Exercice 2.14 ★★★ — Choisir un seuil par les coûts (section 2.5.6)

Dans l'application 2.9, une **fusion erronée** coûte 5 unités et un **doublon raté** 1 unité. Pour chaque seuil de 50 à 100 (de un en un), calculez le coût total (faux positifs et faux négatifs, ces derniers comptés par rapport à **toutes** les paires vraies, blocage compris). Quel seuil minimise le coût ? Comparez avec le seuil obtenu pour un coût de 1 pour 1, et expliquez la différence.

Pistes pour les « À vous » des applications

Piste de l'application 2.1

```
ct = tab_marge["taux"].dropna()
pire = ct.idxmin()
sel = x[(x["annee"] == 2025) & (x["canal"] == pire[0]) & (x["trimestre"] == pire[1])]
print("taux le plus bas :", (pire[0], int(pire[1])), round(ct.min(), 4), "| moyenne des taux des",
      "\n↳ lignes de ce couple :", round((sel["marge_ht"] / sel["ca_ht"]).mean(), 4))
```

```
taux le plus bas : ('Site', 1) 0.3656 | moyenne des taux des lignes de ce couple : 0.3623
```

Le couple (**Site**, trimestre 1) a le taux le plus bas, 36,6 % ; la moyenne des taux de ses lignes est de 36,2 %. Les deux ne sont pas égales : la moyenne donne le même poids à une ligne de 5 € et à une ligne de 150 €, alors que le **rapport des sommes** pèse chaque ligne par son chiffre d'affaires. On prend toujours le rapport des sommes pour un ratio d'agrégat.

Piste de l'application 2.2

```
for coupure in ("2025-03-31", "2025-09-30"):
    a_, t_, n_cl = auc_fuite(coupure)
    print(coupure, "| AUC honnête :", a_, "| AUC avec fuite :", t_, "| écart :", round(t_ - a_, 3), "|",
          "\n↳ clients :", n_cl)
```

2025-03-31 | AUC honnête : 0.787 | AUC avec fuite : 0.902 | écart : 0.115 | clients : 4225
 2025-09-30 | AUC honnête : 0.756 | AUC avec fuite : 0.833 | écart : 0.077 | clients : 4573

L'écart entre les deux AUC passe de 0,115 (coupure de mars) à 0,077 (coupure de septembre), et l'AUC « honnête » de 0,787 à 0,756. La variable qui regarde le futur **paraît** toujours meilleure que la variable honnête ; ce que l'on peut dire de l'évolution des écarts vient des chiffres, pas d'une règle générale, et la bonne conclusion ne change pas : **un prédicteur se calcule avec les seules données antérieures à la date de prévision.**

Piste de l'application 2.3

```
amb = c[c["index"].isin(n[n > 1].index)]
print("noms concernés :", sorted(amb["nom_norm"].unique()), "| produits candidats :",
      ↪ sorted(amb["id_produit"].astype(int).unique().tolist()), "| prix de 2025 :",
      ↪ sorted(amb["prix_2025"].unique().tolist()))
```

noms concernés : ['carnet mat'] | produits candidats : [62, 72] | prix de 2025 : [2.99]

La clé (nom, prix) laisse **226 lignes** ambiguës, qui concernent 1 nom de produit : **deux produits distincts ont le même nom et le même prix** (et le même coût d'achat). Aucune information de la ligne de caisse ne les distingue. On laisse leur id_produit **vide**, on garde le coût d'achat (identique) pour la marge, et l'on **écrit** l'ambiguïté dans la note de méthode.

Piste de l'application 2.4

```
resume = {}
for nom, d, t_ in [("janvier", d1, t1), ("décembre", d12, t12)]:
    rec = d["montant"].fillna((d["quantite"] * d["prix_unitaire"] * (1 - d["remise_pct"].fillna(0) /
    ↪ 100)).round(2))
    ident = d.duplicated(["ticket", "date", "heure", "article", "categorie", "quantite",
    ↪ "prix_unitaire", "montant"])
    resume[nom] = (rec.sum() - t_, rec[ident].sum())
    print(nom, "| écart restant :", round(rec.sum() - t_, 2), "€ (", round((rec.sum() - t_) / t_ *
    ↪ 100, 2), "%) | montant des lignes identiques :", round(rec[ident].sum(), 2), "€ | nombre :",
    ↪ int(ident.sum()))
```

janvier | écart restant : 351.63 € (0.9 %) | montant des lignes identiques : 377.4 € | nombre : 8
 décembre | écart restant : 303.48 € (0.41 %) | montant des lignes identiques : 775.88 € | nombre : 20

En décembre, la remise est **écrite** dans le fichier : la reconstitution des montants vides est exacte, et l'écart restant (303,48 €) ne peut venir que des lignes en trop. Il est **inférieur** au montant de toutes les lignes identiques (775,88 €) : certaines de ces lignes sont des répétitions **légitimes** (section 2.2.3), seules les autres sont de vrais doubles scans. En janvier, la remise n'est pas écrite : la reconstitution suppose une remise nulle (faux pour les lignes remisées pendant les soldes), et l'écart (351,63 €) combine cette hypothèse **et** les lignes en trop. Les deux fichiers portent le même nom de colonne « Montant » mais n'ont pas **la même qualité d'information.**

Piste de l'application 2.5

```
ecarts = (base_mois - ca_sans).round(2)
annul = valides[valides["statut"] == "cancelled"].groupby("mois")["total_num"].sum().round(2)
print(pd.DataFrame({"base - sans annulées": ecarts, "annulées du mois": annul}).T.to_string())
```

		1	2	3	4	5	6	7	8	9
↪ 10	11	12								
base - sans annulées	1136.98	1338.09	1486.97	1563.7	1142.18	1489.61	1293.75	1032.57	1013.75	
↪ 1535.08	1966.24	2552.4								
annulées du mois	1136.98	1338.09	1486.97	1563.7	1142.18	1489.61	1293.75	1032.57	1013.75	
↪ 1535.08	1966.24	2552.4								

L'écart mensuel entre la base et « sans annulées » est **exactement** le montant des commandes annulées du mois : la base enregistre les commandes annulées parmi les ventes, alors que l'export du site, avec son statut, permet de les écarter. L'écart avec « avec annulées » est **nul** (étape 3). Une réconciliation réussie se reconnaît à ceci : chaque écart restant est **expliqué** par une cause identifiée et chiffrée.

Piste de l'application 2.6

```
ecart_mois = comp["ecart_pct"].unstack("canal")
print("mois d'écart absolu maximal :", ecart_mois.abs().idxmax().to_dict(), "| écart :",
      ↪ ecart_mois.abs().max().round(2).to_dict())
print("mois d'écart absolu minimal :", ecart_mois.abs().idxmin().to_dict(), "| écart :",
      ↪ ecart_mois.abs().min().round(2).to_dict())
site_, _ = 0.lire_site()
annulees = set(site_.loc[site_["statut"] == "cancelled", "id_commande"].dropna().astype(int))
print("commandes de la base absentes de la table = commandes annulées du site :", (base_bs - ids) ==
      ↪ annulees)
```

```
mois d'écart absolu maximal : {'Boutique': 4, 'Site': 2} | écart : {'Boutique': 1.48, 'Site': 3.93}
mois d'écart absolu minimal : {'Boutique': 2, 'Site': 9} | écart : {'Boutique': 0.0, 'Site': 1.83}
commandes de la base absentes de la table = commandes annulées du site : True
```

Pour le **Site**, l'écart de chaque mois est le montant des commandes annulées (la base les compte, la table des ventes les écarte) : l'anti-jointure le confirme, car les 181 commandes de la base absentes de la table sont **exactement** les commandes annulées du site. Pour la **Boutique**, l'écart vient des lignes identiques conservées et de la reconstitution des montants vides ; aucune commande ne manque ni n'est inventée, et pourtant le total diffère : les **identifiants** concordent, les **montants** non. Il faut donc **les deux** contrôles.

Piste de l'application 2.7

La duplication de lignes et la multiplication de montants changent toutes deux le total par rapport à la référence, que seul le contrôle **externe** voit ; le contrôle **interne** (somme par canal égale à somme par mois) reste vrai dans les deux cas, puisque la table fautive est cohérente avec elle-même. Une erreur qui **laisse le total inchangé** (deux montants échangés entre deux lignes) n'est détectée ni par l'un ni par l'autre : il faut un contrôle **plus fin**, au niveau d'un sous-total (par catégorie, par jour) comparé à une référence indépendante, ou une vérification de **plausibilité** ligne à ligne.

```
attendu = ventes["quantite"] * ventes["prix_unitaire"] * (1 - ventes["remise_pct"].fillna(0) / 100)
ecart_ligne = (ventes["montant"] - attendu).abs()
print("lignes où montant ≠ quantité × prix × (1 - remise) :", int((ecart_ligne > 0.011).sum()), "sur",
      ↪ len(ventes), "| dont lignes de la caisse :", int(((ecart_ligne > 0.011) & (ventes["source"] ==
      ↪ "caisse")).sum()))
```

```
lignes où montant ≠ quantité × prix × (1 - remise) : 1302 sur 26188 | dont lignes de la caisse : 1302
```

Ce test de plausibilité signale 1 302 lignes sur 26 188, dont 1 302 de la caisse : ce sont celles dont le montant lu inclut une remise que le calcul ignore (la remise n'est écrite que de façon partielle).

Un contrôle qui signale quelque chose n'est utile que si l'on **sait expliquer** ce qu'il signale.

Piste de l'application 2.8

```
cat_rupt = stock.assign(rupture=stock["stock"] == 0).groupby("categorie")["rupture"].agg(["sum",
↪ "mean"]).round(3)
print(cat_rupt.sort_values("sum", ascending=False).head(3).to_string())
m_ignore = stock.loc[stock["mois"] == 10, "stock"].mean()
m_zero = stock.loc[stock["mois"] == 10, "stock"].fillna(0).mean()
print("stock moyen d'octobre : inconnus ignorés", round(m_ignore, 2), "| inconnus à zéro",
↪ round(m_zero, 2))
```

	sum	mean
catégorie		
Décoration	27	0.112
Cuisine	25	0.104
Jardin	24	0.100

stock moyen d'octobre : inconnus ignorés 47.59 | inconnus à zéro 44.82

Remplacer les inconnus par zéro **sous-estime** le stock moyen d'octobre de 5,8 % (il y a 7 inconnus ce mois-là) : on traite comme « vide » ce que l'on ne sait pas. Pour « rupture », au contraire, le zéro est la **vraie** valeur : le stock est nul, ce qui est une information.

Piste de l'application 2.9

```
for cles in (["ville_norm"], ["ville_norm", "annee_naiss"], ["annee_naiss", "debut_nom"]):
    r = 0.juger(0.paires(crm, cles, taille_max=10_000), vraies)
    print(f"{' + '.join(cles):26s} | paires {r['paires']:>9,d} | rappel {r['rappel']}"
↪ ".replace(", ", "
↪ ")")
```

ville_norm	paires 1 979 607	rappel 1.0
ville_norm + annee_naiss	paires 42 538	rappel 0.984
annee_naiss + debut_nom	paires 10 690	rappel 0.864

Le meilleur compromis entre nombre de paires et rappel est donné par les chiffres ci-dessus : la **ville seule** compare beaucoup plus de paires ; la clé année + début du nom compare peu de paires mais son rappel est le plus faible, parce que le **début du nom** est justement ce que les fautes de frappe, les initiales et les inversions abîment. Une clé de blocage doit être **fiable** (peu abîmée) et **grossière** (assez large pour ne pas séparer les vrais doublons).

Piste de l'application 2.10

```
for tol in (0.01, 0.035, 0.10, 0.50):
    A = 0.rapprocher_catalogue(seuil_prix=tol).merge(vp, on="code_fournisseur")
    acc = A[A["accepte"]]
    print(f"{'tol:.3f'} acceptées {len(acc):3d} | à tort {int((acc['id_vrai'] == -1).sum())} | manquées
↪ {int((~A['accepte'] & (A['id_vrai'] != -1)).sum())}")
```

0.010	acceptées 34	à tort 0	manquées 74
0.035	acceptées 108	à tort 0	manquées 0
0.100	acceptées 108	à tort 0	manquées 0
0.500	acceptées 108	à tort 0	manquées 0

La tolérance de 1 % manque des produits légitimes (le catalogue a ± 3 % d'écart sur les coûts) ; à 3,5 % tous les bons produits sont acceptés et toutes les nouveautés rejetées. Au-delà, la tolérance devient inutile ici, car les nouveautés se rejettent déjà par leur **nom** (similarité trop basse). On n'élargirait la tolérance que si l'on **savait** les écarts de prix plus grands (variation de tarif), en

gardant un seuil de nom exigeant : la tolérance doit refléter ce que l'on sait du **fournisseur**, pas être choisie sur les résultats.

Corrigés des exercices

Corrigé 2.1

Ligne 1 : $59,90 / 1,20 = 49,92$ € hors taxe ; coût $2 \times 18,40 = 36,80$ € ; marge 13,12 € ; taux 26,3 %.
 Ligne 2 : $24,00 / 1,20 = 20,00$ € ; coût 14,50 € ; marge 5,50 € ; taux 27,5 %. Ligne 3 : $120,00 / 1,20 = 100,00$ € ; coût $4 \times 18,00 = 72,00$ € ; marge 28,00 € ; taux 28,0 % (la remise de 20 % est **déjà** dans le montant : on ne la retire pas une seconde fois). Global : marge 46,62 € sur 169,92 € de chiffre d'affaires hors taxe, soit 27,4 % ; la moyenne des trois taux vaut 27,3 %, légèrement différente, parce que les trois lignes n'ont pas le même poids.

```
l3 = pd.DataFrame({"ttc": [59.90, 24.00, 120.00], "cout": [2 * 18.40, 14.50, 4 * 18.00]})
l3["ht"] = l3["ttc"] / 1.2; l3["marge"] = l3["ht"] - l3["cout"]; l3["taux"] = l3["marge"] / l3["ht"]
print(l3.round(3).to_string(index=False)); print("global :", round(l3["marge"].sum() /
↳ l3["ht"].sum(), 4), "| moyenne des taux :", round(l3["taux"].mean(), 4))
```

```
   ttc  cout    ht  marge  taux
59.9  36.8  49.917  13.117  0.263
24.0  14.5  20.000   5.500  0.275
120.0  72.0 100.000  28.000  0.280
global : 0.2744 | moyenne des taux : 0.2726
```

Corrigé 2.2

30 décembre 2024 : année ISO 2025, semaine 1. 1er janvier 2025 : année ISO 2025, semaine 1. 29 décembre 2025 : année ISO 2026, semaine 1. 1er janvier 2026 : année ISO 2026, semaine 1. Les derniers jours de décembre peuvent appartenir à la semaine 1 de l'année **suivante**.

```
for j in ["2024-12-30", "2025-01-01", "2025-12-29", "2026-01-01"]:
    i = pd.Timestamp(j).isocalendar(); print(j, "-> année ISO", i.year, "semaine", i.week)
```

```
2024-12-30 -> année ISO 2025 semaine 1
2025-01-01 -> année ISO 2025 semaine 1
2025-12-29 -> année ISO 2026 semaine 1
2026-01-01 -> année ISO 2026 semaine 1
```

Corrigé 2.3

Avec `right=False` (intervalles `[a ; b[`) : 50,00 → « 50 à 100 » ; 99,99 → « 50 à 100 » ; 100,00 → « 100 à 200 » ; 200,00 → « 200 et + ». Avec `right=True` (`[a ; b]`) : 50,00 → « < 50 » ; 99,99 → « 50 à 100 » ; 100,00 → « 50 à 100 » ; 200,00 → « 100 à 200 ».

```
pan = lig.groupby("id_commande")["montant"].sum()
bornes = [0, 50, 100, 200, np.inf]; lab = ["< 50", "50 à 100", "100 à 200", "200 et +"]
test = pd.Series([50.00, 99.99, 100.00, 200.00])
print(pd.cut(test, bornes, right=False, labels=lab).tolist()); print(pd.cut(test, bornes, right=True,
↳ labels=lab).tolist())
print("paniers exactement égaux à une borne :", int(pan.round(2).isin([50, 100, 200]).sum()), "sur",
↳ len(pan))
```

```
['50 à 100', '50 à 100', '100 à 200', '200 et +']
['< 50', '50 à 100', '50 à 100', '100 à 200']
paniers exactement égaux à une borne : 0 sur 36395
```

Corrigé 2.4

```
w = cmd[cmd["date_commande"].dt.year == 2025].assign(we=lambda d: d["date_commande"].dt.dayofweek >= 5)
cl_we = w.groupby("id_client")["we"].agg(part="mean", n="size")
tous = pd.read_csv("donnees/clients.csv")[["id_client"]].merge(cl_we, on="id_client", how="left")
print("clients sans commande en 2025 :", int(tous["n"].isna().sum()), "| moyenne en laissant les vides",
      round(tous["part"].mean(), 3), "| avec des zéros :", round(tous["part"].fillna(0).mean(), 3))
print("part des commandes passées le week-end (toutes les commandes) :", round(w["we"].mean(), 3))
```

```
clients sans commande en 2025 : 2125 | moyenne en laissant les vides : 0.296 | avec des zéros : 0.191
part des commandes passées le week-end (toutes les commandes) : 0.295
```

Pour un client sans commande, la part est **indéfinie** (zéro divisé par zéro) : elle n'est pas nulle. Laisser les vides donne une moyenne **par client** ; remplacer par zéro l'écrase. Aucune des deux ne répond à « quelle part des commandes se passe le week-end ? » : cette question se pose **au niveau des commandes** (dernière ligne affichée), pas des clients. Le niveau d'agrégation doit correspondre à la question.

Corrigé 2.5

a contient les produits 1, 2, 3, 5 ; b les produits 2, 3, 4, 5, 6. inner : produits 2, 3, 5 (3 lignes) ; left : 1, 2, 3, 5 (4 lignes) ; right : 2, 3, 4, 5, 6 (5 lignes) ; outer : 1, 2, 3, 4, 5, 6 (6 lignes).

```
a = pd.DataFrame({"id": [1, 2, 3, 5], "nom": list("wxyz")}); b = pd.DataFrame({"id": [2, 3, 4, 5, 6],
      "stock": [10, 0, 7, 3, 9]})
print({how: len(a.merge(b, on="id", how=how)) for how in ["inner", "left", "right", "outer"]})
```

```
{'inner': 3, 'left': 4, 'right': 5, 'outer': 6}
```

Corrigé 2.6

(a) autant de lignes que lig : jointure m:1, chaque ligne a une seule commande ; (b) **le double** : n-n, chaque nom a deux produits ; (c) autant que cmd : m:1.

```
cli = pd.read_csv("donnees/clients.csv")
print("(a)", len(lig), "->", len(lig.merge(cmd, on="id_commande", validate="m:1")))
print("(b)", len(ventes), "->", len(ventes.merge(prod[["nom_produit", "prix_vente"]],
      on="nom_produit")))
print("(c)", len(cmd), "->", len(cmd.merge(cli[["id_client"]], on="id_client", validate="m:1")))
```

```
(a) 83905 -> 83905
(b) 26188 -> 52376
(c) 36395 -> 36395
```

Corrigé 2.7

```
abime = "Théière".encode("utf-8").decode("cp1252")
repare = abime.encode("cp1252").decode("utf-8")
import ftfy
print(abime, "->", repare, "| ftfy :", ftfy.fix_text(abime))
```

```
ThÃ©iÃ¨re -> Théière | ftfy : Théière
```

La réparation inverse (réencoder en cp1252 puis décoder en UTF-8) ne marche que si **tous** les caractères abîmés existent dans cp1252 : certaines séquences UTF-8 produisent des octets que

cp1252 ne sait pas représenter, et l'information est alors **perdue**. `ftfy` essaie plusieurs réparations et retient celle qui donne du texte plausible.

Corrigé 2.8

```
s = site[~site["est_test"]].assign(jour=lambda d: d["date_heure"].dt.date)
s["brut"] = pd.to_numeric(s["total"].str.replace("€", "").str.replace(" ", "").str.replace(",", "."))
med = s.groupby("jour")["brut"].median()
premier = med[med > 500].index.min()
print("premier jour dont la médiane dépasse 500 :", premier, "| premier jour avec Z :",
      ↪ s.loc[s["en_utc"], "jour"].min())
print("lignes avec Z et montant < 500 :", int((s["en_utc"] & (s["brut"] < 500)).sum()), "| lignes sans
      ↪ Z et montant ≥ 500 :", int((~s["en_utc"] & (s["brut"] >= 500)).sum()))
```

```
premier jour dont la médiane dépasse 500 : 2025-09-15 | premier jour avec Z : 2025-09-15
lignes avec Z et montant < 500 : 23 | lignes sans Z et montant ≥ 500 : 7
```

La rupture de la médiane **commence le jour** de l'apparition du Z (affichage ci-dessus). Les deux conditions ne coïncident pas ligne à ligne : 23 lignes à Z ont un montant brut inférieur à 500 (de **petites** commandes : 5 € en centimes font 500) et 7 lignes sans Z dépassent 500 (de **grosses** commandes en euros) : un seuil sur le montant n'est **pas** un test de format. Et la coïncidence des dates ne **prouve** pas que les montants sont en centimes : elle montre que **deux changements arrivent le même jour**. Ce qui le prouve, c'est l'**autre source** : le total de chaque commande égale la somme de ses lignes (en euros) après division par cent (section 2.2.4), pour les 2 494 lignes à Z comme pour les autres.

Corrigé 2.9

```
m = lig.merge(cmd[["id_commande", "canal", "date_commande"]], on="id_commande", validate="m:1")
m = m[m["date_commande"].dt.year == 2025]
pan_c = m.groupby(["canal", "id_commande"])["montant"].sum().groupby("canal").mean()
lg = m.groupby("canal")["montant"].mean()
print(pd.DataFrame({"panier moyen (a)": pan_c, "montant moyen d'une ligne (b)":
      ↪ lg}).round(2).to_string())
```

	panier moyen (a)	montant moyen d'une ligne (b)
canal		
Boutique	103.08	44.48
Réseaux	102.44	44.43
Site	101.63	44.35

La moyenne des **lignes** (b) mesure le prix moyen d'un article vendu ; le **panier moyen** (a) est le chiffre d'affaires par commande. Une commande compte plusieurs lignes : (b) est plus petit, d'un facteur proche du nombre moyen de lignes par commande.

Corrigé 2.10

```
ca_j = ventes.groupby("date")["montant"].sum()
v7 = ca_j.rolling(7, min_periods=7).mean(); v1 = ca_j.rolling(7, min_periods=1).mean(); vc =
      ↪ ca_j.rolling(7, center=True, min_periods=7).mean()
print("valeurs vides :", int(v7.isna().sum()), int(v1.isna().sum()), int(vc.isna().sum()))
d15 = pd.Timestamp("2025-06-15")
print("15 juin :", round(v7[d15]), round(v1[d15]), round(vc[d15]))
```

```
valeurs vides : 6 0 6
15 juin : 3208 3208 2971
```

min_periods=7 : 6 vides au début ; min_periods=1 : aucun vide, mais les six premières valeurs reposent sur **moins** de sept jours (donc plus bruitées) ; centrée : 3 vides au début **et** 3 à la fin. Pour **décrire** le niveau du 15 juin, la moyenne **centrée** est la plus juste (elle regarde autant avant qu'après) ; pour **prévoir** le 16, on ne peut utiliser que la moyenne **arrière** (min_periods=7) : la centrée utiliserait des jours futurs, c'est une fuite d'information.

Corrigé 2.11

```
ventes["mois"] = ventes["date"].dt.strftime("%Y-%m")
large = ventes.pivot_table(index="categorie", columns="mois", values="montant", aggfunc="sum")
long = large.reset_index().melt(id_vars="categorie", var_name="mois", value_name="ca")
retour = long.pivot(index="categorie", columns="mois", values="ca")
print("aller-retour identique :", bool(retour.equals(large)), "| total large :",
      ↪ round(large.sum().sum(), 2), "| total long :", round(long["ca"].sum(), 2))
try:
    ventes.pivot(index="categorie", columns="mois", values="montant")
except ValueError as e:
    print("pivot direct :", e)
```

```
aller-retour identique : True | total large : 1164636.72 | total long : 1164636.72
pivot direct : Index contains duplicate entries, cannot reshape
```

pivot refuse parce que la table des ventes a **plusieurs lignes** par couple (catégorie, mois) : il ne sait pas **agréger**. pivot_table exige de dire comment (aggfunc), ce que fait aussi un tableau croisé dynamique de tableur, mais sans le dire.

Corrigé 2.12

```
n_nd = int(brut.loc[est_produit].iloc[:, 2:].map(lambda v: str(v).strip() in ("ND", "-")).sum().sum())
print("cellules ND ou - :", n_nd, "| stock moyen d'octobre, inconnus ignorés / à zéro :",
      ↪ round(m_ignore, 2), "/", round(m_zero, 2))
```

```
cellules ND ou - : 71 | stock moyen d'octobre, inconnus ignorés / à zéro : 47.59 / 44.82
```

Le remplacement par zéro **sous-estime** le stock moyen (voir la piste de l'application 2.8) : un inconnu n'est pas un zéro. Pour « rupture », le zéro est exact : on sait que le stock est nul, ce que la mention signifie.

Corrigé 2.13

La distance entre bougie et bougeoir est 3 : on garde boug, on insère e et o avant le i (2 insertions), le i est conservé, puis on remplace le e final de bougie par un r (1 remplacement). La table de programmation dynamique de la section 2.5.3 donne ce minimum dans sa dernière case.

```
from rapidfuzz.distance import Levenshtein
from rapidfuzz import fuzz
dist = Levenshtein.distance("bougie", "bougeoir")
print("distance :", dist, "| similarité :", round(100 * (1 - dist / max(len("bougie"),
      ↪ len("bougeoir"))), 1), "| ratio rapidfuzz :", round(fuzz.ratio("bougie", "bougeoir"), 1))
print("token_set_ratio « bougie nordique » / « nordique bougie » :",
      ↪ round(fuzz.token_set_ratio("bougie nordique", "nordique bougie")), "| ratio simple :",
      ↪ round(fuzz.ratio("bougie nordique", "nordique bougie"), 1))
```

```
distance : 3 | similarité : 62.5 | ratio rapidfuzz : 71.4
token_set_ratio « bougie nordique » / « nordique bougie » : 100 | ratio simple : 53.3
```

Corrigé 2.14

```
def cout_total(seuil, c_fp, c_fn):
    sel = S["score"] >= seuil
    fp = int((sel & ~S["vrai"]).sum()); fn = len(vraies) - int((sel & S["vrai"]).sum())
    return c_fp * fp + c_fn * fn
hypotheses = {"5 pour 1": (5, 1), "1 pour 1": (1, 1)}
opt = {nom: min(range(50, 101), key=lambda t: cout_total(t, *h)) for nom, h in hypotheses.items()}
print("seuil optimal :", opt, "| coûts :", {nom: cout_total(t, *hypotheses[nom]) for nom, t in
    ↪ opt.items()})
```

```
seuil optimal : {'5 pour 1': 83, '1 pour 1': 77} | coûts : {'5 pour 1': 76, '1 pour 1': 37}
```

Quand une **fusion erronée coûte cinq fois plus** qu'un doublon raté, le seuil optimal est **83** ; quand les deux coûts sont égaux, il est de **77**. Un coût plus élevé pour les fusions fautes pousse le seuil vers le haut : on accepte moins de paires et l'on tolère plus de doublons pour éviter de fusionner deux personnes. Un seuil est un **choix de coût**, pas un fait statistique.

Chapitre 3 : Qualité des données et réconciliation — exercices et applications

Ce chapitre du cahier prolonge le chapitre 3 du livre. Les **applications** sont de petites études guidées sur les fichiers désordonnés de la boutique (le CRM, l'export du site, les fichiers de la caisse, le catalogue du fournisseur, les montants saisis) ; vous les refaites pas à pas, puis vous prolongez dans les rubriques « À vous ». Les **exercices** (★ facile, ★★ moyen, ★★★ plus long) renvoient chacun à une section du livre ; leurs **corrigés** sont à la fin. Les données sont **simulées**, et la « vérité » est conservée dans les fichiers `verite_*` : **n'ouvrez la vérité qu'à la fin d'une étude**, pour juger votre travail, comme on ouvrirait le corrigé d'un problème.

Une première cellule charge les bibliothèques et les fichiers ; les autres reprennent les noms ainsi définis.

```
import os, sys, io, sqlite3, warnings
import numpy as np, pandas as pd
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import outils_ch03 as 0          # lecture de la caisse, rapprochements, fonctions utiles du chapitre

D = os.environ["DONNEES"]
lire = lambda nom, **kw: pd.read_csv(os.path.join(D, nom), **kw)
crm, site = lire("crm_clients.csv", dtype=str), lire("site_commandes.csv", dtype=str)
site_l, cmd, lig, prod = lire("site_lignes.csv"), lire("commandes.csv"), lire("lignes_commande.csv"),
↳ lire("produits.csv")
cat, mont = lire("catalogue_fournisseur.csv"), lire("montants_saisis.csv")
caisse, fichiers = 0.lire_caisse()
print(len(crm), "lignes de CRM |", len(site), "lignes d'export du site |", len(caisse), "lignes de
↳ caisse |", len(cat), "lignes de catalogue")
```

7140 lignes de CRM | 6259 lignes d'export du site | 12678 lignes de caisse | 118 lignes de catalogue

Applications

Application 3.1 — Mesurer la qualité du CRM (section 3.1)

Objectif. Calculer, pour le CRM, un indicateur par dimension, puis les lire comme un tableau de bord.

Étape 1 — La complétude. On compte, colonne par colonne, la part des cellules renseignées, puis la part des fiches « complètes » (e-mail, code postal et consentement présents) :

```
completude = (crm.notna().mean() * 100).round(1)
print(completude[completude < 100].to_string())
complet = crm[["email", "code_postal", "consentement_marketing"]].notna().all(axis=1)
print("fiches complètes :", int(complet.sum()), "sur", len(crm), f"({100 * complet.mean():.1f} %)")
```

```
email          96.8
code_postal    94.1
consentement_marketing 69.7
fiches complètes : 4557 sur 7140 (63.8 %)
```

Étape 2 — La validité. On applique six règles de forme et l'on garde, pour chacune, le nombre de lignes en échec. Les colonnes absentes ne comptent pas comme échec de forme (elles relèvent de la complétude) :

```
def echecs_format(serie, motif):
    return serie.notna() & ~serie.str.fullmatch(motif, na=False)

regles = {
    "e-mail mal formé": echecs_format(crm["email"], 0.RE_EMAIL),
    "code postal ≠ 5 chiffres": echecs_format(crm["code_postal"], 0.RE_CP),
    "consentement hors {oui, non}": crm["consentement_marketing"].notna() &
        ~crm["consentement_marketing"].isin(["oui", "non"]),
    "ville non reconnue": 0.canonique_ville(crm["ville"]).isna(),
    "naissance illisible": 0.parse_naissance(crm["date_naissance"]).isna(),
    "ligne de test": crm["email"].eq("test@example.com"),
}
print(0.resume_controles(regles, len(crm)).to_string(index=False))
```

```
          controle  echecs  taux_pct
e-mail mal formé      95      1.3
code postal ≠ 5 chiffres 199      2.8
consentement hors {oui, non} 2428    34.0
ville non reconnue     634      8.9
naissance illisible    666      9.3
ligne de test         140      2.0
```

Étape 3 — L'unicité. Une ligne est redondante si son e-mail normalisé (espaces retirés, minuscules) est déjà apparu plus haut :

```
cle = crm["email"].str.strip().str.lower()
redondante = cle.notna() & cle.ne("test@example.com") & cle.duplicated(keep="first")
print("lignes redondantes par e-mail :", int(redondante.sum()), "(", round(100 * redondante.mean(),
    ↪ 1), "% du fichier)")
```

```
lignes redondantes par e-mail : 677 ( 9.5 % du fichier )
```

Lecture. La complétude est tirée vers le bas par le consentement, la validité par le champ consentement et par la ville (des écritures différentes mais acceptables : un contrôle trop strict qui appelle une **normalisation**), l'unicité mesurée est partielle (corrigé 3.3).

À vous. Ajoutez un septième indicateur de validité : « le téléphone se normalise en dix chiffres » (utilisez `0.telephone_normalise`). Combien de lignes échouent ? Que vous apprend ce résultat sur la **sensibilité** de l'indicateur ?

Application 3.2 — Écrire les contrôles du CRM et les rejouer en SQL (section 3.2)

Objectif. Passer des mesures aux **contrôles** : une gravité, un rapport trié, puis la même règle en SQL.

Étape 1 — Le rapport. On associe à chaque règle de l'application précédente une gravité décidée avant de regarder les résultats :

```
gravite = {"e-mail mal formé": "erreur", "code postal ≠ 5 chiffres": "erreur", "consentement hors
↳ {oui, non}": "avertissement",
    "ville non reconnue": "avertissement", "naissance illisible": "erreur", "ligne de test":
    ↳ "erreur"}
rapport = 0.resume_controles(regles, len(crm))
rapport["gravite"] = rapport["controle"].map(gravite)
print(rapport.sort_values(["gravite", "echecs"], ascending=[True, False]).to_string(index=False))
```

	controle	echecs	taux_pct	gravite
consentement hors {oui, non}		2428	34.0	avertissement
ville non reconnue		634	8.9	avertissement
naissance illisible		666	9.3	erreur
code postal ≠ 5 chiffres		199	2.8	erreur
ligne de test		140	2.0	erreur
e-mail mal formé		95	1.3	erreur

Étape 2 — La quarantaine. Les lignes de gravité « erreur » sont écartées vers une table à part ; le reste continue son chemin :

```
masque_erreur = np.zeros(len(crm), bool)
for nom, m in regles.items():
    if gravite[nom] == "erreur":
        masque_erreur |= m.values
quarantaine, propres = crm[masque_erreur], crm[~masque_erreur]
print("en quarantaine :", len(quarantaine), "| conservées :", len(propres))
```

en quarantaine : 1070 | conservées : 6070

Étape 3 — Les contraintes et les requêtes de contrôle. On charge le CRM dans une base en mémoire. Une table avec des contraintes refuse l'insertion d'une ligne invalide :

```
con = sqlite3.connect(":memory:")
crm.to_sql("crm", con, index=False)
con.execute("CREATE TABLE clients_propres (id_crm INTEGER PRIMARY KEY, code_postal TEXT CHECK
↳ (code_postal IS NULL OR length(code_postal) = 5))")
for cp in ["75012", "1234"]:
    try:
        con.execute("INSERT INTO clients_propres VALUES (NULL, ?)", (cp,)); print(cp, "accepté")
    except sqlite3.IntegrityError as e:
        print(cp, "refusé :", e)
```

75012 accepté

1234 refusé : CHECK constraint failed: code_postal IS NULL OR length(code_postal) = 5

```
SELECT COUNT(*) AS codes_postaux_invalides FROM crm WHERE code_postal IS NOT NULL AND
↳ length(code_postal) <> 5;
```

codes_postaux_invalides
199

À vous. Écrivez la requête SQL qui compte les lignes dont le consentement est présent mais hors de la liste oui/non, puis vérifiez qu'elle redonne le chiffre de pandas.

Application 3.3 — Plage ou dépendance ? (section 3.2.3)

Objectif. Comparer trois règles pour repérer les erreurs de saisie des 6 000 lignes de montants_saisis.csv, en s'aidant de la vérité pour juger chaque règle.

Étape 1 — Les trois règles. Une borne fixe, la règle statistique des 1,5 écart interquartile, et la règle de dépendance entre colonnes :

```
q1, q3 = mont["montant"].quantile([0.25, 0.75])
r_fixe = mont["montant"] > 1000
r_iqr = mont["montant"] > q3 + 1.5 * (q3 - q1)
r_dep = (mont["montant"] <= 0) | (mont["montant"] > mont["quantite"] * mont["prix_unitaire"] + 0.01)
verite = lire("verite_montants.csv")
vrai = mont.merge(verite, on="id_ligne")["anomalie"].notna()
for nom, r in {"borne fixe > 1000 €": r_fixe, "1,5 écart interquartile": r_iqr, "dépendance":
    ↪ r_dep}.items():
    print(f"{nom:26s} signalées {int(r.sum()):4d} | vraies {int((r & vrai).sum()):3d} | fausses
    ↪ alertes {int((r & ~vrai).sum()):3d}")
```

borne fixe > 1000 €	signalées	28	vraies	28	fausses alertes	0
1,5 écart interquartile	signalées	403	vraies	52	fausses alertes	351
dépendance	signalées	85	vraies	85	fausses alertes	0

Étape 2 — Qui trouve quoi ? On regarde, par type d'anomalie, la part trouvée par chaque règle :

```
tab = mont.merge(verite, on="id_ligne")
tab["fixe"], tab["iqr"], tab["dep"] = r_fixe.values, r_iqr.values, r_dep.values
res = tab[tab["anomalie"].notna()].groupby("anomalie")[["fixe", "iqr",
    ↪ "dep"]].mean().mul(100).round(0)
print(res.to_string())
```

anomalie	fixe	iqr	dep
decimale_x10	9.0	96.0	100.0
decimale_x100	81.0	100.0	100.0
placeholder_9999	100.0	100.0	100.0
signe_inverse	0.0	0.0	100.0
zero	0.0	0.0	100.0

Lecture. La règle de dépendance retrouve **tous** les types d'anomalie, sans fausse alerte, parce qu'elle compare le montant à la quantité et au prix. La règle statistique ne voit jamais les montants négatifs ni nuls (elle regarde vers le haut) et noie ses vraies alertes dans des centaines de grosses commandes légitimes. La borne fixe n'a aucune fausse alerte, mais elle ne trouve que les valeurs énormes : presque aucune décimale décalée de ×10 (le tableau donne la part trouvée par type), et rien du côté des signes et des zéros. Une règle de plage **ne voit que le haut de la distribution**.

À vous. Quelle valeur de borne fixe donnerait autant de vraies alertes que la règle des 1,5 écart interquartile ? Combien de fausses alertes de plus ?

Application 3.4 — Détecter une rupture dans une série (section 3.2.5)

Objectif. Repérer le jour où l'export du site change d'unité, sans connaître la date, et vérifier que la caisse n'a pas de rupture comparable.

Étape 1 — Le montant médian par jour.

```
site["date"] = pd.to_datetime(site["created_at"].str.replace("Z", ""))
    ↪ format="ISO8601").dt.normalize()
site["t"] = site["total"].map(0.montant_site_en_nombre)
jour = site.groupby("date")["t"].median()
rapport_jour = jour / jour.rolling(7).median().shift(1)
```

```
print(rapport_jour[rapport_jour > 10].head(3).round(0).to_string())
```

```
date
2025-09-15    147.0
2025-09-16     86.0
2025-09-17     46.0
```

Étape 2 — Un seuil, plusieurs fenêtres. On vérifie que la date trouvée ne dépend pas de la fenêtre glissante :

```
for fen in (3, 7, 14):
    r_ = jour / jour.rolling(fen).median().shift(1)
    print("fenêtre", fen, "jours : premier jour détecté", r_[r_ > 10].index[0].date())
```

```
fenêtre 3 jours : premier jour détecté 2025-09-15
fenêtre 7 jours : premier jour détecté 2025-09-15
fenêtre 14 jours : premier jour détecté 2025-09-15
```

Étape 3 — La même question sur la caisse. On compare le montant médian par ticket d'un mois à l'autre :

```
tickets = caisse.dropna(subset=["montant"]).groupby(["fichier", "ticket"])["montant"].sum()
med = tickets.groupby("fichier").median()
print((med / med.shift(1)).round(2).describe()[["min", "max"]].to_string())
```

```
min    0.85
max    1.12
```

À vous. Que se passerait-il si la rupture était une division par dix (des euros devenus des dizaines d'euros) ? Votre seuil de 10 la détecterait-il ? Comment le choisir ?

Application 3.5 — Réconcilier le site avec la base (section 3.3.3)

Objectif. Refaire la cascade du chiffre d'affaires du site, dans l'ordre : format, lignes parasites, définitions.

Étape 1 — Compter et sommer.

```
t_export = site["total"].map(0.montant_site_en_nombre)
base_site = cmd[(cmd["canal"] == "Site") & (cmd["date_commande"] >= "2025-01-01")]
ca_base = base_site["id_commande"].map(lig.groupby("id_commande")["montant"].sum()).sum()
print("lignes :", len(site), "contre", len(base_site), "| somme :", round(t_export.sum(), 2),
      ↪ "contre", round(ca_base, 2))
```

```
lignes : 6259 contre 6078 | somme : 25012599.35 contre 617715.45
```

Étape 2 — Les unités, les tests, les copies.

```
apres = site["created_at"].str.endswith("Z")
t_eur = t_export.where(~apres, t_export / 100)
test = site["customer_email"].eq("test@example.com")
copie = site["order_ref"].duplicated(keep="first") & ~test
etapes = {"unité": t_eur.sum() - t_export.sum(), "tests": -t_eur[test].sum(), "copies":
      ↪ -t_eur[copie].sum()}
print({k: round(float(v), 2) for k, v in etapes.items()})
print("écart restant :", abs(round(t_export.sum() + sum(etapes.values()) - ca_base, 2)))
```

```
{'unité': -24382796.13, 'tests': -36.24, 'copies': -12051.53}
écart restant : 0.0
```

Étape 3 — Les totaux journaliers. On vérifie que, jour par jour, l'export nettoyé et la base donnent la même chose (et pas seulement l'année) :

```
propre = site[~test & ~copie].assign(eur=t_eur[~test & ~copie])
par_jour_site = propre.groupby("date")["eur"].sum()
base_j = base_site.assign(m=base_site["id_commande"].map(lig.groupby("id_commande")["montant"].sum,
↳ ())).groupby(pd.to_datetime(base_site["date_commande"]))["m"].sum()
print("jours comparés :", len(base_j), "| écart journalier maximal :", round(float((par_jour_site -
↳ base_j).abs().max()), 4), "€")
```

```
jours comparés : 365 | écart journalier maximal : 0.0 €
```

À vous. Retirez de l'export les commandes annulées et dites quel chiffre vous annonceriez à la gérante, avec quelle phrase de définition.

Application 3.6 — Réconcilier la caisse avec la base (section 3.3.4)

Objectif. Lire douze fichiers de formats différents, puis expliquer l'écart entre la somme des lignes et le total que la caisse imprime.

Étape 1 — Les formats. La fonction `0.lire_caisse_fichier` renvoie, pour un fichier, les lignes, le total affiché et les caractéristiques du format :

```
for nom in ["caisse_2025-03.csv", "caisse_2025-08.csv", "caisse_2025-11.csv"]:
    df, total, meta = 0.lire_caisse_fichier(os.path.join(D, "caisse", nom))
    print(nom, "|", meta["encodage"], "| séparateur", repr(meta["separateur"]), "|",
↳ meta["colonnes"], "colonnes |", len(df), "lignes | total", total)
```

```
caisse_2025-03.csv | cp1252 | séparateur ';' | 8 colonnes | 916 lignes | total 39038.9
caisse_2025-08.csv | utf-8-sig | séparateur ';' | 8 colonnes | 814 lignes | total 42873.5
caisse_2025-11.csv | utf-8-sig | séparateur ',' | 9 colonnes | 1452 lignes | total 60586.62
```

Étape 2 — Les trois temps.

```
base_b = lig.merge(cmd[["id_commande", "date_commande", "canal"]], on="id_commande")
base_b = base_b[(base_b["canal"] == "Boutique") & (base_b["date_commande"] >= "2025-01-01")]
print("compter :", len(caisse), "contre", len(base_b))
print("sommer :", round(caisse["montant"].sum(), 2), "(lu) |", round(fichiers["total_affiche"].sum(),
↳ 2), "(affiché) |", round(base_b["montant"].sum(), 2), "(base)")
```

```
compter : 12678 contre 12611
sommer : 547896.42 (lu) | 560973.91 (affiché) | 560973.91 (base)
```

Étape 3 — Le rapprochement ligne à ligne. La clé est ticket + article + quantité + prix + rang :

```
prod_nom = prod[["id_produit", "nom_produit"]]
r = 0.rapprocher_caisse_base(caisse, cmd, lig, prod_nom)
print(r["_merge"].value_counts().to_string())
copies = r[r["_merge"] == "left_only"]
vides = r[(r["_merge"] == "both") & r["montant"].isna()]
print("copies :", len(copies), "| montants vides retrouvés en base :", len(vides))
```

```
_merge
both          12611
left_only      67
right_only      0
copies : 67 | montants vides retrouvés en base : 398
```

Étape 4 — La cascade.

```
qp = vides["qte"] * vides["prix_unitaire"]
etapes = [caisse["montant"].sum(), -copies["montant"].sum(), qp.sum(), -(qp -
↳ vides["montant_base"]).sum()]
print([round(float(e), 2) for e in etapes], "| total obtenu :", round(sum(etapes), 2), "| base :",
↳ round(base_b["montant"].sum(), 2))
```

```
[547896.42, -3126.29, 16518.59, -314.81] | total obtenu : 560973.91 | base : 560973.91
```

À vous. Refaites la cascade **mois par mois** (utilisez la colonne fichier) et construisez un tableau de douze lignes où l'écart restant vaut zéro partout.

Application 3.7 — Réconcilier le catalogue du fournisseur (section 3.3.5)

Objectif. Rapprocher deux listes sans clé commune, et mesurer l'effet de la **tolérance de prix**.

Étape 1 — La clé « nom normalisé ».

```
cle_nom = lambda s: s.map(lambda x: 0.sans_accents(str(x)).lower().strip())
cat["cle"], prod["cle"] = cle_nom(cat["designation"]), cle_nom(prod["nom_produit"])
m = cat.merge(prod[["id_produit", "cle", "cout_achat"]], on="cle", how="left")
m["ecart_prix"] = (m["prix_achat_ht"] / m["cout_achat"] - 1).abs()
print(len(m), "lignes après jointure pour", len(cat), "lignes de catalogue ;",
↳ int(m["id_produit"].isna().sum()), "sans correspondance")
```

```
196 lignes après jointure pour 118 lignes de catalogue ; 40 sans correspondance
```

Étape 2 — La tolérance de prix. On fait varier le seuil et l'on compte appariés, ambigus et non appariés :

```
for tol in (0.01, 0.035, 0.10):
    ok = m[m["ecart_prix"] <= tol]
    amb = ok[ok["code_fournisseur"].duplicated(keep=False)]
    print(f"tolérance {tol:5.1%} : appariés {ok['code_fournisseur'].nunique():3d} | codes ambigus
↳ {ok.loc[amb, 'code_fournisseur'].nunique():2d} | non appariés {len(cat) -
↳ ok['code_fournisseur'].nunique():3d}")
```

```
tolérance 1.0% : appariés 22 | codes ambigus 0 | non appariés 96
tolérance 3.5% : appariés 78 | codes ambigus 2 | non appariés 40
tolérance 10.0% : appariés 78 | codes ambigus 5 | non appariés 40
```

Étape 3 — La vérité. Avec la vérité, on mesure la **justesse** des appariements sans ambiguïté :

```
vprod = lire("verite_produits.csv")
ok = m[m["ecart_prix"] <= 0.035]
sans_amb = ok[~ok["code_fournisseur"].duplicated(keep=False)].merge(vprod, on="code_fournisseur",
↳ suffixes=("", "_vrai"))
print("appariements sans ambiguïté :", len(sans_amb), "| exacts :", int((sans_amb["id_produit"] ==
↳ sans_amb["id_produit_vrai"]).sum()))
```

```
appariements sans ambiguïté : 76 | exacts : 76
```

À vous. À 10 % de tolérance, y a-t-il des appariements **faux** parmi ceux que vous croyez certains ? Qu'en concluez-vous sur le choix d'un seuil trop large ?

Application 3.8 — Tolérances, règles et rapport d'exceptions (section 3.4)

Objectif. Évaluer cinq règles de réconciliation avant et après nettoyage, puis construire le rapport d'exceptions par propriétaire.

Étape 1 — Les règles. On reprend les objets des applications précédentes (`t_export`, `ca_base`, `propre`, `caisse`, `base_b`, `copies`, `vides`) :

```
def dans_la_tolerance(a, b, abs_tol=0.01, rel_tol=0.0):
    return abs(a - b) <= max(abs_tol, rel_tol * abs(b))

regles_r = [{"R1", "CA du site", t_export.sum(), ca_base, propre["eur"].sum(), ("R2", "Commandes du
↪ site", len(site), len(base_site), len(propre)),
            ("R3", "CA de la caisse", caisse["montant"].sum(), base_b["montant"].sum(), sum(etapes)),
↪ ("R4", "Lignes de la caisse", len(caisse), len(base_b), len(caisse) - len(copies))}]
for i, n, a, b, ap in regles_r:
    print(i, f"{n:20s}", "avant :", "OK " if dans_la_tolerance(a, b, 1.0) else "ÉCART", "| après :",
↪ "OK" if dans_la_tolerance(ap, b, 1.0) else "ÉCART")
```

R1 CA du site	avant : ÉCART après : OK
R2 Commandes du site	avant : ÉCART après : OK
R3 CA de la caisse	avant : ÉCART après : OK
R4 Lignes de la caisse	avant : ÉCART après : OK

Étape 2 — Les exceptions. On rassemble les exceptions avec leur propriétaire et on les compte :

```
def exceptions(source, nature, ref, montant, proprietaire):
    return pd.DataFrame({"source": source, "nature": nature, "reference": ref, "montant": montant,
↪ "proprietaire": proprietaire, "statut": "à traiter"})

exc = pd.concat([exceptions("Caisse", "copie de scan", copies["fichier"] + " / " + copies["ticket"],
↪ copies["montant"], "équipe caisse"),
                exceptions("Caisse", "montant vide", vides["fichier"] + " / " + vides["ticket"],
↪ vides["montant_base"], "équipe caisse"),
                exceptions("Site", "copie d'export", site.loc[copie, "order_ref"], t_eur[copie],
↪ "équipe web"),
                exceptions("Site", "commande de test", site.loc[test, "order_ref"], t_eur[test],
↪ "équipe web")], ignore_index=True)
print(exc.groupby("proprietaire").agg(exceptions=("reference", "count"), montant=("montant",
↪ "sum")).round(2).to_string())
```

	exceptions	montant
proprietaire		
équipe caisse	465	19330.07
équipe web	181	12087.77

À vous. Ajoutez la règle R5 « taux d'appariement du catalogue $\geq 90\%$ » et les exceptions du catalogue (propriétaire : « achats »). Quelle équipe a le plus d'exceptions à traiter ? Laquelle en a pour le plus d'argent ?

Application 3.9 — pandera et Great Expectations sur l'export du site (section 3.5)

Objectif. Déclarer les contrôles de l'export du site avec pandera, puis les rejouer avec Great Expectations, et comparer les comptes aux vôtres.

Étape 1 — Le schéma pandera.

```
import pandera.pandas as pa
schema_site = pa.DataFrameSchema({
    "order_ref": pa.Column(str, unique=True),
    "status": pa.Column(str, pa.Check.isin(["paid", "cancelled"])),
    "currency": pa.Column(str, pa.Check.isin(["EUR"])),
```

```

    "customer_email": pa.Column(str, pa.Check(lambda s: s != "test@example.com", name="pas un e-mail
    ↳ de test")),
})
try:
    schema_site.validate(site[["order_ref", "status", "currency", "customer_email"]], lazy=True)
except pa.errors.SchemaErrors as e:
    echecs = e.failure_cases
print(echecs.groupby(["column", "check"])["index"].nunique().to_string())

```

column	check	
currency	isin(['EUR'])	1272
customer_email	pas un e-mail de test	60
order_ref	field_uniqueness	242
status	isin(['paid', 'cancelled'])	2444

Étape 2 — La même chose avec Great Expectations.

```

import great_expectations as gx
E = gx.expectations
with 0.silencieux():
    ctx = gx.get_context(mode="ephemeral")
    lot_def = ctx.data_sources.add_pandas("boutique").add_dataframe_asset("site",
    ↳ ).add_batch_definition_whole_dataframe("tout")
    suite = ctx.suites.add(gx.ExpectationSuite(name="site"))
    for e in [E.ExpectColumnValuesToBeUnique(column="order_ref"),
    ↳ E.ExpectColumnValuesToBeInSet(column="status", value_set=["paid", "cancelled"]),
    E.ExpectColumnValuesToBeInSet(column="currency", value_set=["EUR"])] :
        suite.add_expectation(e)
    res = ctx.validation_definitions.add(gx.ValidationDefinition(name="site", data=lot_def,
    ↳ suite=suite)).run(batch_parameters={"dataframe": site})
for x in res.results:
    print(f"{x.expectation_config.type:40s} {'OK' if x.success else 'KO'} | anormales :
    ↳ {x.result['unexpected_count']}")

```

expect_column_values_to_be_unique	KO anormales : 242
expect_column_values_to_be_in_set	KO anormales : 2444
expect_column_values_to_be_in_set	KO anormales : 1272

Lecture. Les deux outils donnent les **mêmes** comptes pour le statut (2444) et la devise (1272) ; le test de l'e-mail (60 lignes) n'existe que dans pandera (la règle de Great Expectations n'a pas été écrite). Pour l'unicité, ils comptent 242 lignes : les **deux** membres de chaque paire de numéros identiques, alors que `deduplicated(keep="first")` n'en compte que 121 (la copie). Même règle, autre **convention de comptage** : avant de comparer un nombre d'échecs entre deux outils, on vérifie ce qu'il compte.

À vous. Les « statuts » en échec sont-ils de vraies erreurs ou des écritures différentes (casse) d'une même valeur ? Modifiez la règle (ou normalisez avant) pour ne signaler que ce qui est réellement erroné.

Exercices

Exercice 3.1 ★ — La complétude du CRM (section 3.1.2)

Calculez la complétude de chaque colonne du CRM, puis la part des fiches où l'e-mail, le téléphone et la ville sont **tous** renseignés. Pourquoi ce taux est-il égal à celui de l'e-mail seul ?

Exercice 3.2 ★ — Valider un téléphone (section 3.1.3)

Après suppression de tous les séparateurs, un téléphone est valide s'il se compose de dix chiffres et commence par 0 (les numéros écrits +99 sont ramenés à 0). Combien de téléphones sont invalides ? Que dit ce chiffre sur l'intérêt de cet indicateur ? Quelle règle supplémentaire détecterait les lignes de test ?

Exercice 3.3 ★★ — Choisir une clé d'unicité (section 3.1.3 et 3.1.8)

On veut mesurer la redondance du CRM. Comparez trois clés : l'e-mail normalisé, le téléphone normalisé, et « e-mail **ou** téléphone ». Pour chacune, combien de lignes sont redondantes ? Ouvrez ensuite la vérité (`verite_crm.csv`) : combien de lignes redondantes la vérité compte-t-elle, et combien de groupes de lignes mélangent des **clients différents** pour chaque clé ?

Exercice 3.4 ★★ — Le tableau de bord avant et après correction (section 3.1.7)

Dans le tableau de bord du livre, l'exactitude et la cohérence de l'export du site sont à 59,8 %. Recalculez l'indicateur « total conforme à la base » après avoir corrigé l'unité (division par cent après le 15 septembre) et retiré les commandes de test et les copies. Que devient-il ?

Exercice 3.5 ★ — La moyenne qui cache (section 3.1.8)

Pour chaque source, comparez la **moyenne** des indicateurs de validité et le **plus mauvais** indicateur de validité (utilisez `0.indicateurs_qualite`). Dans quel cas la moyenne rassure-t-elle à tort ?

Exercice 3.6 ★ — Trop strict ou normalisable ? (section 3.2.2)

La règle « la devise du site vaut EUR » échoue pour une partie des lignes. Combien ? Montrez que ces échecs sont de simples écritures différentes (eur, €) en normalisant avant de contrôler, et dites quel contrôle vous garderiez en production.

Exercice 3.7 ★★ — Un contrôle de dépendance (section 3.2.3)

Pour les fichiers d'octobre à décembre de la caisse (qui contiennent la colonne de remise), vérifiez que le montant égale $\text{quantité} \times \text{prix} \times (1 - \text{remise} / 100)$ à un centime près, sur les lignes où le montant est présent. Pourquoi ce contrôle est-il impossible pour les fichiers de janvier à septembre ?

Exercice 3.8 ★★ — La caisse a-t-elle une rupture ? (section 3.2.5)

Calculez le panier moyen (par ticket) par mois dans la caisse. Quel est le plus grand rapport entre deux mois consécutifs ? Ce rapport ressemble-t-il à celui du site, le 15 septembre ? Comment l'expliquez-vous ?

Exercice 3.9 ★★ — Des contraintes SQL (section 3.2.7)

Chargez le CRM dans une base SQLite en mémoire. Écrivez (1) une table avec des contraintes CHECK sur le code postal et le consentement ; (2) la requête qui renvoie les e-mails présents plus d'une fois (après `lower(trim(...))`), avec leur nombre d'occurrences ; (3) la requête qui compte les commandes du site sans aucune ligne.

Exercice 3.10 ★★ — La cascade de la caisse en décembre (section 3.3.4)

Reprenez la cascade de la section 3.3.4, mais pour le seul mois de **décembre**. Quels sont le nombre de copies, le nombre de montants vides et l'écart restant ?

Exercice 3.11 ★★★ — La cascade du site en septembre (section 3.3.3)

Le mois de septembre est coupé en deux par le changement d'unité. Faites la cascade du chiffre d'affaires du site pour **septembre seul** : combien de commandes avant et après le 15 ? Quel montant l'erreur d'unité ajoute-t-elle à la somme brute du mois ? L'écart restant est-il nul ?

Exercice 3.12 ★★★ — Tolérance et exceptions par mois (section 3.4)

Pour la caisse, calculez par mois l'écart relatif entre la somme des montants lus et le total affiché. Avec une tolérance relative de 0,5 %, combien de mois passent **avant** nettoyage ? Après nettoyage (copies retirées, vides complétés) ? Construisez enfin le tableau d'exceptions par fichier (copies et montants vides) et dites quel mois est le plus chargé.

Corrigés

Corrigé 3.1

```
print((crm.notna().mean() * 100).round(1)[lambda s: s < 100].to_string())
f = crm[["email", "telephone", "ville"]].notna().all(axis=1)
print("e-mail, téléphone et ville renseignés :", round(100 * f.mean(), 1), "% | e-mail seul :",
      ↪ round(100 * crm["email"].notna().mean(), 1), "%")
```

```
email          96.8
code_postal    94.1
consentement_marketing 69.7
e-mail, téléphone et ville renseignés : 96.8 % | e-mail seul : 96.8 %
```

Le téléphone et la ville sont renseignés à 100 % : la fiche « complète » sur ces trois colonnes ne dépend donc que de l'e-mail, et les deux taux sont égaux. Un indicateur composé n'est jamais plus fin que sa colonne la plus lacunaire.

Corrigé 3.2

```
tel = 0.telephone_normalise(crm["telephone"])
print("téléphones invalides :", int(tel.isna().sum()), "| numéros nuls :", int((tel ==
      ↪ "0000000000").sum()))
```

```
téléphones invalides : 0 | numéros nuls : 140
```

Aucun téléphone n'est invalide : **toutes** les écritures se normalisent, y compris 0000000000, la valeur bidon des lignes de test. Un indicateur de validité de forme ne détecte pas les valeurs absurdes mais bien formées. Pour détecter les lignes de test, il faut une **règle de contenu** (« le téléphone n'est pas une suite de zéros », « l'e-mail n'est pas celui de l'équipe »), ce qui relève de la validité **métier**, au-delà de la forme.

Corrigé 3.3

```
v = lire("verite_crm.csv").assign(id_crm=lambda d: d["id_crm"].astype(str))
reel = crm[crm["email"].ne("test@example.com")].copy()
reel["tel"] = 0.telephone_normalise(reel["telephone"]); reel["cle"] =
↳ reel["email"].str.strip().str.lower()
reel = reel.merge(v[["id_crm", "id_client"]], on="id_crm")
print("lignes redondantes selon la vérité :", int(v["est_doublon"].sum()))
for nom, col in {"e-mail": "cle", "téléphone": "tel"}.items():
    k = reel[col].dropna()
    melanges = int((reel.loc[k.index].groupby(col)["id_client"].nunique() > 1).sum())
    print(f"{nom:10s} redondantes {int(k.duplicated().sum()):4d} | groupes qui mélangent des clients :
↳ {melanges}")
ou = (reel["cle"].notna() & reel["cle"].duplicated()) | (reel["tel"].notna() &
↳ reel["tel"].duplicated())
print("e-mail ou téléphone : redondantes", int(ou.sum()))
```

```
lignes redondantes selon la vérité : 1000
e-mail      redondantes 677 | groupes qui mélangent des clients : 3
téléphone   redondantes 1000 | groupes qui mélangent des clients : 0
e-mail ou téléphone : redondantes 1003
```

La vérité compte 1 000 lignes redondantes. Le **téléphone** en trouve 1 000, soit le compte exact, sans aucun groupe qui mélange des clients ; l'**e-mail** n'en trouve que 677, et 3 de ses groupes mélangent des clients différents (des homonymes dont l'adresse coïncide) ; « e-mail ou téléphone » en trouve 1 003, soit 3 de plus que la vérité : ce sont ces homonymes. Le meilleur indicateur est celui dont la clé est **la plus stable d'une copie à l'autre** : ici le numéro de téléphone (les copies le conservent, à la mise en forme près), pas l'e-mail (absent ou abîmé dans environ une copie sur trois). Notez qu'un **compte** égal à la vérité ne prouve pas que chaque ligne est la bonne ; ici le contrôle par groupe (aucun mélange de clients) le confirme.

Corrigé 3.4

```
t_export = site["total"].map(0.montant_site_en_nombre)
apres = site["created_at"].str.endswith("Z")
t_eur = t_export.where(~apres, t_export / 100)
idc = site["order_ref"].str.extract(r"WEB-(\d{6})")[0].astype(float)
base_t = idc.map(lig.groupby("id_commande")["montant"].sum())
garde = ~site["customer_email"].eq("test@example.com") & ~site["order_ref"].duplicated()
for nom, serie in {"lecture brute": t_export, "unité corrigée": t_eur}.items():
    ok = ((serie - base_t).abs() <= 0.01)[garde & base_t.notna()]
    print(f"{nom:16s} commandes exactes : {100 * ok.mean():.1f} %")
```

```
lecture brute      commandes exactes : 59.9 %
unité corrigée     commandes exactes : 100.0 %
```

Une fois l'unité corrigée, **toutes** les commandes sont exactes (100 %). Le tableau de bord à 59,8 % ne mesurait pas un défaut diffus mais **un seul défaut de lot** : un changement d'unité sur 40 % des lignes.

Corrigé 3.5

```
ind = 0.indicateurs_qualite(crm, site, lire("site_lignes.csv"), caisse, fichiers, cmd, lig, prod, REF)
val = ind[ind["dimension"] == "Validité"].groupby("source")["valeur"].agg(["mean", "min"]).round(1)
print(val.to_string())
```

	mean	min
source		
CRM	95.0	90.3
Site	79.9	61.0

Pour le site, la moyenne des indicateurs de validité (autour de 80 %) cache un indicateur à 61 % (le statut en minuscules) et un autre à 99 % (les commandes de test) : la moyenne rassure à tort sur une colonne qui échoue pour **quatre lignes sur dix**. On regarde donc **chaque indicateur**, en particulier le plus mauvais.

Corrigé 3.6

```
ecart = ~site["currency"].eq("EUR")
norm = site["currency"].str.upper().replace({"€": "EUR"})
print("devises ≠ EUR :", int(ecart.sum()), "| après normalisation :", int((~norm.eq("EUR")).sum()))
```

```
devises ≠ EUR : 1272 | après normalisation : 0
```

La règle stricte échoue pour environ un cinquième des lignes, mais **aucune** n'est une vraie erreur : eur et € sont deux écritures d'euros. En production, on garde le contrôle **après** normalisation (« la devise normalisée vaut EUR »), ce qui échoue seulement si une autre devise apparaît.

Corrigé 3.7

```
trim4 = caisse[caisse["fichier"] >= "caisse_2025-10.csv"].dropna(subset=["montant"])
attendu = (trim4["qte"] * trim4["prix_unitaire"] * (1 - trim4["remise_pct"] / 100)).round(2)
echec = (trim4["montant"] - attendu).abs() > 0.011
print("lignes testées :", len(trim4), "| en échec :", int(echec.sum()))
```

```
lignes testées : 4135 | en échec : 0
```

Aucune ligne n'échoue : le montant est cohérent avec la quantité, le prix et la remise. Pour janvier à septembre, la colonne de remise **n'existe pas** dans le fichier : on ne peut contrôler que l'inégalité $\text{montant} \leq \text{quantité} \times \text{prix}$, pas l'égalité. Un contrôle dépend de l'**information disponible** dans la source.

Corrigé 3.8

```
pm = caisse.dropna(subset=["montant"]).groupby(["fichier",
↳ "ticket"])["montant"].sum().groupby("fichier").mean()
r_ = (pm / pm.shift(1)).dropna()
print("panier moyen par mois :", pm.round(1).tolist())
print("rapport maximal entre deux mois :", round(float(max(r_.max(), 1 / r_.min())), 2))
```

```
panier moyen par mois : [92.9, 99.6, 94.9, 106.1, 102.3, 103.4, 108.6, 122.4, 100.9, 102.6, 95.9, 98.4]
rapport maximal entre deux mois : 1.21
```

Le rapport maximal est de 1,21 (de l'ordre de vingt pour cent, les variations de saison et de mix de produits) : **aucune rupture** comparable à celle du site (un rapport de l'ordre de 100). C'est

cohérent avec ce que nous avons vu : les trois formats de la caisse changent la **forme** (séparateur, décimale) mais pas le **sens** des montants, qui restent en euros.

Corrigé 3.9

```
con = sqlite3.connect(":memory:")
crm.to_sql("crm", con, index=False); site.to_sql("site", con, index=False);
↳ site_l.to_sql("site_lignes", con, index=False)
con.execute("CREATE TABLE propres (id INTEGER PRIMARY KEY, code_postal TEXT CHECK (code_postal IS
↳ NULL OR length(code_postal) = 5), consentement TEXT CHECK (consentement IN ('oui', 'non'))))"
print(con.execute("SELECT lower(trim(email)), COUNT(*) FROM crm WHERE email IS NOT NULL AND email <>
↳ 'test@example.com' GROUP BY 1 HAVING COUNT(*) > 1 ORDER BY 2 DESC LIMIT 3").fetchall())
print(con.execute("SELECT COUNT(*) FROM site s LEFT JOIN site_lignes l ON l.order_ref = s.order_ref
↳ WHERE l.order_ref IS NULL").fetchone())
```

```
[('zomonin.valgren@example.org', 3), ('yulono.wenmer@mail.example', 3),
↳ ('tamonia.nevtier21@courrier.test', 3)]
(60,)
```

La première requête renvoie les adresses les plus répétées (trois fois chacune) ; la seconde compte les commandes sans ligne : ce sont les commandes de test, que l'on a trouvées en 3.2.4. La table propres refuserait toute insertion avec un code postal à quatre chiffres ou un consentement hors liste.

Corrigé 3.10

```
r = 0.rapprocher_caisse_base(caisse, cmd, lig, prod[["id_produit", "nom_produit"]])
dec = r[r["fichier"] == "caisse_2025-12.csv"]
copies = dec[dec["_merge"] == "left_only"]
vides = dec[(dec["_merge"] == "both") & dec["montant"].isna()]
qp = vides["qte"] * vides["prix_unitaire"]
lu = caisse.loc[caisse["fichier"] == "caisse_2025-12.csv", "montant"].sum()
total = fichiers.set_index("fichier").loc["caisse_2025-12.csv", "total_affiche"]
fin = lu - copies["montant"].sum() + qp.sum() - (qp - vides["montant_base"]).sum()
print("copies :", len(copies), "| vides :", len(vides), "| lu :", round(lu, 2), "| total affiché :",
↳ total, "| après cascade :", round(fin, 2), "| écart restant :", abs(round(fin - total, 2)))
```

```
copies : 6 | vides : 63 | lu : 70924.34 | total affiché : 73384.87 | après cascade : 73384.87 | écart
↳ restant : 0.0
```

L'écart restant est nul : le mois de décembre se réconcilie comme l'année entière, avec ses propres copies et ses montants vides (le nombre de copies de décembre, comme celui des vides, est proportionnel au nombre de lignes du mois, le plus élevé de l'année).

Corrigé 3.11

```
t_export = site["total"].map(0.montant_site_en_nombre)
apres = site["created_at"].str.endswith("Z")
t_eur = t_export.where(~apres, t_export / 100)
mois = pd.to_datetime(site["created_at"].str.replace("Z", ""), format="ISO8601").dt.month
sept = mois.eq(9)
test = site["customer_email"].eq("test@example.com"); copie = site["order_ref"].duplicated() & ~test
base_s = cmd[(cmd["canal"] == "Site") & cmd["date_commande"].str.startswith("2025-09")]
ca_base = base_s["id_commande"].map(lig.groupby("id_commande")["montant"].sum()).sum()
print("commandes avant/après le 15 :", int((sept & ~apres).sum()), "/", int((sept & apres).sum()))
print("somme brute :", round(t_export[sept].sum(), 2), "| erreur d'unité :", round((t_export[sept] -
↳ t_eur[sept]).sum(), 2), "| base :", round(ca_base, 2))
print("écart restant :", abs(round(t_eur[sept & ~test & ~copie].sum() - ca_base, 2)))
```

```
commandes avant/après le 15 : 248 / 294
somme brute : 3137730.81 | erreur d'unité : 3081125.52 | base : 55537.5
écart restant : 0.0
```

Septembre compte 248 commandes en euros (avant le 15) et 294 en centimes (à partir du 15). La somme brute du mois vaut 3 137 731 € alors que la base en donne 55 538 € : **56 fois trop**, parce que l'erreur d'unité ajoute 3 081 126 € à la somme. Une fois l'unité, les tests et les copies traités, l'écart restant est **nul**.

Corrigé 3.12

```
lu = caisse.groupby("fichier")["montant"].sum()
aff = fichiers.set_index("fichier")["total_affiche"]
r = 0.rapprocher_caisse_base(caisse, cmd, lig, prod[["id_produit", "nom_produit"]])
copies = r[r["_merge"] == "left_only"].groupby("fichier")["montant"].sum()
vides = r[(r["_merge"] == "both") & r["montant"].isna()].groupby("fichier")["montant_base"].sum()
apres = lu - copies.reindex(lu.index, fill_value=0) + vides.reindex(lu.index, fill_value=0)
t = pd.DataFrame({"ecart_avant_pct": (100 * (lu - aff) / aff).round(2), "ecart_apres_pct": (100 *
↳ (apres - aff) / aff).round(3)})
print("mois dans la tolérance de 0,5 % : avant", int((t["ecart_avant_pct"].abs() <= 0.5).sum()), "|
↳ après", int((t["ecart_apres_pct"].abs() <= 0.5).sum()))
exc = pd.DataFrame({"copies": r[r["_merge"] == "left_only"].groupby("fichier").size(), "vides":
↳ r[(r["_merge"] == "both") &
↳ r["montant"].isna()].groupby("fichier").size()}.fillna(0).astype(int)
print(exc.assign(total=exc.sum(axis=1)).sort_values("total", ascending=False).head(3).to_string())
```

```
mois dans la tolérance de 0,5 % : avant 0 | après 12
```

	copies	vides	total
fichier			
caisse_2025-12.csv	6	63	69
caisse_2025-11.csv	8	48	56
caisse_2025-10.csv	4	38	42

Avant nettoyage, aucun des douze mois n'est dans la tolérance de 0,5 % ; après, **tous** le sont, avec un écart de l'ordre du centime. Le fichier le plus chargé est celui de **décembre** (le plus grand nombre de lignes) : le nombre d'exceptions suit l'activité, un taux par ligne (3 % de vides, 0,5 % de copies) serait plus parlant qu'un compte brut.

Chapitre 4 : Documentation et dictionnaires de données — exercices et applications

Ce chapitre du cahier prolonge le chapitre 4 du livre. Les **applications** sont de petites études guidées, à refaire pas à pas : fiche d'un jeu, journal d'un nettoyage, dictionnaires, test d'un dictionnaire, glossaire, lignage, empreintes, et le jeu d'un chiffre depuis le brut. Les **exercices** (★ facile, ★★ moyen, ★★★ plus long) vous demandent d'appliquer une idée du livre ; chacun renvoie à la section concernée, et un **corrigé** suit. Les fichiers `verite_*.csv` ne servent qu'à **juger** vos choix une fois le travail fait.

```
import os, sys, json, shutil, tempfile
import numpy as np
import pandas as pd

sys.path.insert(0, "build")
import outils_ch04 as O

TMP4C = tempfile.mkdtemp(prefix="doc4c_", dir=os.environ.get("TMPDIR"))
```

Applications

Application 4.1 — La fiche de l'export du site (section 4.1.2)

Objectif. Remplir la fiche d'un jeu de données que l'on vient de recevoir : l'export des commandes du site (`site_commandes.csv`). On **mesure** ce que le fichier livre, on **constate** ses défauts, puis on les consigne dans la rubrique « limites connues ».

Étape 1 — Lire sans rien deviner, et regarder le grain. On lit tout en texte et l'on vérifie que la clé annoncée (`order_ref`) identifie bien une ligne.

```
so = O.charger("site_commandes", dtype=str)
print("lignes :", len(so), "| colonnes :", len(so.columns))
print("order_ref en double :", int(so["order_ref"].duplicated().sum()))
print("commandes de test (e-mail test@example.com) :",
      int(so["customer_email"].str.strip().str.lower().eq("test@example.com").sum()))
print("statuts :", so["status"].value_counts().to_dict())
```

```
lignes : 6259 | colonnes : 9
order_ref en double : 121
commandes de test (e-mail test@example.com) : 60
statuts : {'paid': 3629, 'PAID': 1537, 'Paid': 907, 'cancelled': 186}
```

Lecture. Le fichier compte 6 259 lignes pour 9 colonnes. La clé annoncée n'est **pas unique** : 121 `order_ref` apparaissent en double. 60 lignes sont des commandes de test, et le statut s'écrit de trois façons (paid, PAID, Paid), plus 186 lignes cancelled. Ce sont quatre limites à consigner dans la fiche.

Étape 2 — Le changement d'unité. La fiche mentionne que total change d'unité à une date. On le **vérifie** : on convertit le texte en nombre et l'on compare les totaux médians avant et après le 15 septembre.

```
nombre = pd.to_numeric(so["total"].str.replace(r"^\d\.", "", regex=True).str.replace(",", "."),
↳ errors="coerce")
apres = so["created_at"] >= "2025-09-15"
print("médiane du total avant le 15/09 :", nombre[~apres].median(), "| après :",
↳ nombre[apres].median())
print("rapport :", round(nombre[apres].median() / nombre[~apres].median(), 1))
```

```
médiane du total avant le 15/09 : 82.93 | après : 7808.0
rapport : 94.2
```

Lecture. Le total médian passe de 82,93 € avant le 15 septembre à 7 808 après : un rapport de l'ordre de **100** (94 ici, la différence tenant à la variation normale du panier d'une période à l'autre). Le changement d'unité est **confirmé par les données** : la rubrique de la fiche n'est plus une ru-meur.

Étape 3 — La fiche. On consigne ce que l'on sait, y compris les défauts découverts.

```
0.fiche_jeu(so, **{
    "nom": "site_commandes (v1)", "source": "export de la plateforme du site", "date d'extraction":
↳ "2025-12-31",
    "périmètre": "commandes du canal Site, année 2025", "une ligne =" : "une commande, SAUF doublons
↳ d'export",
    "clé": "order_ref (pas unique : doublons)", "limites connues": "doublons, commandes de test,
↳ statut en 3 casses",
    "limite n° 2": "total en texte ; en centimes à partir du 15/09", "droits": "adresses e-mail :
↳ données personnelles"})
```

```
nom                : site_commandes (v1)
source             : export de la plateforme du site
date d'extraction  : 2025-12-31
périmètre          : commandes du canal Site, année 2025
une ligne =        : une commande, SAUF doublons d'export
clé                : order_ref (pas unique : doublons)
limites connues    : doublons, commandes de test, statut en 3 casses
limite n° 2        : total en texte ; en centimes à partir du 15/09
droits             : adresses e-mail : données personnelles
lignes            : 6 259
colonnes           : 9
empreinte du contenu : 37254f3f50e0
```

À vous. Ajoutez la rubrique « version » et le propriétaire du jeu. Que diriez-vous à la personne qui vous a livré l'export, pour obtenir la date d'extraction exacte ?

Application 4.2 — Le journal du nettoyage de l'export du site (sections 4.1.3 et 4.1.4)

Objectif. Nettoyer l'export du site en consignait **chaque** décision dans un journal, vérifier l'équation de conservation, puis juger le nettoyage avec la vérité.

Étape 1 — La fonction de nettoyage. Cinq étapes : commandes de test, doublons d'export, annulées (selon la définition du glossaire), puis conversion du total en euros.

```
def nettoyer_site(brut):
    j = 0.Journal("site", brut)
    df = brut.copy()
    test = df["customer_email"].str.strip().str.lower().eq("test@example.com")
    df = j.etape(df, df[~test].copy(), "retirer les commandes de test", "ce ne sont pas des clients")
    df = j.etape(df, df.drop_duplicates("order_ref").copy(), "une ligne par order_ref", "une nouvelle
    ↪ tentative d'export ne crée pas de commande")
    df["status"] = df["status"].str.lower()
    df = j.etape(df, df[df["status"] != "cancelled"].copy(), "retirer les commandes annulées",
    ↪ "exclues du chiffre d'affaires (glossaire)")
    nb = pd.to_numeric(df["total"].str.replace(r"^\d\.", "", regex=True).str.replace(",", "."),
    ↪ errors="coerce")
    cent = df["created_at"] >= "2025-09-15"
    df["total_eur"] = np.where(cent, nb / 100, nb)
    df = j.etape(df, df, "total : texte vers nombre ; centimes vers euros dès le 15/09", "l'unité
    ↪ dépend de la date", modifiees=int(cent.sum()))
    return df, j
```

```
site_propre, jn = nettoyer_site(so)
print(jn.table()[["etape", "avant", "apres", "retirees", "modifiees"]].to_string(index=False))
print("lues, retirées, finales :", jn.conservation())
```

etape	avant	apres	retirees	modifiees
lecture	6259	6259	0	0
1	6259	6199	60	0
2	6199	6078	121	0
3	6078	5897	181	0
4	5897	5897	0	2367

lues, retirées, finales : (6259, 362, 5897)

Lecture. 6 259 lignes lues, 60 + 121 + 181 retirées, 5 897 lignes finales : l'équation de conservation tombe juste. L'étape 4 ne retire rien mais **modifie 2 367 lignes** (les 2 367 commandes d'après le 15 septembre, dont le total passe des centimes aux euros).

Étape 2 — Juger avec la vérité. Le fichier de vérité dit quelles lignes étaient de test, des doublons d'export ou des annulées, et quel était le vrai total.

```
ver = 0.charger("verite_site")
base = ver[~ver["default"].isin(["test", "doublon_export"]).drop_duplicates("order_ref")]
print("vérité : tests", int((ver["default"] == "test").sum()), "| doublons", int((ver["default"] ==
    ↪ "doublon_export").sum()),
    ↪ "| annulées", int((base["default"] == "annulee").sum()))
chk = site_propre.merge(ver.drop_duplicates("order_ref")[["order_ref", "total_vrai"]],
    ↪ on="order_ref")
print("écart maximal entre total_eur et le vrai total :", round((chk["total_eur"] -
    ↪ chk["total_vrai"]).abs().max(), 2), "€")
print("commandes restantes :", len(site_propre), "| chiffre d'affaires :",
    ↪ 0.eur(site_propre["total_eur"].sum()))
```

```
vérité : tests 60 | doublons 121 | annulées 181
écart maximal entre total_eur et le vrai total : 0.0 €
commandes restantes : 5897 | chiffre d'affaires : 600 164,13 €
```

Lecture. Le nettoyage retire **exactement** ce que la vérité dit : 60 tests, 121 doublons, 181 annulées, et le total converti coïncide avec le vrai total à zéro euro près. Le chiffre d'affaires nettoyé est de

600 164,13 €. Ce résultat parfait n'a rien de normal : les règles sont **exactes** parce que les défauts ont été fabriqués pour l'être ; sur une vraie donnée, le journal montrerait aussi ses fausses fusions, comme au livre (section 4.1.5).

À vous. Que se passerait-il si l'on oubliait la conversion des centimes ? Calculez le chiffre d'affaires sans la conversion et comparez.

Application 4.3 — Le dictionnaire de clients et de produits (sections 4.2.2 et 4.2.3)

Objectif. Fabriquer un squelette pour deux référentiels, l'enrichir avec les informations de métier, le ranger en CSV, et le relire.

Étape 1 — Les squelettes.

```
clients = 0.charger("clients")
produits = 0.charger("produits")
print(0.squelette(clients)[["colonne", "type", "manquants_pct", "distincts", "min",
↪ "max"]].to_string(index=False))
print()
print(0.squelette(produits)[["colonne", "type", "distincts", "min", "max"]].to_string(index=False))
```

	colonne	type	manquants_pct	distincts	min	max
	id_client	int64	0.0	6000	1	6000
	date_inscription	str	0.0	2541	2018-01-01	2025-12-30
	annee_naissance	int64	0.0	68	1940	2007
	ville	str	0.0	20	Ville A	Ville T
	canal_acquisition	str	0.0	3	Boutique	Site
	fidelite	int64	0.0	2	0	1
	email_valide	int64	0.0	2	0	1
	consentement_marketing	int64	0.0	2	0	1

	colonne	type	distincts	min	max
	id_produit	int64	120	1	120
	nom_produit	str	60	Affiche design	Étagère compact
	categorie	str	6	Bien-être	Papeterie
	prix_vente	float64	64	2.9	152.9
	cout_achat	float64	117	1.53	71.1
	fournisseur	str	8	Fournisseur A	Fournisseur H
	date_lancement	str	118	2018-01-30	2023-11-25

Étape 2 — L'enrichissement et le rangement. dico_complet fusionne le squelette et les informations saisies à la main (META). On range le résultat en CSV puis on le relit.

```
dc, dp = 0.dico_complet(clients, "clients"), 0.dico_complet(produits, "produits")
chemin = os.path.join(TMP4C, "dictionnaire_produits_v1.csv")
dp.to_csv(chemin, index=False)
relu = pd.read_csv(chemin)
print("relu :", relu.shape, "| colonnes du dictionnaire :", list(relu.columns))
print(relu[["colonne", "libelle", "unite", "valeurs_permises",
↪ "sensibilite"]].fillna("").to_string(index=False))
```

```
relu : (7, 11) | colonnes du dictionnaire : ['colonne', 'type', 'manquants_pct', 'exemple', 'libelle',
↪ 'unite', 'valeurs_permises', 'obligatoire', 'codage_manquant', 'regle_ou_source', 'sensibilite']
↪      colonne      libelle      unite
↪      valeurs_permises      sensibilite
↪      id_produit      Identifiant du produit
↪      nom_produit      Désignation commerciale
↪      categorie      Catégorie
↪      Cuisine|Maison|Décoration|Papeterie|Jardin|Bien-être
↪      prix_vente      Prix de vente catalogue TTC      €
↪
```

```

    cout_achat    Coût d'achat unitaire HT          €
↪  confidentielle
    fournisseur      Fournisseur
↪
date_lancement    Date de mise au catalogue date (aaaa-mm-jj)
↪

```

Étape 3 — Un piège que le squelette signale. `nom_produit` a-t-il autant de valeurs distinctes que de lignes ? Et que dit alors le dictionnaire ?

```

print("produits :", len(produits), "| noms distincts :", produits["nom_produit"].nunique())
print("règle inscrite dans le dictionnaire :", dp.loc[dp["colonne"] == "nom_produit",
↪  "regle_ou_source"].iloc[0])

```

```

produits : 120 | noms distincts : 60
règle inscrite dans le dictionnaire : catalogue ; NON unique (voir limites)

```

Lecture. Le catalogue compte 120 produits mais seulement **60 noms distincts** : chaque nom est porté par deux produits (à des prix différents). Une jointure sur le nom doublerait les lignes. Le dictionnaire le dit en toutes lettres (« NON unique ») : c'est cette phrase qui évite à un collègue de joindre sur `nom_produit`.

À vous. Quelles colonnes de clients marqueriez-vous « sensibles » et pourquoi ? Comparez à la colonne `sensibilite` du dictionnaire.

Application 4.4 — Le test du dictionnaire (section 4.2.5)

Objectif. Transformer le dictionnaire en **contrôle** : le laisser arrêter la chaîne, puis s'en servir comme cahier des charges d'un nettoyage.

Étape 1 — Un contrôle qui bloque. On enveloppe la vérification dans une fonction qui lève une erreur.

```

def controle(df, dico, nom):
    anomalies = 0.verifier_dictionnaire(df, dico)
    if anomalies:
        raise AssertionError(f"{nom} ne respecte pas son dictionnaire :\n- " + "\n- ".join(anomalies))
    return "ok"

print("clients :", controle(clients, dc, "clients"))
degrade = clients.drop(columns="email_valide").assign(pays="Pays P1")
degrade.loc[3, "fidelite"] = 5
try:
    controle(degrade, dc, "clients dégradé")
except AssertionError as e:
    print(e)

```

```

clients : ok
clients dégradé ne respecte pas son dictionnaire :
- colonne absente du dictionnaire : pays
- fidelite : 1 valeurs hors domaine (0|1)
- colonne du dictionnaire absente du fichier : email_valide

```

Lecture. Le contrôle lève une erreur qui **nomme les trois anomalies** : la colonne `pays` ajoutée, la valeur 5 hors du domaine `0|1` de `fidelite`, et la colonne `email_valide` disparue. La chaîne s'arrêterait là, avant qu'un calcul faux ne parte.

Étape 2 — Le dictionnaire du CRM comme cahier des charges. Sur le CRM brut, le test liste ce qu'il faut nettoyer. On corrige le **consentement** (sept écritures), puis on relance.

```

crm = 0.charger("crm_clients", dtype=str)
dcrm = 0.dico_complet(crm, "crm_clients")
print("avant :", [a for a in 0.verifier_dictionnaire(crm, dcrm) if a.startswith("consentement")])
oui = {"oui", "o", "1", "true"}
crm["consentement_marketing"] = crm["consentement_marketing"].str.strip().str.lower().map(lambda v:
↪ np.nan if pd.isna(v) else ("oui" if v in oui else v))
print("après :", [a for a in 0.verifier_dictionnaire(crm, dcrm) if a.startswith("consentement")])

```

```

avant : ['consentement_marketing : 2428 valeurs hors domaine (oui|non)', 'consentement_marketing :
↪ 2161 valeurs vides alors que la colonne est obligatoire']
après : ['consentement_marketing : 2161 valeurs vides alors que la colonne est obligatoire']

```

Lecture. La normalisation fait disparaître les 2 428 consentements hors domaine (0, 1, TRUE, OUI... deviennent oui). Il reste les **2 161 vides** : ce sont des consentements **non renseignés**, que le nettoyage ne peut pas inventer.

À vous. Il reste des consentements vides alors que la colonne est obligatoire. Est-ce un défaut des données ou du dictionnaire ? Argumentez, puis proposez une modification du dictionnaire (et notez-la dans un journal des changements).

Application 4.5 — Un glossaire pour la boutique (section 4.2.6)

Objectif. Mesurer ce qu'une définition change, puis la figer dans un glossaire exploitable par un programme.

Étape 1 — « Client actif » selon trois définitions, par canal d'acquisition.

```

cmd = 0.charger("commandes"); cmd["jour"] = pd.to_datetime(cmd["date_commande"])
cli = 0.charger("clients").set_index("id_client")
ref = pd.Timestamp("2025-12-31")
def actifs(jours, mini=1):
    r = cmd[cmd["jour"] > ref - pd.Timedelta(days=jours)].groupby("id_client").size()
    return r[r >= mini].index
res = pd.DataFrame({"1 commande, 12 mois": cli.loc[actifs(365), "canal_acquisition"].value_counts(),
↪ "1 commande, 6 mois": cli.loc[actifs(183), "canal_acquisition"].value_counts(),
    "2 commandes, 12 mois": cli.loc[actifs(365, 2),
↪ "canal_acquisition"].value_counts()})
print(res.assign(**{"inscrits": cli["canal_acquisition"].value_counts()}).to_string())

```

	1 commande, 12 mois	1 commande, 6 mois	2 commandes, 12 mois	inscrits
canal_acquisition				
Boutique	1916	1561	1323	2957
Site	1496	1216	1019	2329
Réseaux	463	371	312	714

Lecture. Au 31 décembre 2025, la Boutique compte 1 916 clients actifs selon la première définition, sur 2 957 inscrits ; le Site 1 496 sur 2 329 ; les Réseaux 463 sur 714. Les trois définitions ne changent pas le classement des canaux (Boutique, Site, Réseaux), et la proportion de clients ayant au moins deux commandes parmi ceux qui en ont une est voisine d'un canal à l'autre (de l'ordre de 67 à 69 %) : le choix de la définition déplace le **total**, pas la comparaison entre canaux. Il faut le savoir pour ne pas s'inquiéter à tort quand deux collègues citent des totaux différents.

Étape 2 — « Panier moyen » selon trois définitions.

```

lig = 0.charger("lignes_commande")
x = lig.merge(cmd, on="id_commande")
x25 = x[x["date_commande"] >= "2025-01-01"]
ttc = x25["montant"].sum() / x25["id_commande"].nunique()
ht = ttc / 1.2
par_canal = (x25.groupby("canal")["montant"].sum() / x25.groupby("canal")["id_commande"].nunique())

```

```
print("TTC remises déduites :", 0.eur(ttc), "| HT :", 0.eur(ht), "| moyenne des paniers moyens des  
↪ canaux :", 0.eur(par_canal.mean()))
```

TTC remises déduites : 102,33 € | HT : 85,27 € | moyenne des paniers moyens des canaux : 102,38 €

Lecture. Le panier moyen 2025 est de 102,33 € TTC, remises déduites, et de 85,27 € hors taxe. La moyenne des paniers moyens des canaux (102,38 €) en est proche *ici*, mais rien ne l'y oblige : si les canaux avaient des tailles très différentes, l'écart serait plus grand. La règle du glossaire (calculer sur le total) protège de ce risque, même quand il est faible.

Étape 3 — Le glossaire, en YAML. On écrit les définitions retenues sous une forme qu'un programme peut lire, puis on la relit.

```
import yaml
glossaire = {"client_actif": {"definition": "au moins une commande payée sur les 12 derniers mois",  
↪ "calcul": "COUNT(DISTINCT id_client), commandes du 01/01 au 31/12", "proprietaire": "gérante"},  
            "panier_moyen": {"definition": "CA TTC remises déduites / commandes distinctes, sur le  
↪ total", "calcul": "SUM(montant) / COUNT(DISTINCT id_commande)", "proprietaire":  
↪ "analyse"}}
chemin = os.path.join(TMP4C, "glossaire_v1.yaml")
open(chemin, "w", encoding="utf-8").write(yaml.safe_dump(glossaire, allow_unicode=True,  
↪ sort_keys=False))
print(list(yaml.safe_load(open(chemin, encoding="utf-8"))), "|", yaml.safe_load(open(chemin,  
↪ encoding="utf-8"))["panier_moyen"]["calcul"])
```

```
['client_actif', 'panier_moyen'] | SUM(montant) / COUNT(DISTINCT id_commande)
```

À vous. Ajoutez l'entrée « commande annulée » du livre et une entrée « trimestre ». Quel serait le contre-exemple de chacune ?

Application 4.6 — Le lignage d'un rapport et l'analyse d'impact (sections 4.3.2 et 4.3.3)

Objectif. Décrire les dépendances d'un livrable, remonter en amont, mesurer l'impact en aval, et dessiner le graphe.

Étape 1 — Construire le lignage. On ajoute à la synthèse du livre un fichier de paramètres (le taux de taxe) dont dépend le calcul de marge.

```
L = 0.Lignage()
for nom in ["commandes", "lignes_commande", "produits", "parametres.yaml"]:  
    L.ajouter(nom, "source")
L.ajouter("ca_par_canal.sql", "requete", ["commandes", "lignes_commande"])
L.ajouter("marge.py", "script", ["lignes_commande", "produits", "parametres.yaml"])
L.ajouter("synthese_t4.csv", "table", ["ca_par_canal.sql", "marge.py"])
L.ajouter("message_gerante.md", "sortie", ["synthese_t4.csv"])
print("sources du message :", L.sources("message_gerante.md"))
print("si le taux de taxe change :", L.aval("parametres.yaml"))
print("si `commandes` change :", L.aval("commandes"))
```

```
sources du message : ['commandes', 'lignes_commande', 'parametres.yaml', 'produits']
si le taux de taxe change : ['marge.py', 'message_gerante.md', 'synthese_t4.csv']
si `commandes` change : ['ca_par_canal.sql', 'message_gerante.md', 'synthese_t4.csv']
```

Étape 2 — Les éléments sans effet. Quels éléments ne dépendent *ni* des paramètres *ni* des commandes ? Et existe-t-il une source dont rien ne dépend (donc inutile) ?

```
touches = set(L.aval("parametres.yaml")) | set(L.aval("commandes"))
print("éléments non touchés :", sorted(set(L.noeuds) - touches - {"parametres.yaml", "commandes"}))
print("sources inutilisées :", [n for n in L.noeuds if not L.noeuds[n]["depend"] and not L.aval(n)])
```

```
éléments non touchés : ['lignes_commande', 'produits']
sources inutilisées : []
```

Lecture. Aucune source n'est inutilisée. Les éléments que ni le taux de taxe ni les commandes n'atteignent sont `lignes_commande` et `produits` : ce sont d'autres sources. Si le taux de taxe change, il faut relancer `marge.py`, la synthèse et le message ; si les commandes changent, `ca_par_canal.sql`, la synthèse et le message : `marge.py` n'est **pas** touché, puisqu'il ne lit pas les commandes.

Étape 3 — Le dessin.

Lignage de la synthèse, avec le fichier de paramètres

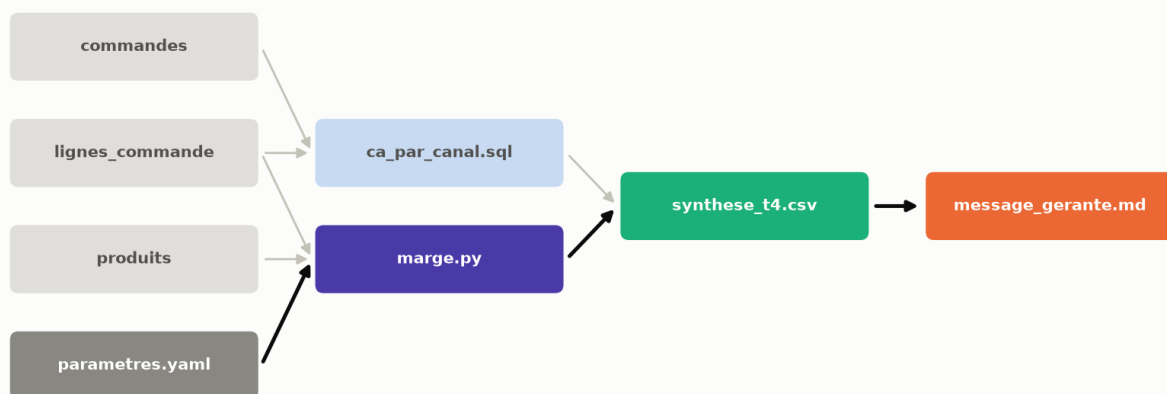


Figure 1: Lignage de la synthèse avec son fichier de paramètres : en couleur pleine, tout ce qui dépend du taux de taxe.

À vous. Ajoutez un nœud `graphique_t4.png` qui dépend de `synthese_t4.csv`. Que devient la liste « si commandes change » ?

Application 4.7 — Empreintes et manifeste (sections 4.3.4 et 4.3.6)

Objectif. Construire un manifeste, détecter une modification, et comprendre ce qu'une empreinte de **tableau** dit de l'ordre des lignes.

Étape 1 — Le manifeste d'une livraison et sa vérification.

```
fichiers = ["clients.csv", "produits.csv", "commandes.csv", "lignes_commande.csv"]
manif = {f: 0.empreinte_fichier(os.path.join(0.donnees(), f))[:12] for f in fichiers}
def verifier(manif, dossier):
    return {f: 0.empreinte_fichier(os.path.join(dossier, f))[:12] == h for f, h in manif.items()}
print(verifier(manif, 0.donnees()))
```

```
{'clients.csv': True, 'produits.csv': True, 'commandes.csv': True, 'lignes_commande.csv': True}
```

Étape 2 — Une modification discrète. On copie la livraison, on change **une** valeur d'un fichier, et l'on vérifie à nouveau.

```
copie = os.path.join(TMP4C, "livraison"); os.makedirs(copie)
for f in fichiers:
    shutil.copy(os.path.join(0.donnees(), f), copie)
p = os.path.join(copie, "produits.csv")
open(p, "w", encoding="utf-8").write(open(p, encoding="utf-8").read().replace("Cuisine", "Cuisinier",
↵ 1))
```

```
print(verifier(manif, copie))
```

```
{'clients.csv': True, 'produits.csv': False, 'commandes.csv': True, 'lignes_commande.csv': True}
```

Lecture. La modification d'un seul mot dans `produits.csv` est détectée (`False`), les trois autres fichiers restent conformes.

Étape 3 — L'empreinte d'un tableau dépend de l'ordre des lignes. Mélanger les lignes ne change pas le **contenu**, mais change l'empreinte, sauf si l'on trie avant de calculer.

```
m = produits.sample(frac=1, random_state=1)
print("même empreinte après mélange :", 0.empreinte_df(m) == 0.empreinte_df(produits))
print("même empreinte après tri sur la clé :", 0.empreinte_df(m.sort_values("id_produit")) ==
      ↪ 0.empreinte_df(produits.sort_values("id_produit")))
```

```
même empreinte après mélange : False
même empreinte après tri sur la clé : True
```

Lecture. Le même contenu, dans un autre ordre, donne une **autre** empreinte ; trié sur la clé avant le calcul, la même. Pour comparer des **contenus**, on trie avant de calculer l'empreinte (ou l'on calcule l'empreinte du fichier brut, qui dépend de l'ordre des octets).

À vous. Dans un manifeste, faut-il calculer l'empreinte du fichier ou du tableau lu ? Donnez un argument pour chaque choix.

Application 4.8 — Rejouer un chiffre depuis le brut (section 4.3.8)

Objectif. Documenter un second chiffre, le rejouer, le recouper par un calcul indépendant, et vérifier que la documentation **détecte** un fichier remplacé.

Étape 1 — La documentation et le jeu. Chiffre visé : le CA TTC du canal Réseaux au troisième trimestre 2024.

```
doc = {"question": "CA TTC du canal Réseaux, T3 2024",
      "sources": {"commandes": {"fichier": "commandes.csv", "empreinte": manif["commandes.csv"]},
                  "lignes_commande": {"fichier": "lignes_commande.csv", "empreinte":
                  ↪ manif["lignes_commande.csv"]}},
      "filtres": [{"canal", "=", "Réseaux"}, {"date_commande", ">=", "2024-07-01"},
                  ↪ {"date_commande", "<=", "2024-09-30"}],
      "mesure": "montant"}
res = 0.rejouer(doc)
print(res)
```

```
{'empreintes_ok': True, 'valeur': 30903.86, 'lignes': 657}
```

Étape 2 — Un recouplement indépendant. On refait le calcul **sans** la fonction `rejouer`, avec une jointure écrite à la main.

```
x = lig.merge(cmd[["id_commande", "canal", "date_commande"]], on="id_commande")
y = x[(x["canal"] == "Réseaux") & (x["date_commande"].between("2024-07-01", "2024-09-30"))]
print("recouplement :", round(y["montant"].sum(), 2), "| lignes :", len(y), "| identique :",
      ↪ round(y["montant"].sum(), 2) == res["valeur"])
```

```
recouplement : 30903.86 | lignes : 657 | identique : True
```

Lecture. Le jeu donne 30 903,86 € sur 657 lignes, et le calcul indépendant (une jointure écrite à la main) le confirme.

Étape 3 — Le fichier remplacé. On rejoue la documentation sur la copie modifiée de l'application 4.7 : les chiffres sortent, mais l'empreinte ne correspond plus.

```
res_copie = 0.rejouer(doc, dossier=copie)
print("sur la copie dont produits.csv a changé :", res_copie)
```

```
sur la copie dont produits.csv a changé : {'empreintes_ok': True, 'valeur': 30903.86, 'lignes': 657}
```

Lecture. Sur la copie, l'empreinte est « ok » : la documentation ne décrit que **deux sources** (commandes et lignes_commande), pas produits. Une documentation ne protège que ce qu'elle déclare.

À vous. Dans la copie, seul produits.csv a été modifié : pourquoi le rejeu ne le voit-il pas ? Que faudrait-il ajouter à la documentation pour qu'elle le voie ?

Exercices

Exercice 4.1 ★ — Qu'est-ce qui manque ? (section 4.1.1)

Une collègue vous transmet un fichier avec ce seul mot d'accompagnement : « ventes_final2.xlsx : ventes nettoyées, chiffres en euros, voir mon notebook. » Listez **six** informations qui manquent pour que vous puissiez refaire son travail et défendre ses chiffres.

Exercice 4.2 ★ — La fiche du catalogue fournisseur (section 4.1.2)

Remplissez la fiche de catalogue_fournisseur.csv avec 0.fiche_jeu : source, date d'extraction (inventée mais précisée), périmètre, grain, clé, limites. Vérifiez par le code que la clé annoncée est bien unique, et signalez-le dans les limites si ce n'est pas le cas.

Exercice 4.3 ★★ — L'équation de conservation (section 4.1.3)

Un journal de nettoyage indique : 5 000 lignes lues ; étape 1 : 120 lignes retirées ; étape 2 : 300 lignes modifiées, aucune retirée ; étape 3 : 480 lignes retirées ; étape 4 : 35 lignes retirées. Le rapport final annonce **4 380** lignes. L'équation de conservation est-elle vérifiée ? Si non, de combien est l'écart et que cela signifie-t-il ?

Exercice 4.4 ★★ — Nommer ses fichiers (section 4.1.6)

Renommez, selon la convention nom_AAAA-MM-JJ_vN.ext, ces cinq fichiers : clients_final.csv (extrait le 5 mars 2025, première version), clients_final2.csv (même extraction, corrigé), Copie de ventes (3).xlsx (ventes du T1 2025, troisième version), export 12-02.csv (export de caisse du 2 décembre 2025), rapport_VRAI.docx (rapport du T2 2025, deuxième version). Que gagne-t-on à écrire la date en ISO ?

Exercice 4.5 ★ — Des commentaires qui disent pourquoi (section 4.1.6)

Réécrivez ces trois commentaires pour qu'ils expliquent **la raison** et non le geste, en inventant une raison plausible tirée des données de la boutique : (a) `df = df[df["montant"] > 0]` # garde les montants positifs ; (b) `df["total"] = df["total"] / 100` # divise par 100 ; (c) `df = df.drop_duplicates("email")` # supprime les doublons.

Exercice 4.6 ★★ — Ce que le squelette ne sait pas dire (section 4.2.2)

Fabriquez le squelette de `site_commandes` (lu en texte). Notez **trois** choses qu'il vous apprend, puis **trois** choses qu'il ne peut pas vous apprendre et que seul un humain peut écrire.

Exercice 4.7 ★★ — Le dictionnaire de cinq colonnes (section 4.2.3)

Écrivez le dictionnaire (libellé, type technique, type logique, unité, valeurs permises, obligatoire, codage des manquants, règle ou source) de `order_ref`, `created_at`, `status`, `total` et `currency` dans `site_commandes`. Faites apparaître le changement d'unité de `total` et les trois écritures de `status`.

Exercice 4.8 ★★ — Un test sur le statut (section 4.2.5)

Écrivez l'entrée de dictionnaire de `status` avec ses valeurs permises (`paid`, `cancelled`), lancez `verifier_dictionnaire` sur le fichier brut, puis normalisez la casse et relancez. Combien de lignes sont hors domaine avant, après ?

Exercice 4.9 ★ — Une définition unique de « commande annulée » (section 4.2.6)

Écrivez l'entrée de glossaire de « commande annulée » (définition, calcul, contre-exemple, propriétaire). Combien de commandes `cancelled` compte le fichier brut du site, et combien après avoir retiré les doublons d'export ?

Exercice 4.10 ★★ — Le lignage d'une campagne (section 4.3.2)

Une campagne d'e-mails suit cette chaîne : `rapprochement.py` lit `crm.csv` et `site.csv` ; il produit `clients_uniques.csv` ; `campagne.csv` est fabriqué à partir de `clients_uniques.csv` et de `consentements.csv` ; `courriel_envoye.log` découle de `campagne.csv`. Écrivez le lignage et répondez : si `consentements.csv` change, quoi relancer ? Et d'où vient exactement `courriel_envoye.log` ?

Exercice 4.11 ★★★ — Un journal d'audit qui ne s'efface pas (section 4.3.5)

Écrivez cinq événements dans un journal d'audit, puis une fonction `ajout_seulement(ancien, nouveau)` qui vérifie que l'ancien état du fichier est un **préfixe** du nouveau. Montrez qu'elle accepte un ajout et refuse la modification d'une ligne ancienne.

Exercice 4.12 ★★★ — Une documentation qui se trompe (section 4.3.8)

Une collègue vous a laissé cette documentation : « CA TTC du canal Site, T4 2025 = 211 433,79 € ; `filtres : canal == 'Site', date_commande >= '2025-10-01', date_commande <= '2025-12-30'.` » Rejouez-la. Le chiffre annoncé est-il retrouvé ? Trouvez la cause de l'écart, corrigez la documentation et chiffrez l'effet de l'erreur.

Corrigé 4.1

Il manque, au minimum : (1) la **source** (d'où viennent les données brutes, quel système) ; (2) la **date d'extraction** ; (3) le **périmètre** (quelles années, quels canaux, quelles lignes exclues) ; (4) le **grain** (une ligne = une commande, une ligne de commande, un client ?) ; (5) les **règles de nettoyage** avec leurs effectifs (« nettoyées » ne dit pas ce qui a été retiré) ; (6) la **définition** des chiffres : TTC ou HT, remises déduites ou non, périodes. Et aussi : la **version** du fichier, l'**unité** réelle (« euros » sans date de validité est suspect), le **lieu du notebook** et la façon de le relancer. La phrase de la collègue est une affirmation, pas une documentation.

Corrigé 4.2

```
cat = 0.charger("catalogue_fournisseur", dtype=str)
0.fiche_jeu(cat, **{"nom": "catalogue_fournisseur (v1)", "source": "catalogue envoyé par le
↳ fournisseur", "date d'extraction": "2025-11-20 (supposée)",
    "périmètre": "produits proposés par le fournisseur, pas tous ceux de la boutique",
    ↳ "une ligne =" : "un article du catalogue fournisseur",
    "clé": "code_fournisseur", "limites connues": "désignations réécrites (casse,
    ↳ accents, abréviations), codes différents de ceux de la boutique"})
print("code_fournisseur unique :", cat["code_fournisseur"].is_unique)
```

```
nom                : catalogue_fournisseur (v1)
source             : catalogue envoyé par le fournisseur
date d'extraction  : 2025-11-20 (supposée)
périmètre          : produits proposés par le fournisseur, pas tous ceux de la boutique
une ligne =        : un article du catalogue fournisseur
clé                : code_fournisseur
limites connues    : désignations réécrites (casse, accents, abréviations), codes différents de
↳ ceux de la boutique
lignes             : 118
colonnes           : 5
empreinte du contenu : 29a5a9a41a19
code_fournisseur unique : True
```

La clé est bien unique (True), mais elle **n'est pas** celle de la boutique : la mention « codes différents de ceux de la boutique » est la limite la plus utile, parce qu'elle annonce qu'il faudra un **rapprochement** (section 2.5) avant toute jointure.

Corrigé 4.3

Lignes finales attendues : $5\,000 - 120 - 480 - 35 = 4\,365$ (l'étape 2 ne retire rien). Le rapport annonce 4 380 : **15 lignes de trop**. L'équation de conservation n'est donc **pas** vérifiée : une étape a **créé** des lignes que le journal ne mentionne pas (par exemple une jointure qui a dupliqué des lignes, ou une concaténation faite deux fois). On ne cherche pas l'erreur dans le résultat, on la cherche dans le journal, à l'étape où l'effectif « après » ne vaut pas « avant » moins « retirées ».

Corrigé 4.4

clients_2025-03-05_v1.csv ; clients_2025-03-05_v2.csv ; ventes_t1_2025_v3.xlsx (ou ventes_2025-01_v3.xlsx) ; export_caisse_2025-12-02.csv ; rapport_t2_2025_v2.docx. L'écriture **année-mois-jour** a un grand avantage : l'**ordre alphabétique** des noms est l'**ordre chronologique**, et il n'y a aucune ambiguïté entre 12-02 (le 12 février, ou le 2 décembre ?).

Corrigé 4.5

- (a) # les montants négatifs sont des avoirs (retours), traités à part dans l'analyse des retours ; (b) # le site exporte les totaux en centimes depuis le 15/09/2025 ; (c) # une même personne peut avoir été saisie plusieurs fois : on garde la saisie la plus ancienne (id_crm le plus petit). Chaque commentaire répond à la question qu'une relectrice se poserait : **pourquoi** ?

Corrigé 4.6

```
0.squelette(0.charger("site_commandes", dtype=str))
```

Le squelette **apprend** : (1) toutes les colonnes sont du texte, y compris total et created_at (donc rien n'est typé) ; (2) promo_code est vide pour environ 84 % des lignes (l'absence de code est un cas normal, pas une erreur) ; (3) order_ref a **moins de valeurs distinctes que de lignes** (des doublons d'export). Il **ne peut pas** dire : (1) que total est en euros avant le 15 septembre et en **centimes** après ; (2) que paid, PAID et Paid sont le **même** statut ; (3) que test@example.com désigne des commandes de test à écarter. Aucune de ces trois informations n'est dans les valeurs : elles viennent de la source ou de l'enquête.

Corrigé 4.7

Colonne	Libellé	Type technique → logique	Unité	Valeurs permises	Oblig.	Manquants	Règle ou source
order_ref	Référence de la commande	str → identifiant		WEB- + 6 chiffres (ou WEB-T pour un test)	oui	jamais vide	plateforme du site ; non unique (doublons d'export)
created_at	Date et heure de création	str → date-heure		ISO ; sans fuseau avant le 15/09, en UTC (Z) après	oui	jamais vide	plateforme
status	Statut de la commande	str → catégorie		paid, cancelled après mise en minuscules ; écrit paid, PAID ou Paid	oui	jamais vide	plateforme
total	Total de la commande TTC, remises déduites	str → montant	€ jusqu'au 14/09/2025, centimes à partir du 15/09	positif	oui	jamais vide	somme des lignes ; texte avec €, virgule ou point
currency	Devise	str → catégorie		EUR ; écrit EUR, eur ou €	oui	jamais vide	plateforme

Le dictionnaire écrit **les pièges** que le squelette ne voit pas : l'unité qui change à une date, la casse du statut, le format du total.

Corrigé 4.8

```
dsite = pd.DataFrame([{"colonne": "status", "type": "str", "valeurs_permises": "paid|cancelled",
↳ "obligatoire": "oui"}])
so_c = 0.charger("site_commandes", dtype=str)[["status"]]
print("avant :", 0.verifier_dictionnaire(so_c, dsite))
so_c["status"] = so_c["status"].str.lower()
print("après :", 0.verifier_dictionnaire(so_c, dsite) or "aucune anomalie")
```

```
avant : ['status : 2444 valeurs hors domaine (paid|cancelled)']
après : aucune anomalie
```

Avant la normalisation, le test signale **2 444** lignes (celles écrites PAID ou Paid) comme **hors domaine** ; après `str.lower()`, il ne reste plus d'anomalie. C'est la preuve que la **normalisation** de la casse est une étape de nettoyage **exigée par le dictionnaire**.

Corrigé 4.9

Entrée de glossaire : **commande annulée** — *définition* : commande dont le statut vaut cancelled après mise en minuscules ; *calcul* : `lower(status) == 'cancelled'` ; *traitement* : exclue du chiffre d'affaires et du nombre de commandes ; *contre-exemple* : une commande **retournée** après livraison n'est pas annulée ; *propriétaire* : la gérante.

```
so9 = 0.charger("site_commandes", dtype=str)
print("lignes cancelled dans le brut :", int((so9["status"].str.lower() == "cancelled").sum()))
print("commandes annulées distinctes :", int(so9.loc[so9["status"].str.lower() == "cancelled",
↳ "order_ref"].nunique()))
```

```
lignes cancelled dans le brut : 186
commandes annulées distinctes : 181
```

Le fichier brut compte **186** lignes cancelled mais seulement **181 commandes annulées distinctes** : cinq lignes sont des **doublons d'export** de commandes annulées. Compter des lignes plutôt que des commandes distinctes est exactement l'erreur de grain que la définition doit empêcher.

Corrigé 4.10

```
L10 = 0.Lignage()
for nom in ["crm.csv", "site.csv", "consentements.csv"]:
    L10.ajouter(nom, "source")
L10.ajouter("rapprochement.py", "script", ["crm.csv", "site.csv"])
L10.ajouter("clients_uniques.csv", "table", ["rapprochement.py"])
L10.ajouter("campagne.csv", "table", ["clients_uniques.csv", "consentements.csv"])
L10.ajouter("courriel_envoye.log", "sortie", ["campagne.csv"])
print("si consentements.csv change :", L10.aval("consentements.csv"))
print("sources de courriel_envoye.log :", L10.sources("courriel_envoye.log"))
```

```
si consentements.csv change : ['campagne.csv', 'courriel_envoye.log']
sources de courriel_envoye.log : ['consentements.csv', 'crm.csv', 'site.csv']
```

Si `consentements.csv` change, il faut relancer **campagne.csv** puis **courriel_envoye.log** ; `rapprochement.py` et `clients_uniques.csv` ne sont pas touchés. Le journal d'envoi a **trois sources** : `crm.csv`, `site.csv` et `consentements.csv`. Un lignage de ce type rend visible un risque juridique : le fichier des consentements est une **entrée directe** de l'envoi.

Corrigé 4.11

```

a1 = os.path.join(TMP4C, "audit_hier.jsonl"); a2 = os.path.join(TMP4C, "audit_aujourd'hui.jsonl")
evts = [{"2026-01-05 09:00", "analyste", "réception", "crm_v1.csv", "aaa"}, ("2026-01-05 09:30",
↳ "analyste", "nettoyage", "crm_propre.csv", "bbb"),
      ("2026-01-05 10:00", "gérante", "validation", "crm_propre.csv", "bbb"), ("2026-01-06 08:00",
↳ "analyste", "réception", "site_v1.csv", "ccc"),
      ("2026-01-06 08:20", "analyste", "nettoyage", "site_propre.csv", "ddd")]
for e in evts[:3]:
    O.consigner(a1, *e)
shutil.copy(a1, a2)
for e in evts[3:]:
    O.consigner(a2, *e)

def ajout_seulement(ancien, nouveau):
    a, n = open(ancien, encoding="utf-8").read(), open(nouveau, encoding="utf-8").read()
    return n.startswith(a)

print("ajout accepté :", ajout_seulement(a1, a2))
open(a2, "w", encoding="utf-8").write(open(a2, encoding="utf-8").read().replace("validation",
↳ "relecture", 1))
print("après retouche d'une ligne ancienne :", ajout_seulement(a1, a2))

```

```

ajout accepté : True
après retouche d'une ligne ancienne : False

```

Un journal d'audit **n'est valable que s'il n'est jamais retouché** : la vérification « l'ancien contenu est un préfixe du nouveau » accepte un ajout et refuse la moindre modification d'une ligne passée (ici, « validation » changé en « relecture »). Pour une garantie plus forte, on peut aussi **enchaîner** les empreintes (chaque ligne contient l'empreinte de la précédente), de sorte qu'on ne puisse pas retoucher le passé sans casser la chaîne.

Corrigé 4.12

```

doc12 = {"sources": {"commandes": {"fichier": "commandes.csv", "empreinte":
↳ O.empreinte_fichier(os.path.join(O.donnees(), "commandes.csv"))[:12]},
        "lignes_commande": {"fichier": "lignes_commande.csv", "empreinte":
↳ O.empreinte_fichier(os.path.join(O.donnees(),
↳ "lignes_commande.csv"))[:12]}},
        "filtres": [("canal", "=", "Site"), ("date_commande", ">=", "2025-10-01"), ("date_commande",
↳ "<=", "2025-12-30")], "mesure": "montant"}
faux = O.rejouer(doc12)
doc12["filtres"][2] = ("date_commande", "<=", "2025-12-31")
juste = O.rejouer(doc12)
print("avec <= 2025-12-30 :", faux["valeur"], "| avec <= 2025-12-31 :", juste["valeur"], "| écart :",
↳ round(juste["valeur"] - faux["valeur"], 2))

```

```
avec <= 2025-12-30 : 210063.78 | avec <= 2025-12-31 : 211433.79 | écart : 1370.01
```

Le chiffre annoncé (211 433,79 €) n'est **pas** retrouvé : la borne haute du filtre est le **30 décembre** au lieu du **31**, de sorte que le dernier jour du trimestre est oublié. La documentation corrigée (<= '2025-12-31') redonne 211 433,79 €. L'effet de l'erreur est l'écart affiché : le chiffre d'affaires **d'une seule journée** (le 31 décembre), que personne n'aurait vu sans rejouer. C'est l'intérêt d'une documentation **exécutable** : une erreur de rédaction devient une erreur détectable.

Pistes pour les « À vous » des applications

- **4.1.** La fiche gagne version : v1 et propriétaire : équipe du site (à confirmer). À la personne qui a fait l'export, on demande par écrit : « à quelle date et à quelle heure l'export a-t-il été lancé, et avec quel filtre de période ? ».
- **4.2.** Sans la conversion, le chiffre d'affaires nettoyé serait de **23 891 020,95 €** au lieu de 600 164,13 €, soit près de **quarante fois** trop : les 2 367 commandes d'après le 15 septembre pèseraient cent fois leur valeur.
- **4.3.** Dans `clients`, cinq colonnes sont marquées sensibles : l'identifiant (indirecte), l'année de naissance, la ville et le consentement (personnelles), et la validité de l'e-mail (indirecte). Le canal d'acquisition, la carte de fidélité et la date d'inscription ne le sont pas, mais **croisés** avec la ville et l'année de naissance ils peuvent aider à retrouver une personne : c'est l'objet du chapitre 5.
- **4.4.** Un consentement vide est d'abord un défaut **de saisie** (ou d'absence de recueil) : le dictionnaire a raison de dire « obligatoire ». On peut décider de le traiter comme « non » par prudence, à **écrire** dans le dictionnaire (vide = non renseigné, traité comme refus) et dans le journal des changements, avec la date et la raison.
- **4.5.** « Commande annulée » : statut cancelled après mise en minuscules, exclue du chiffre d'affaires ; contre-exemple : une commande retournée. « Trimestre » : trimestre civil, du 1er janvier, avril, juillet ou octobre au dernier jour du troisième mois, bornes **incluses** ; contre-exemple : une extraction arrêtée au 15 du dernier mois.
- **4.6.** Avec `graphique_t4.png` qui dépend de `synthese_t4.csv`, la liste « si commandes change » devient `['ca_par_canal.sql', 'graphique_t4.png', 'message_gerante.md', 'synthese_t4.csv']` (le graphique y entre).
- **4.7.** L'empreinte du **fichier** prouve que les octets sont les mêmes (rapide, indépendante de la lecture, mais sensible à un simple changement de fin de ligne). L'empreinte du **tableau** ignore ces détails de format, à condition de trier et de fixer les types, mais suppose que la lecture soit elle-même reproductible.
- **4.8.** `rejouer` ne vérifie que les sources déclarées dans `sources`. Pour que la modification de `produits.csv` soit vue, il faudrait déclarer `produits.csv` (et son empreinte) dans la documentation, même si le calcul ne s'en sert pas directement, parce qu'une valeur recalculée par un autre chemin peut en dépendre demain.

Chapitre 5 : ⊕ Confidentialité et anonymisation des données — exercices et applications

Ce chapitre du cahier prolonge le chapitre 5 du livre (complémentaire). Il comprend **sept applications guidées** (petites études sur les fichiers de la boutique, à refaire pas à pas) puis **douze exercices** ★/★★/★★★ avec leurs corrigés. Il est **autonome** : la cellule d'initialisation ci-dessous recharge les données et les fonctions d'aide (build/outils_ch05.py). Rappel : les données sont **simulées**, les noms de personnes sont **inventés**, et rien ici ne remplace un conseil juridique.

```
import sys, os, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
import outils_ch05 as O

T = O.charger()
cli, pro, crm, vcrm, ident, cmd, lig = (T[k] for k in ["clients", "profil", "crm", "vcrm", "ident",
↪ "cmd", "lig"])
x = O.partage(T) # clients + profil, sans nom ni e-mail
QI = ["ville", "annee_naissance", "canal_acquisition", "fidelite"]
print(len(crm), "lignes de CRM |", len(x), "clients |", len(cmd), "commandes")
```

7140 lignes de CRM | 6000 clients | 36395 commandes

Applications

Application 5.1 — Classer les colonnes et normaliser le consentement (section 5.1 du livre)

Objectif. Classer chaque colonne du CRM, puis lire le consentement comme le ferait une campagne prudente.

Étape 1 — Un classement à écrire. Complétez le dictionnaire suivant (famille et conduite), puis comptez les colonnes de chaque famille.

```
classement = {"id_crm": ("identifiant interne", "pseudonyme"), "prenom": ("direct", "retirer"),
↪ "nom": ("direct", "retirer"),
    "email": ("direct", "retirer ou clé"), "telephone": ("direct", "retirer ou clé"),
↪ "ville": ("quasi", "généraliser"),
```

```

    "code_postal": ("quasi", "département"), "date_naissance": ("quasi", "tranche"),
    ↪ "date_inscription": ("quasi", "année"),
    "consentement_marketing": ("administrative", "normaliser"), "source_saisie":
    ↪ ("technique", "conserver")}
fam = pd.Series({c: f for c, (f, _) in classement.items()})
print(fam.value_counts().to_string())
print("colonnes du CRM non classées :", sorted(set(crm.columns) - set(classement)))

```

```

direct          4
quasi           4
identifiant interne  1
administrative   1
technique       1
colonnes du CRM non classées : []

```

À vous. Quelle colonne de `crm_clients.csv` ne figure pas dans votre classement ? Pourquoi est-elle utile à l'analyste mais sans risque ?

Étape 2 — Lire le consentement. On normalise en trois états : accord (oui, Oui, OUI, 0, 1, TRUE), refus explicite (non) et **absence d'information** (vide).

```

def consentement(v):
    if pd.isna(v):
        return "absent"
    return "accord" if v in 0.OUI else "refus"

vraies = crm[crm["prenom"] != "Test"].copy() # on écarte les 140 lignes de test
vraies["consent"] = vraies["consentement_marketing"].map(consentement)
print(vraies["consent"].value_counts().to_string())
print(vraies.groupby("source_saisie")["consent"].value_counts(normalize=True).unstack().round(3,
    ↪ ).to_string())

```

```

consent
accord    4839
absent    2161
consent    absent  accord
source_saisie
caisse      0.310    0.690
import      0.317    0.683
site        0.305    0.695

```

À vous. Quelle source de saisie a le plus de consentements absents ? À quoi attribuez-vous la différence (formulaire en ligne, caisse, import) et que proposeriez-vous à la gérante ?

Application 5.2 — Attaque par dictionnaire et hachage à clé (section 5.2 du livre)

Objectif. Refaire l'attaque de 5.2.2, puis l'améliorer, puis la déjouer.

Étape 1 — Les adresses sans numéro. On hache les e-mails et l'on tente le dictionnaire simple.

```

empreintes = ident["email"].map(0.sha256)
dico = 0.annuaire(ident)
r1 = empreintes.isin(dico.keys())
print("retrouvés par le dictionnaire simple :", int(r1.sum()), f"({r1.mean() * 100:.1f} %)")

```

```
retrouvés par le dictionnaire simple : 5142 (85.7 %)
```

Étape 2 — Un dictionnaire plus riche. Certaines adresses portent un numéro de 1 à 99 après le nom (`prenom.nom63@...`). On ajoute ces variantes.

```

riche = dict(dico)
for p, n in zip(ident["prenom"], ident["nom"]):
    for dom in O.DOMAINES:
        for k in range(1, 100):
            e = f"{O.sans_accent(p).lower()}.{O.sans_accent(n).lower()}{k}@{dom}"
            riche[O.sha256(e)] = e
r2 = empreintes.isin(riche.keys())
print("candidates :", len(riche), "| retrouvés :", int(r2.sum()), f"({r2.mean() * 100:.1f} %)")

```

```
candidates : 1798500 | retrouvés : 6000 (100.0 %)
```

Étape 3 — La clé. Hachez maintenant avec `O.hmac256(email, cle)` et refaites l'attaque avec le dictionnaire riche.

```

cle = b"cle-de-demonstration-du-cahier"
a_cle = ident["email"].map(lambda e: O.hmac256(e, cle))
print("retrouvés après hachage à clé :", int(a_cle.isin(riche.keys()).sum()))

```

```
retrouvés après hachage à clé : 0
```

À vous. Combien de hachages l'attaquant a-t-il calculés pour le dictionnaire riche ? Que se passerait-il s'il connaissait la clé ?

Application 5.3 — Le coût du bruit et de la synthèse (section 5.2.4 du livre)

Objectif. Mesurer ce que coûtent trois protections du revenu : le bruit, l'arrondi, la synthèse.

Étape 1 — Le bruit. Ajoutez un bruit gaussien d'écart-type 2 000, 5 000, puis 10 000 € au revenu et suivez la corrélation avec l'âge et l'écart-type.

```

rng = np.random.default_rng(1)
corr = lambda a, b: round(float(np.corrcoef(a, b)[0, 1]), 3)
lignes = [("brut", corr(x["age"], x["revenu_annuel"]), round(x["revenu_annuel"].std()))]
for s in (2000, 5000, 10000):
    b = x["revenu_annuel"] + rng.normal(0, s, len(x))
    lignes.append((f"bruit {s}", corr(x["age"], b), round(b.std())))
print(pd.DataFrame(lignes, columns=["version", "corrélation âge-revenu",
    ↪ "écart-type"]).to_string(index=False))

```

version	corrélation âge-revenu	écart-type
brut	0.360	11212
bruit 2000	0.355	11382
bruit 5000	0.336	12208
bruit 10000	0.250	15049

Étape 2 — Une synthèse qui garde la corrélation. On ajuste une loi normale à deux variables (moyennes et covariance de l'âge et du revenu) et l'on tire de nouvelles lignes.

```

mu = x[["age", "revenu_annuel"]].mean().values
cov = np.cov(x[["age", "revenu_annuel"]].values.T)
fausses = pd.DataFrame(np.random.default_rng(2).multivariate_normal(mu, cov, len(x)), columns=["age",
    ↪ "revenu_annuel"])
print("corrélation synthétique :", corr(fausses["age"], fausses["revenu_annuel"]), "| revenus
    ↪ synthétiques négatifs :", int((fausses["revenu_annuel"] < 0).sum()))

```

```
corrélation synthétique : 0.362 | revenus synthétiques négatifs : 41
```

À vous. Le jeu synthétique reproduit-il la corrélation ? Reproduit-il la **forme** de la distribution du revenu (asymétrie) ? Comparez l'asymétrie (`skew()`) du revenu réel et du revenu synthétique.

Application 5.4 — Unicité et k-anonymat sur vos propres combinaisons (sections 5.3.1 à 5.3.3 du livre)

Objectif. Explorer quelles combinaisons de colonnes isolent le plus de clients, puis choisir une recette de généralisation.

Étape 1 — Toutes les paires. Parmi les quatre quasi-identifiants, quelle paire isole le plus de clients ?

```
from itertools import combinations
res = []
for c in combinations(QI, 2):
    s = 0.stats_k(x, list(c))
    res.append((" + ".join(c), s["uniques"], s["sous_k"]))
print(pd.DataFrame(res, columns=["paire", "clients uniques", "clients dans un groupe < 5"]).sort_values("clients uniques", ascending=False).to_string(index=False))
```

	paire	clients uniques	clients dans un groupe < 5
ville + annee_naissance		203	1293
annee_naissance + canal_acquisition		13	74
annee_naissance + fidelite		4	31
ville + canal_acquisition		0	0
ville + fidelite		0	0
canal_acquisition + fidelite		0	0

Étape 2 — Largeur de la tranche et taille des régions. On généralise l'année de naissance par tranches de 5, 10 ou 20 ans, et les villes par régions de 2, 5 ou 10 villes. Quelle recette garde les quatre colonnes avec le moins de lignes à supprimer pour $k = 5$?

```
def recette(largeur, villes_par_region):
    g = x.copy()
    g["tranche"] = (g["annee_naissance"] // largeur) * largeur
    g["region"] = (g["ville"].str[-1].map(ord) - 65) // villes_par_region
    s = 0.stats_k(g, ["region", "tranche", "canal_acquisition", "fidelite"])
    return s["groupes"], s["sous_k"], round(s["sous_k"] / len(g) * 100, 1)
tab = pd.DataFrame([(l, v): recette(l, v) for l in (5, 10, 20) for v in (2, 5, 10)], index=["groupes", "lignes < 5", "% à supprimer"]).T
tab.index.names = ["tranche (ans)", "villes par région"]
print(tab.astype({"groupes": int, "lignes < 5": int}).to_string())
```

tranche (ans)	villes par région	groupes	lignes < 5	% à supprimer
5	2	663	683	11.4
	5	301	177	2.9
	10	163	64	1.1
10	2	366	272	4.5
	5	160	76	1.3
	10	84	25	0.4
20	2	228	120	2.0
	5	95	25	0.4
	10	48	8	0.1

À vous. Quelle recette donne le moins de suppressions ? Laquelle perd le plus de résolution ? Laquelle choisiriez-vous si le prestataire s'intéresse surtout à l'effet de l'âge ?

Application 5.5 — Recoupement par date et montant (section 5.3.5 du livre)

Objectif. Reprendre l'attaque de 5.3.5 sur le canal **Boutique** et tester des parades.

Étape 1 — L'unicité des tickets. On prend les commandes de la boutique physique en 2025, avec leur montant.

```
cb = cmd[(cmd["canal"] == "Boutique") & (cmd["date_commande"] >= "2025-01-01")].copy()
cb["total"] = cb["id_commande"].map(lig.groupby("id_commande")["montant"].sum().round(2))
uniq = lambda cols: round((cb.groupby(cols)["total"].transform("size") == 1).mean() * 100, 1)
print("commandes :", len(cb), "| uniques (date, montant exact) :", uniq(["date_commande", "total"]),
      "\n | (montant seul) :", uniq(["total"]), "%")
```

```
commandes : 5442 | uniques (date, montant exact) : 96.6 % | (montant seul) : 17.9 %
```

Étape 2 — Les parades. Retirez la date exacte (on garde la semaine), arrondissez le montant à 10 €, ou les deux.

```
cb["semaine"] = pd.to_datetime(cb["date_commande"]).dt.strftime("%G-%S%V")
cb["total_10"] = (cb["total"] / 10).round() * 10
for nom, cols in {"date + montant exact": ["date_commande", "total"], "semaine + montant exact":
                  ["semaine", "total"],
                  "date + montant à 10 €": ["date_commande", "total_10"], "semaine + montant à 10 €":
                  ["semaine", "total_10"]}.items():
    print(f"{nom:28s} uniques : {uniq(cols)} %")
```

```
date + montant exact      uniques : 96.6 %
semaine + montant exact   uniques : 82.5 %
date + montant à 10 €     uniques : 50.4 %
semaine + montant à 10 €  uniques : 7.8 %
```

À vous. Quelle parade fait le plus baisser l'unicité ? La date retirée, ou le montant arrondi ? À partir de quel niveau d'unicité accepteriez-vous d'envoyer le fichier, et que feriez-vous des commandes encore uniques ?

Application 5.6 — l-diversité et bruit de Laplace (sections 5.3.6 et 5.3.7 du livre)

Objectif. Mesurer l'homogénéité des groupes d'un fichier 5-anonyme, puis publier des comptages bruités.

Étape 1 — Les groupes homogènes. On prend la table 5-anonyme (région, tranche, canal, carte) et l'attribut « très insatisfait » (satisfaction $\leq 2,5$).

```
g = 0.generaliser(x)
k5 = g[0.taille_groupes(g, 0.QI_ENV0I) >= 5].copy()
k5["tres_insatisfait"] = (k5["satisfaction_moy"] <= 2.5).astype(int)
grp = k5.groupby(0.QI_ENV0I).agg(n=("tres_insatisfait", "size"), part=("tres_insatisfait", "mean"))
print("part de très insatisfaits :", round(k5["tres_insatisfait"].mean() * 100, 1), "% | groupes :",
      "\n | len(grp), | groupes sans aucun :", int((grp["part"] == 0).sum()))
print(grp.sort_values("part").tail(3).round(3).to_string())
```

```
part de très insatisfaits : 4.1 % | groupes : 136 | groupes sans aucun : 52
n    part
region  tranche  canal_acquisition  fidelite
Région 4 1985-1994 Réseaux          1         6 0.167
          2005-2014 Boutique        0         6 0.167
Région 1 1945-1954 Réseaux          1         7 0.286
```

Étape 2 — Des comptages bruités. On publie, pour chaque région et tranche, le nombre de clients très insatisfaits, avec un bruit de Laplace d'échelle $1/\varepsilon$ ($\varepsilon = 1$), arrondi puis ramené à 0 s'il est négatif.

```
vrai = k5.groupby(["region", "tranche"])["tres_insatisfait"].sum()
rng = np.random.default_rng(3)
pub = pd.Series([max(0, round(float(0.bruit_laplace(v, 1.0, rng)[0]))) for v in vrai.values],
    ↪ index=vrai.index)
err = (pub - vrai).abs()
print("cellules :", len(vrai), "| erreur absolue moyenne :", round(err.mean(), 2), "| cellules à vrai
    ↪ comptage nul :", int((vrai == 0).sum()), "| dont publiées > 0 :", int(((vrai == 0) & (pub >
    ↪ 0)).sum()))
```

```
cellules : 28 | erreur absolue moyenne : 0.71 | cellules à vrai comptage nul : 5 | dont publiées > 0 : 1
```

À vous. Combien de cellules ont un vrai comptage nul, et combien de fois publie-t-on un comptage positif pour elles ? Que pensez-vous de l'utilité de ces comptages pour les petites cellules ?

Application 5.7 — Le fichier d'envoi, les petits effectifs et le contrôle (section 5.4 du livre)

Objectif. Fabriquer le fichier d'envoi, appliquer la règle des petits effectifs sur un tableau et passer la liste de contrôle automatique.

Étape 1 — Le fichier d'envoi pour deux valeurs de k.

```
cle = b"cle-du-projet-prestataire"
for k in (3, 5, 10):
    f, retirees = 0.preparer_envoi(x, cle, k=k)
    c = 0.controle_avant_envoi(f, 0.QI_ENV01, k=k)
    print(f"k = {k:2d} : {len(f)} lignes envoyées, {retirees} retirées, k obtenu {c['k_min']}, prêt :
    ↪ {c['ok']}")
```

```
k = 3 : 5960 lignes envoyées, 40 retirées, k obtenu 3, prêt : True
k = 5 : 5921 lignes envoyées, 79 retirées, k obtenu 5, prêt : True
k = 10 : 5738 lignes envoyées, 262 retirées, k obtenu 10, prêt : True
```

Étape 2 — Un tableau de synthèse sans petits effectifs. On croise la tranche de dix ans et le canal pour le nombre de clients avec carte, et l'on masque les cellules de moins de 10.

```
t = pd.crosstab(g["tranche"], g["canal_acquisition"], values=g["fidelite"],
    ↪ aggfunc="sum").fillna(0).astype(int)
print(t.where(t >= 10, "<10").to_string())
```

canal_acquisition	Boutique	Réseaux	Site
tranche			
1935-1944	<10	<10	<10
1945-1954	25	<10	18
1955-1964	93	14	77
1965-1974	178	43	158
1975-1984	286	70	232
1985-1994	235	55	199
1995-2004	148	33	101
2005-2014	64	17	43

À vous. Combien de cellules sont masquées avec le seuil de 10 ? Une cellule masquée peut-elle être retrouvée si l'on publie aussi les totaux par ligne ?

Exercices

Exercice 5.1 ★ — Quatre colonnes à classer (section 5.1.2 du livre)

Classez en identifiant direct, quasi-identifiant ou donnée sensible (et dites pourquoi) : (a) l'adresse électronique d'un client ; (b) la ville ; (c) un article acheté qui est un livre de prière ; (d) le numéro de la carte de fidélité d'un client.

Exercice 5.2 ★ — Un consentement contradictoire (section 5.1.4 du livre)

Un client a trois lignes dans le CRM : oui, '(vide) et non'. Quelle règle de lecture retenez-vous pour une campagne ? Et pour une statistique sur la part de clients qui acceptent ? Justifiez.

Exercice 5.3 ★★ — Combien de lignes manque-t-on ? (section 5.1.5 du livre)

Parmi les lignes du CRM qui appartiennent à un client en double, quelle part retrouve-t-on avec une recherche sur l'**e-mail exact**, sur l'e-mail **normalisé**, puis sur la combinaison **nom et prénom normalisés** (minuscules, accents retirés, espaces supprimés) ? Utilisez `verite_crm.csv` pour juger.

Exercice 5.4 ★ — Le dictionnaire d'un numéro de téléphone (section 5.2.2 du livre)

Un numéro de téléphone compte 10 chiffres. Combien de numéros doit-on essayer, au pire, pour retrouver un numéro haché ? Si l'on teste un million de numéros par seconde, combien de temps cela représente-t-il ? Mesurez ensuite sur votre machine le débit de calcul de SHA-256 sur 200 000 numéros.

Exercice 5.5 ★★ — Un sel par ligne (section 5.2.3 du livre)

On pseudonymise les e-mails du CRM et du site avec un hachage où **chaque ligne reçoit un sel aléatoire différent** (sel stocké dans le fichier). Peut-on encore relier le CRM et le site par ces empreintes ? Vérifiez par le calcul et expliquez la différence avec le hachage à clé.

Exercice 5.6 ★★ — Rééchantillonner des lignes n'est pas anonymiser (section 5.2.4 du livre)

Fabriquez un jeu « synthétique » en tirant des **lignes entières** avec remise dans la table `x`. Comparez la corrélation âge-revenu à l'original, puis la part des lignes qui sont des **copies exactes** de lignes réelles. Que concluez-vous sur la protection ?

Exercice 5.7 ★ — Le k-anonymat à la main (section 5.3.2 du livre)

Voici dix clients (région, tranche, carte) : (R1, 1980-89, oui), (R1, 1980-89, oui), (R1, 1980-89, non), (R1, 1990-99, oui), (R2, 1980-89, oui), (R2, 1980-89, oui), (R2, 1980-89, oui), (R2, 1990-99, non), (R2, 1990-99, non), (R2, 1990-99, non). Quel est k ? Combien de clients sont uniques ? Que devient k si l'on retire la carte ? Vérifiez avec `pandas`.

Exercice 5.8 ★★ — Choisir la largeur de la tranche (section 5.3.3 du livre)

Pour des tranches de 5, 10, 15, 20 ans (régions de 5 villes, canal et carte conservés), calculez la part des lignes à supprimer pour $k = 5$, puis la corrélation âge-revenu obtenue avec le milieu de la tranche. Quelle largeur choisissez-vous, et pourquoi ?

Exercice 5.9 ★★★ — Rendre un fichier de commandes inattaquable (section 5.3.5 du livre)

On veut publier les commandes du site en 2025 (pseudonyme, date, montant, mode de livraison). Cherchez une combinaison de transformations (date → mois, montant → arrondi, suppression du mode de livraison...) pour que **moins de 5 %** des commandes soient uniques sur (date ou mois, montant ou arrondi, mode de livraison). Quel est le prix de cette protection pour une analyse du panier moyen par mois ?

Exercice 5.10 ★★ — Mesurer la diversité (section 5.3.6 du livre)

Dans la table 5-anonyme (région, tranche, canal, carte), calculez pour chaque groupe le **nombre de valeurs distinctes** de `satisfaction_moy` arrondie à l'entier (de 1 à 5). Combien de groupes ont moins de 3 valeurs distinctes ? Comment répareriez-vous ces groupes ?

Exercice 5.11 ★★ — Masquer sans trahir (section 5.4.2 du livre)

Voici un tableau de 3 lignes × 3 colonnes, totaux publiés : lignes (A : 12, 9, 3), (B : 4, 15, 11), (C : 20, 8, 2). Totaux de ligne : 24, 30, 30 ; totaux de colonne : 36, 32, 16. Avec un seuil de 5, quelles cellules sont masquées ? Montrez qu'on les retrouve, puis proposez une suppression complémentaire minimale qui empêche cette reconstitution.

Exercice 5.12 ★★★ — La note de transmission (section 5.4.5 du livre)

Rédigez la note de transmission du fichier d'envoi de l'application 5.7 ($k = 5$) : finalité, colonnes transmises et colonnes retirées, transformations, valeur de k et nombre de lignes retirées, résultat du contrôle automatique, destinataire, canal, durée de conservation, clé (où elle est, qui y a accès). Produisez par le code les chiffres de la note.

Corrigés

Corrigé 5.1

- (a) Une adresse électronique est un **identifiant direct** : elle désigne une personne à elle seule.
- (b) La ville est un **quasi-identifiant** : elle n'identifie qu'en se combinant avec d'autres colonnes.
- (c) Un livre de prière est un **achat qui révèle potentiellement une donnée sensible** (une conviction religieuse) : la boutique ne collecte pas la religion, mais l'historique d'achats permet de l'**inférer**, et la protection renforcée peut s'appliquer.
- (d) Le numéro de carte de fidélité est un **identifiant direct** dès qu'il quitte l'organisation : il se relie à la personne dans le système de la boutique.

Corrigé 5.2

Pour une **campagne**, on applique la règle de **précaution** : un non suffit à exclure le client, et un vide n'est pas un accord ; ce client n'est donc **pas contacté**. Pour une **statistique** (« quelle part des clients accepte ? »), on ne force pas la valeur en oui ou en non, car cela fausserait le taux : on crée une catégorie « **lignes contradictoires** », on la compte à part et l'on signale la règle retenue. La règle de lecture pour l'**action** (précautionneuse) et la règle de lecture pour la **mesure** (transparente) ne sont pas les mêmes, et c'est normal.

Corrigé 5.3

```
d = crm.merge(vcrm, on="id_crm").query("id_client > 0")
d = d[d.groupby("id_client")["id_crm"].transform("size") > 1].copy()
vrai = ident.set_index("id_client")
d["email_vrai"], d["nom_vrai"], d["prenom_vrai"] = (d["id_client"].map(vrai[c]) for c in ["email",
↪ "nom", "prenom"])
net = lambda s: s.fillna("").map(0.sans_accents).str.lower().str.replace(r"\s+", "", regex=True)
exact = d["email"] == d["email_vrai"]
norm = d["email"].fillna("").str.strip().str.lower() == d["email_vrai"].str.lower()
nomprenom = (net(d["nom"]) + net(d["prenom"])) == (net(d["nom_vrai"]) + net(d["prenom_vrai"]))
print(len(d), "lignes | e-mail exact :", int(exact.sum()), "| e-mail normalisé :", int(norm.sum()),
↪ "| nom + prénom normalisés :", int(nomprenom.sum()), "| l'un ou l'autre :", int((norm |
↪ nomprenom).sum()))
```

```
1950 lignes | e-mail exact : 1362 | e-mail normalisé : 1624 | nom + prénom normalisés : 1477 | l'un ou
↪ l'autre : 1804
```

La recherche exacte sur l'e-mail ne retrouve qu'environ sept lignes sur dix ; la normalisation de l'e-mail en retrouve davantage ; le nom et le prénom normalisés en retrouvent d'autres, mais **pas toutes** : les lignes au prénom abrégé (P.), au nom et au prénom inversés ou à faute de frappe résistent. Sur 1 950 lignes, 1 362 sont retrouvées par l'e-mail exact (69,8 %), 1 624 par l'e-mail normalisé (83,3 %), 1 477 par le nom et le prénom normalisés et 1 804 par l'un ou l'autre (92,5 %) : il en reste **146** (7,5 %). Pour retrouver le reste il faut le rapprochement approximatif de la section 2.5. **L'effacement est une opération de rapprochement.**

Corrigé 5.4

Au pire, il faut essayer 10^{10} numéros. À un million d'essais par seconde, cela fait 10^4 secondes, soit environ **2 heures 47** : un attaquant motivé le fait sans difficulté, et beaucoup plus vite avec du matériel adapté. Un numéro de téléphone **n'est pas** un secret que protège un hachage nu. Mesurons le débit de la machine sur 200 000 numéros (on affiche un booléen : la durée dépend de la machine).

```
import time
nums = [f"0{i:09d}" for i in range(200_000)]
t0 = time.perf_counter(); empreintes = [0.sha256(n) for n in nums]; dt = time.perf_counter() - t0
debit = len(nums) / dt
print("plus de 100 000 hachages par seconde :", debit > 100_000, "| temps estimé pour 10^10 numéros
↪ supérieur à une heure :", 1e10 / debit > 3600)
```

```
plus de 100 000 hachages par seconde : True | temps estimé pour 10^10 numéros supérieur à une heure :
↪ True
```

Corrigé 5.5

```
import hashlib
site = pd.read_csv(os.path.join(0.D, "site_commandes.csv"))
nm = lambda s: s.dropna().str.strip().str.lower()
avec_sel = lambda source, i, e: hashlib.sha256(f"{source}-{sel{i}}-{e}".encode()).hexdigest()
↳ # un sel différent par ligne
crm_sel = {avec_sel("crm", i, e) for i, e in enumerate(nm(crm["email"]))}
site_sel = {avec_sel("site", i, e) for i, e in enumerate(nm(site["customer_email"]))}
cle_unique = lambda s: {0.hmac256(e, b"cle") for e in nm(s)}
print("empreintes communes avec un sel par ligne :", len(crm_sel & site_sel), "| avec une clé unique
↳ :", len(cle_unique(crm["email"]) & cle_unique(site["customer_email"])))
```

```
empreintes communes avec un sel par ligne : 0 | avec une clé unique : 2845
```

Avec un sel **différent pour chaque ligne**, deux lignes qui portent la même adresse reçoivent des empreintes différentes : plus aucun rapprochement n'est possible. C'est parfois **voulu** (interdire de relier les fichiers d'un prestataire) mais cela détruit l'utilité du lien. Le hachage à **clé unique** conserve le lien (même adresse, même empreinte) tout en interdisant l'attaque par dictionnaire à qui n'a pas la clé.

Corrigé 5.6

```
boot = x.sample(len(x), replace=True, random_state=5).reset_index(drop=True)
reels = set(zip(x["age"], x["revenu_annuel"], x["ville"], x["canal_acquisition"]))
copies = pd.Series(list(zip(boot["age"], boot["revenu_annuel"], boot["ville"],
↳ boot["canal_acquisition"]))).isin(reels).mean()
print("corrélation âge-revenu :", round(float(np.corrcoef(boot["age"], boot["revenu_annuel"])[0, 1]),
↳ 3), "| lignes identiques à une ligne réelle :", f"{copies * 100:.0f} %", "| clients réels absents
↳ du jeu :", int((~x["id_client"].isin(boot["id_client"])).sum()))
```

```
corrélation âge-revenu : 0.358 | lignes identiques à une ligne réelle : 100 % | clients réels absents
↳ du jeu : 2193
```

Le rééchantillonnage **préserve** la corrélation, mais **toutes** les lignes tirées sont des copies de lignes réelles (avec des répétitions, et **2 193 clients réels sur 6 000** absents du jeu, soit plus d'un sur trois). Ce jeu « synthétique » n'apporte **aucune protection** : on y retrouve les individus tels quels. La synthèse ne protège que si elle fabrique de **nouvelles** lignes (par un modèle) et si l'on teste ensuite qu'elle ne reproduit pas d'individus.

Corrigé 5.7

```
d = pd.DataFrame({"region": ["R1"] * 4 + ["R2"] * 6, "tranche": ["80", "80", "80", "90", "80", "80",
↳ "80", "90", "90", "90"],
                  "carte": ["oui", "oui", "non", "oui", "oui", "oui", "oui", "non", "non", "non"]})
g1 = d.groupby(["region", "tranche", "carte"]).size()
print(g1.to_dict(), "| k =", g1.min(), "| clients uniques :", int((g1 == 1).sum()))
print("sans la carte : k =", d.groupby(["region", "tranche"]).size().min())
```

```
{('R1', '80', 'non'): 1, ('R1', '80', 'oui'): 2, ('R1', '90', 'oui'): 1, ('R2', '80', 'oui'): 3,
↳ ('R2', '90', 'non'): 3} | k = 1 | clients uniques : 2
sans la carte : k = 1
```

Les groupes sont (R1, 80, oui) : 2 ; (R1, 80, non) : 1 ; (R1, 90, oui) : 1 ; (R2, 80, oui) : 3 ; (R2, 90, non) : 3. Le plus petit groupe compte 1 : **k = 1**, avec **deux** clients uniques. En retirant la carte, les groupes sont (R1, 80) : 3 ; (R1, 90) : 1 ; (R2, 80) : 3 ; (R2, 90) : 3 : k reste égal à 1 à cause du client (R1, 90). Pour atteindre k = 3, il faudrait en outre fusionner les tranches de la région R1,

ou retirer ce client.

Corrigé 5.8

```
rows = []
for largeur in (5, 10, 15, 20):
    g = x.copy()
    g["tranche"] = (g["annee_naissance"] // largeur) * largeur
    g["region"] = (g["ville"].str[-1].map(ord) - 65) // 5
    s = 0.5 * stats_k(g, ["region", "tranche", "canal_acquisition", "fidelite"])
    milieu = 2025 - (g["tranche"] + (largeur - 1) / 2)
    rows.append((largeur, s["sous_k"], round(s["sous_k"] / len(g) * 100, 1),
    ↪ round(float(np.corrcoef(milieu, g["revenu_annuel"])[0, 1]), 3)))
print(pd.DataFrame(rows, columns=["largeur (ans)", "lignes < 5", "% à supprimer", "corrélation avec
    ↪ le milieu"]).to_string(index=False))
```

largeur (ans)	lignes < 5	% à supprimer	corrélation avec le milieu
5	177	2.9	0.360
10	76	1.3	0.354
15	51	0.9	0.340
20	25	0.4	0.337

Plus la tranche est large, moins il faut supprimer de lignes (2,9 %, 1,3 %, 0,9 % puis 0,4 % pour 5, 10, 15 et 20 ans), et plus la corrélation s'affaiblit (0,360, 0,354, 0,340 puis 0,337). Le choix est un **compromis** : la tranche de 10 ans supprime peu de lignes tout en gardant presque toute la corrélation ; la tranche de 5 ans impose de supprimer plus de deux fois plus de clients ; au-delà de 15 ans la perte d'information devient visible sans gain de protection important. Le chiffre définitif dépend de la question que le prestataire se pose.

Corrigé 5.9

```
cs = cmd[(cmd["canal"] == "Site") & (cmd["date_commande"] >= "2025-01-01")].copy()
cs["total"] = cs["id_commande"].map(lig.groupby("id_commande")["montant"].sum().round(2))
cs["mois"] = cs["date_commande"].str[:7]
for p in (5, 20, 50):
    cs[f"t{p}"] = (cs["total"] / p).round() * p
cs["t0"] = cs["total"]
res = {}
for temps in ("date_commande", "mois"):
    for mont in ("t0", "t5", "t20", "t50"):
        for liv in (True, False):
            cols = [temps, mont] + (["mode_livraison"] if liv else [])
            res[(temps, mont, "avec livraison" if liv else "sans")] =
    ↪ round((cs.groupby(cols)["total"].transform("size") == 1).mean() * 100, 1)
print(pd.Series(res, name="% de commandes uniques").unstack(level=2).to_string())
```

		avec livraison	sans
date_commande	t0	98.4	97.2
	t20	51.3	27.3
	t5	82.4	67.2
	t50	29.2	11.1
	t0	71.3	54.3
mois	t20	2.4	0.7
	t5	9.5	3.0
	t50	1.0	0.3

Avec la **date** et le **montant exact**, presque toutes les commandes sont uniques (98,4 % avec le mode de livraison) ; passer au **mois** et arrondir le montant à 20 € ou 50 € fait tomber l'unicité à 2,4 % et 1,0 % (avec le mode de livraison), alors que l'arrondi à 5 € ne suffit pas (9,5 %). Le

prix pour l'analyse du panier moyen par mois se mesure en comparant les moyennes mensuelles exactes et arrondies.

```
exact = cs.groupby("mois")["total"].mean()
arrondi = cs.groupby("mois")["t20"].mean()
print("écart maximal entre moyennes mensuelles exactes et arrondies à 20 € :", round(float((exact -
↳ arrondi).abs().max()), 2), "€ | panier moyen annuel :", round(float(cs["total"].mean()), 2), "€")
```

```
écart maximal entre moyennes mensuelles exactes et arrondies à 20 € : 0.38 € | panier moyen annuel :
↳ 101.63 €
```

L'arrondi se compense en moyenne : le panier moyen par mois est quasiment inchangé (au plus 0,38 € d'écart pour un panier moyen annuel de 101,63 €). Ce qu'on perd, c'est toute analyse **individuelle** (une commande précise, l'effet d'une promotion un jour donné).

Corrigé 5.10

```
g = 0.generaliser(x)
k5 = g[0.taille_groupes(g, 0.QI_ENV0I) >= 5].copy()
k5["sat_arr"] = k5["satisfaction_moy"].round().astype(int)
div = k5.groupby(0.QI_ENV0I)["sat_arr"].nunique()
tail = k5.groupby(0.QI_ENV0I).size()
print("groupes :", len(div), "| à moins de 3 valeurs distinctes :", int((div < 3).sum()), "| taille
↳ médiane de ces groupes :", int(tail[div < 3].median()), "| taille médiane des autres :",
↳ int(tail[div >= 3].median()))
```

```
groupes : 136 | à moins de 3 valeurs distinctes : 10 | taille médiane de ces groupes : 7 | taille
↳ médiane des autres : 26
```

Dix groupes sur 136 ont moins de 3 valeurs distinctes, et ce sont **les plus petits** (taille médiane 7, contre 26 pour les autres) : cinq ou six personnes ont rarement cinq niveaux de satisfaction différents. Pour les réparer : **fusionner** ces groupes avec un groupe voisin (tranche élargie, canaux regroupés), ce qui augmente k et la diversité ; **généraliser** la valeur sensible (trois classes plutôt que cinq) ; ou **retirer** la colonne sensible si elle n'est pas nécessaire à l'analyse.

Corrigé 5.11

Avec le seuil de 5, trois cellules sont masquées : (A, colonne 3) = 3, (B, colonne 1) = 4 et (C, colonne 3) = 2. Chaque ligne n'a qu'une cellule masquée : on la retrouve par soustraction du total de ligne, A : $24 - (12 + 9) = 3$, B : $30 - (15 + 11) = 4$, C : $30 - (20 + 8) = 2$. Il faut donc une **suppression complémentaire** : que chaque ligne et chaque colonne qui contient une cellule masquée en contienne **au moins deux**. Cherchons l'ensemble minimal par essai exhaustif.

```
import itertools
M = np.array([[12, 9, 3], [4, 15, 11], [20, 8, 2]])
base = {(0, 2), (1, 0), (2, 2)}
def sans_fuite(S):
    return all(sum((r, c) in S for c in range(3)) != 1 for r in range(3)) and all(sum((r, c) in S for
↳ r in range(3)) != 1 for c in range(3))
autres = [(r, c) for r in range(3) for c in range(3) if (r, c) not in base]
for k in range(0, 7):
    sol = [set(e) | base for e in itertools.combinations(autres, k) if sans_fuite(set(e) | base)]
    if sol:
        print("suppressions complémentaires minimales :", k, "| exemple :", sorted(sol[0] - base), "|
↳ ensemble masqué :", sorted(sol[0]))
        break
```

```
suppressions complémentaires minimales : 3 | exemple : [(0, 0), (1, 1), (2, 1)] | ensemble masqué :
↳ [(0, 0), (0, 2), (1, 0), (1, 1), (2, 1), (2, 2)]
```

Il faut masquer **trois cellules de plus**, soit six cellules en tout (par exemple (A, colonne 1), (B, colonne 2) et (C, colonne 2), ce qui masque deux cellules par ligne) : avec deux cellules masquées dans chaque ligne et chaque colonne, les totaux ne donnent plus que la **somme** des cellules masquées, jamais leur valeur. Une alternative plus simple est de **fusionner** des colonnes jusqu'à ce que toutes les cellules dépassent le seuil.

Corrigé 5.12

```
cle = b"cle-du-projet-prestataire"
envoi, retirees = 0.preparer_envoi(x, cle, k=5)
c = 0.controle_avant_envoi(envoi, 0.QI_ENV0I, k=5)
print("colonnes envoyées :", list(envoi.columns))
print("colonnes retirées :", sorted(set(x.columns) - {"revenu_annuel", "satisfaction_moy",
↳ "fidelite", "canal_acquisition"}))
print("lignes envoyées :", len(envoi), "sur", len(x), "| retirées :", retirees, "| k =", c["k_min"],
↳ "| lignes uniques :", c["uniques"], "| contrôle automatique :", c["ok"])

colonnes envoyées : ['pseudonyme', 'region', 'tranche', 'canal_acquisition', 'fidelite',
↳ 'revenu_arrondi', 'satisfaction_moy']
colonnes retirées : ['age', 'annee_naissance', 'depense_2025', 'id_client', 'ville']
lignes envoyées : 5921 sur 6000 | retirées : 79 | k = 5 | lignes uniques : 0 | contrôle automatique :
↳ True
```

Voici la note, que les chiffres ci-dessus permettent de remplir.

Note de transmission. *Finalité* : étudier la relation entre l'âge, le revenu, le canal et la satisfaction (étude commandée par la gérante, sans contact avec les clients). *Colonnes transmises* : pseudonyme, région, tranche de dix ans, canal d'acquisition, carte de fidélité, revenu arrondi au millier, satisfaction moyenne. *Colonnes retirées* : identifiant interne, ville, année de naissance exacte, revenu exact, dépense, âge exact. *Transformations* : pseudonyme à clé (HMAC-SHA-256), généralisation (ville → région, année → tranche), arrondi du revenu, suppression des groupes de moins de 5 personnes. *Valeur de k* : 5 pour les quatre quasi-identifiants ; aucune ligne unique ; 79 lignes retirées sur 6 000. *Contrôle automatique* : aucune colonne suspecte, k = 5 (voir ci-dessus). *Destinataire et canal* : le prestataire, par un espace de partage sécurisé à accès nominatif. *Conservation* : destruction du fichier à la fin de l'étude, au plus tard six mois après l'envoi, attestée par écrit. *Clé* : conservée par la gérante dans un coffre de mots de passe, hors du dossier d'envoi ; le prestataire ne la reçoit pas. *Limites* : le fichier est pseudonymisé, pas anonymisé ; l'inférence sur les valeurs de satisfaction n'est pas éliminée (section 5.3.6).

Pistes des applications

Les pistes ci-dessous donnent les ordres de grandeur attendus pour les questions « À vous » ; les chiffres exacts sortent du code des applications.

Application 5.1. Le classement compte 4 identifiants directs, 4 quasi-identifiants et une colonne de chacune des trois autres familles ; aucune colonne n'est oubliée. Le consentement est absent pour 30,9 % des lignes réelles : 31,0 % pour la caisse, 31,7 % pour les imports, 30,5 % pour le site : l'écart est faible et dans ce jeu le vide est simulé indépendamment de la source. Dans une

situation réelle, une source dont le vide est nettement supérieur désignerait un formulaire ou un import à corriger en amont. Le non n'apparaît que sur les lignes de test (volontairement écartées ici).

Application 5.2. Le dictionnaire riche contient 1 798 500 empreintes (17 985 adresses sans numéro, plus 99 variantes numérotées de chacune) : il retrouve **les 6 000 adresses**, contre 5 142 (85,7 %) avec le dictionnaire simple, et aucune après hachage à clé. Si l'attaquant connaissait la clé, le hachage à clé n'offrirait pas plus de protection que le hachage nu : la clé **est** le secret.

Application 5.3. Le bruit de 2 000 € ramène la corrélation de 0,360 à 0,355 ; celui de 10 000 € à 0,250 (et l'écart-type passe de 11 212 à 15 049 €). La synthèse normale reproduit la corrélation (0,362) mais pas la **forme** : le revenu réel est asymétrique (queue vers les hauts revenus) alors que la loi normale est symétrique ; elle produit même 41 revenus négatifs.

Application 5.4. La paire (ville, année de naissance) isole le plus de clients (203 uniques, 1 293 clients dans un groupe de moins de 5). Les recettes aux tranches larges et aux grandes régions suppriment le moins de lignes (0,1 % pour 20 ans et régions de 10 villes) mais perdent le plus de résolution ; si l'on s'intéresse à l'effet de l'âge, on préfère une **tranche étroite** et une **région large** (tranche de 5 ans, régions de 10 villes : 1,1 % de lignes à supprimer ; l'âge est conservé, la géographie sacrifiée).

Application 5.5. Sur les 5 442 commandes de la boutique, la date et le montant exact en désignent 96,6 % de façon unique. L'arrondi du montant à 10 € fait davantage baisser l'unicité (50,4 %) que le passage de la date à la semaine (82,5 %) ; combiner les deux est le plus efficace (7,8 %). Les commandes qui restent uniques après transformation se **suppriment** du fichier ou se **regroupent** dans une catégorie « autres ».

Application 5.6. La part de clients très insatisfaits est de 4,1 % ; 52 groupes sur 136 n'en comptent aucun, et le groupe le plus exposé en compte 28,6 % (2 sur 7). Avec $\varepsilon = 1$, l'erreur absolue moyenne des 28 comptages publiés est de 0,71 ; 5 cellules ont un vrai comptage nul et une est publiée avec une valeur positive : pour des cellules dont le vrai comptage vaut 0, 1 ou 2, c'est une erreur du même ordre que le comptage lui-même, ce qui illustre la règle « les petits groupes se masquent, ils ne se brulent pas ».

Application 5.7. Plus k est élevé, plus on retire de lignes : 40 pour $k = 3$, 79 pour $k = 5$, 262 pour $k = 10$. Le tableau masqué avec le seuil de 10 comporte 4 cellules < 10 : trois dans la ligne « 1935-1944 » et une (Réseaux, 1945-1954). Si l'on publie aussi les totaux de ligne et de colonne, tout se retrouve : la cellule (1945-1954, Réseaux) se déduit du total de sa ligne, puis la cellule (1935-1944, Réseaux) du total de sa colonne, et les deux autres cellules de la ligne « 1935-1944 » des totaux de leurs colonnes. Il faut donc une suppression complémentaire, ou fusionner les trois premières tranches.

Projet du volume et auto-évaluation — exercices et applications

Ce chapitre du cahier clôt le volume II. Il contient le **projet du volume : nettoyer et réconcilier deux sources désordonnées** (la caisse de la boutique et l'export du site web) en un **jeu de données fiable** des ventes de 2025, avec son journal de nettoyage, ses contrôles, son rapport d'écarts et son dictionnaire ; puis l'**auto-évaluation** (quarante questions). Il ne demande aucune notion nouvelle : chaque étape s'appuie sur un chapitre du livre.

Projet du volume

P.1 Le cahier des charges

La gérante vous écrit : « *La comptable m'annonce un chiffre d'affaires de 2025 que je n'arrive pas à retrouver avec la caisse et le site. Peux-tu me fabriquer une table unique des ventes de l'année, dont je puisse dire d'où vient chaque chiffre ? Et si quelque chose cloche dans les fichiers, dis-le-moi.* »

Vous traduisez en **six exigences** :

1. **Une table unique** des commandes de 2025 pour les deux canaux dont on a un export (Boutique par la caisse, Site par la plateforme), avec une ligne par commande. Le troisième canal, Réseaux, n'apparaît dans **aucun** des deux fichiers : on le signalera.
2. **Un nettoyage jouable** : un script qui part des fichiers bruts, sans retouche manuelle.
3. **Un journal** : chaque étape consigne ses effectifs et ses totaux avant et après.
4. **Des contrôles** qui échouent bruyamment (types, plages, unicité, totaux).
5. **Une réconciliation** avec la base de données de l'entreprise : chaque écart est **expliqué**, ou signalé.
6. **Un dictionnaire de données** de la table finale.

La méthode suit dix étapes, chacune appuyée sur un chapitre du livre :

Étape	Question	Chapitre du livre
P.2 Inventaire	Que contient chaque source ? Quel est son grain ?	3.1, 4.1
P.3 Lire la caisse	Comment lire douze fichiers qui n'ont pas le même format ?	1.4, 2.2
P.4 Nettoyer la caisse	Que faire des montants vides et des lignes en double ?	1.1, 1.3

Étape	Question	Chapitre du livre
P.5 Nettoyer le site	Doublons d'export, tests, annulations, unités : comment s'y retrouver ?	1.3, 1.4
P.6 Réconcilier	Les totaux s'accordent-ils avec la base ? Sinon, pourquoi ?	3.3
P.7 Assembler	Comment fabriquer la table unique ?	2.2
P.8 Contrôler	Les contrôles passent-ils ?	3.2, 3.5
P.9 Documenter	Quel journal, quel dictionnaire ?	4.1, 4.2
P.10 Juger	Peut-on livrer ? Que reste-t-il comme écart ?	3.4

Les données. donnees/caisse/caisse_2025-01.csv ... caisse_2025-12.csv, donnees/site_commandes.csv et donnees/site_lignes.csv, plus la base de référence (commandes.csv, lignes_commande.csv). Elles sont **simulées** : des erreurs ont été **injectées** et leur vérité est dans les fichiers verite_*.csv, que nous n'ouvrons qu'à la fin (P.10), comme on ouvre une correction.

P.2 Étape 1 : l'inventaire des sources

Avant de nettoyer, on regarde ce que l'on a : combien de fichiers, de lignes, quelles colonnes, quel grain (livre, 3.1 et 4.1).

```
import io, glob, os, re
import numpy as np, pandas as pd

fichiers = sorted(glob.glob("donnees/caisse/caisse_2025-*.csv"))
print("fichiers de caisse :", len(fichiers))
for f in (fichiers[0], fichiers[6], fichiers[10]):
    with open(f, "rb") as h:
        debut = h.read(160)
        print(os.path.basename(f), "| BOM :", debut.startswith(b"\xef\xbb\xbf"), "| séparateur détecté",
              "↪ :", ";" if debut.count(b";") > debut.count(b",") else ",")
site = pd.read_csv("donnees/site_commandes.csv", dtype=str)
print("export du site :", len(site), "lignes,", site["order_ref"].nunique(), "références distinctes",
      "↪ :", list(site.columns))
```

```
fichiers de caisse : 12
caisse_2025-01.csv | BOM : False | séparateur détecté : ;
caisse_2025-07.csv | BOM : True | séparateur détecté : ;
caisse_2025-11.csv | BOM : True | séparateur détecté : ,
export du site : 6259 lignes, 6138 références distinctes ; ['order_ref', 'created_at', 'status',
↪ 'customer_email', 'customer_name', 'total', 'currency', 'promo_code', 'shipping_mode']
```

Lecture. Douze fichiers de caisse, dont le format change en juillet (BOM, nouveau format de date) et en octobre (séparateur ,). L'export du site compte 6 259 lignes pour 6 138 références distinctes : 121 lignes sont des répétitions.

On constate déjà : la caisse change de **format** en cours d'année, et l'export du site contient **plus de lignes que de références** : il y a des répétitions à examiner.

P.3 Étape 2 : lire la caisse, douze fichiers, trois formats

Une seule fonction de lecture, **paramétrée** par le mois, plutôt que douze lectures copiées : encodage, séparateur, décimale, date, noms de colonnes (livre, 1.4 et 2.2). On contrôle chaque fichier contre sa propre ligne de total.

```
COLONNES = {"N° ticket": "ticket", "Ticket": "ticket", "Date": "date", "Heure": "heure", "Article":
↳ "article", "Catégorie": "categorie",
    "Qté": "qte", "Quantité": "qte", "Prix unitaire": "prix_unitaire", "Remise (%)":
↳ "remise_pct", "Montant": "montant"}

def lire_caisse(chemin):
    mois = int(re.search(r"_2025-(\d\d)", chemin).group(1))
    enc, sep, dec = ("cp1252", ";", ".",) if mois <= 6 else (("utf-8-sig", ";", ".") if mois <= 9 else
↳ ("utf-8-sig", ",", "."))
    with open(chemin, encoding=enc) as f:
        lignes = f.read().splitlines()
        total = float(lignes[-1].split(sep)[-1].replace(dec, "."))
        debut = next(i for i, l in enumerate(lignes) if re.match(r'^(N° ticket|Ticket)?' +
↳ re.escape(sep), l))
        df = pd.read_csv(io.StringIO("\n".join(lignes[debut:-1])), sep=sep, dtype=str)
        df["numero_ligne"] = df.index + debut + 2 # numéro de ligne dans le fichier
↳ (pour retrouver une erreur)
        df = df[df.iloc[:, 0] != df.columns[0]].rename(columns=COLONNES)
        for c in ["prix_unitaire", "montant"]:
            df[c] = pd.to_numeric(df[c].str.replace(",", ".")) if dec == "," else df[c], errors="coerce")
        df["qte"] = df["qte"].astype(int)
        df["date"] = pd.to_datetime(df["date"], format="%d/%m/%y" if 7 <= mois <= 9 else "%d/%m/%Y")
        df["remise_pct"] = pd.to_numeric(df.get("remise_pct"), errors="coerce")
        df["fichier"] = os.path.basename(chemin)
        return df, total

lectures = [lire_caisse(f) for f in fichiers]
caisse = pd.concat([d for d, _ in lectures], ignore_index=True)
controle = pd.DataFrame({"fichier": [os.path.basename(f) for f in fichiers], "lignes": [len(d) for d,
↳ _ in lectures],
    "somme_lue": [round(d["montant"].sum(), 2) for d, _ in lectures],
↳ "total_affiche": [t for _, t in lectures]})
controle["ecart"] = (controle["total_affiche"] - controle["somme_lue"]).round(2)
print(controle.head(3).to_string(index=False))
print("lignes lues :", len(caisse), "| montants vides :", int(caisse["montant"].isna().sum()), "|
↳ total affiché des douze fichiers :", round(controle["total_affiche"].sum(), 2))
print("somme des montants présents :", round(caisse["montant"].sum(), 2))
```

	fichier	lignes	somme_lue	total_affiche	ecart
caisse_2025-01.csv	955	37826.31	38882.41	1056.10	
caisse_2025-02.csv	770	32273.46	33079.41	805.95	
caisse_2025-03.csv	916	37863.67	39038.90	1175.23	
lignes lues : 12678 montants vides : 399 total affiché des douze fichiers : 560973.91					
somme des montants présents : 547896.42					

Lecture. 12 678 lignes lues, dont 399 sans montant. La somme des montants présents (547 896,42 €) est inférieure de 13 077,49 € au total affiché par la caisse (560 973,91 €) : l'écart est dominé par les montants vides, mais les lignes en double, elles, **gonflent** la somme en sens inverse.

Les douze fichiers se lisent avec la **même** fonction, et **aucun** ne retombe sur son total : l'écart a deux sources possibles, les **montants vides** (qui font baisser la somme) et les **lignes en double** (qui la font monter).

P.4 Étape 3 : nettoyer la caisse

On traite les deux problèmes **l'un après l'autre**, en mesurant chaque fois l'effet (livre, 1.1 et 1.3). D'abord les montants vides : ils se reconstituent par quantité × prix unitaire (en retirant la remise quand le fichier la donne, à partir d'octobre).

```
journal = []
def noter(etape, df, note=""):
    journal.append({"etape": etape, "lignes": len(df), "somme": round(float(df["montant_net"].sum()),
↪ 2) if "montant_net" in df else None, "note": note})

caisse["remise_pct"] = caisse["remise_pct"].fillna(0)
caisse["montant_calcule"] = (caisse["qte"] * caisse["prix_unitaire"] * (1 - caisse["remise_pct"] /
↪ 100)).round(2)
caisse["montant_net"] = caisse["montant"].fillna(caisse["montant_calcule"])
caisse["montant_reconstitue"] = caisse["montant"].isna()
noter("caisse lue", caisse, "douze fichiers, montants vides reconstitués par quantité x prix")
print("montants reconstitués :", int(caisse["montant_reconstitue"].sum()), "| somme après
↪ reconstitution :", round(caisse["montant_net"].sum(), 2))
print("écart avec le total affiché :", round(caisse["montant_net"].sum() -
↪ controle["total_affiche"].sum(), 2))
```

```
montants reconstitués : 399 | somme après reconstitution : 564472.59
écart avec le total affiché : 3498.68
```

Lecture. Après reconstitution des 399 montants, la somme dépasse le total affiché de 3 498,68 € : la reconstitution a bien comblé le vide, et il reste un **excédent** à expliquer.

Il reste un écart **positif** : il vient des lignes en double. Mais attention : une ligne identique à une autre dans un même ticket peut être un **vrai double achat** (deux articles identiques) ou un **double scan**. Le fichier seul ne permet pas de trancher ; on s'appuie sur la **base de référence**, qui contient le total de chaque commande (livre, 3.3).

```
cmd = pd.read_csv("donnees/commandes.csv"); lig = pd.read_csv("donnees/lignes_commande.csv")
base = lig.merge(cmd[["id_commande", "canal", "date_commande"]], on="id_commande")
base = base[base["date_commande"] >= "2025-01-01"]
base_boutique = base[base["canal"] == "Boutique"].groupby("id_commande")["montant"].sum().round(2)
caisse["id_commande"] = caisse["ticket"].str[1:].astype(int)
tk = caisse.groupby("id_commande")["montant_net"].sum().round(2)
cmp_ = pd.concat([tk.rename("caisse"), base_boutique.rename("base")], axis=1)
cmp_["ecart"] = (cmp_["caisse"] - cmp_["base"]).round(2)
print("tickets en caisse :", len(tk), "| commandes Boutique en base :", len(base_boutique))
print("tickets dont le total diffère de la base :", int((cmp_["ecart"].abs() > 0.005).sum()), "| tous
↪ positifs :", bool((cmp_["ecart"] > -0.005).all()))
```

```
tickets en caisse : 5442 | commandes Boutique en base : 5442
tickets dont le total diffère de la base : 102 | tous positifs : True
```

Lecture. La caisse et la base contiennent les **mêmes** 5 442 commandes ; 102 tickets ont un total supérieur à celui de la base, aucun n'est inférieur. Un écart toujours positif évoque des lignes en trop, pas des lignes manquantes.

```
suspects = cmp_[cmp_["ecart"] > 0.005].index
caisse["rang"] = caisse.groupby(["id_commande", "article", "qte", "prix_unitaire"]).cumcount()
cand = caisse[caisse["id_commande"].isin(suspects) & (caisse["rang"] > 0)]
retire = cand[cand["montant_net"].round(2).values == cmp_.loc[cand["id_commande"],
↪ "ecart"].round(2).values]
caisse["doublon_scan"] = caisse.index.isin(retire.index)
propre = caisse[~caisse["doublon_scan"]].copy()
noter("doublons de scan retirés", propre, f"{int(caisse['doublon_scan'].sum())} lignes dont le
↪ montant égale exactement l'écart du ticket")
print("lignes retirées :", int(caisse["doublon_scan"].sum()), "| écart restant avec le total affiché
↪ :", round(propre["montant_net"].sum() - controle["total_affiche"].sum(), 2))
```

```
lignes retirées : 68 | écart restant avec le total affiché : 180.78
```

Lecture. 68 lignes sont retirées, et l'écart avec le total affiché tombe de 3 498,68 € à 180,78 €. Le reliquat n'est pas encore expliqué : nous y reviendrons à l'étape de jugement (P.10).

Attention. La règle « supprimer la ligne dont le montant égale l'écart du ticket » est **prudente** : elle ne retire que ce que l'on peut **prouver** avec la base. Un `drop_duplicates` aveugle aurait supprimé aussi des vrais doubles achats. Le reliquat qui subsiste est documenté, pas caché.

P.5 Étape 4 : nettoyer l'export du site

L'export du site cumule cinq problèmes : doublons d'export, commandes de test, annulations, statuts écrits de trois façons, montants en texte et **unité qui change en cours d'année** (livre, 1.3 et 1.4). On les traite un par un, en notant chaque effectif.

```
site = pd.read_csv("donnees/site_commandes.csv", dtype=str)
n0 = len(site)
site = site.drop_duplicates("order_ref")
n1 = len(site)
test = site["customer_email"].str.strip().str.lower().eq("test@example.com")
site = site[~test].copy()
site["statut"] = site["status"].str.lower()
print("lignes brutes :", n0, "| après doublons :", n1, "| après commandes de test :", len(site))
print("statuts normalisés :", site["statut"].value_counts().to_dict())
```

```
lignes brutes : 6259 | après doublons : 6138 | après commandes de test : 6078
statuts normalisés : {'paid': 5897, 'cancelled': 181}
```

Lecture. 6 259 lignes brutes, 6 138 après suppression des 121 répétitions, 6 078 après retrait des 60 commandes de test. Les trois écritures du statut (paid, PAID, Paid) se ramènent à un seul statut paid ; il reste 181 commandes annulées.

```
def en_nombre(t):
    return float(t.replace("€", "").replace(" ", "").strip().replace(" ", "").replace(",", "."))

site["total_num"] = site["total"].map(en_nombre)
site["date"] = pd.to_datetime(site["created_at"].str.replace("Z", "", regex=False).str.replace("T", " ", ""))
avant = site.loc[site["date"] < "2025-09-15", "total_num"]
apres = site.loc[site["date"] >= "2025-09-15", "total_num"]
print("total moyen avant le 15/09 :", round(avant.mean(), 2), "| après :", round(apres.mean(), 2), "| rapport :", round(apres.mean() / avant.mean(), 1))
```

```
total moyen avant le 15/09 : 103.3 | après : 9913.89 | rapport : 96.0
```

Le montant moyen **est multiplié par cent** à partir du 15 septembre : ce n'est pas un miracle commercial, c'est un **changement d'unité** (centimes) non annoncé. On le corrige à **partir de la date de rupture**, puis on vérifie que la distribution redevient continue.

```
site.loc[site["date"] >= "2025-09-15", "total_num"] /= 100
print("total moyen avant :", round(site.loc[site["date"] < "2025-09-15", "total_num"].mean(), 2), "| après correction :", round(site.loc[site["date"] >= "2025-09-15", "total_num"].mean(), 2))
site["id_commande"] = site["order_ref"].str[4:].astype(int)
site["annulee"] = site["statut"].eq("cancelled")
print("commandes du site après nettoyage :", len(site), "| dont annulées :", int(site["annulee"].sum()), "| somme :", round(site["total_num"].sum(), 2))
```

```
total moyen avant : 103.3 | après correction : 99.14
commandes du site après nettoyage : 6078 | dont annulées : 181 | somme : 617715.45
```

Lecture. Le montant moyen d'une commande est de 103,30 € avant le 15 septembre et de 9 913,89 € après : un rapport de 96. Divisé par cent à partir de cette date, il devient 99,14 €, cohérent avec les mois précédents. Il reste 6 078 commandes, dont 181 annulées, pour 617 715,45 € avant exclusion des annulations.

P.6 Étape 5 : réconcilier avec la base

Avec la base, on construit l'« escalier des écarts » : on part du chiffre d'affaires de la base pour le canal Site et l'on retrouve celui de l'export, étape par étape (livre, 3.3).

```
base_site = base[base["canal"] == "Site"].groupby("id_commande")["montant"].sum().round(2)
ca_base_site = base_site.sum()
m = site.set_index("id_commande")["total_num"].round(2)
joint = pd.concat([m.rename("export"), base_site.rename("base")], axis=1)
print("commandes en base :", len(base_site), "| dans l'export nettoyé :", len(m), "| clés communes :",
      ↪ int(joint.dropna().shape[0]))
print("écart maximal par commande :", round((joint["export"] - joint["base"]).abs().max(), 2))
annule = site.loc[site["annulee"], "total_num"].sum()
print("CA base :", round(ca_base_site, 2), "| export nettoyé :", round(m.sum(), 2), "| dont
      ↪ annulations :", round(annule, 2), "| export hors annulations :", round(m.sum() - annule, 2))
```

```
commandes en base : 6078 | dans l'export nettoyé : 6078 | clés communes : 6078
écart maximal par commande : 0.0
CA base : 617715.45 | export nettoyé : 617715.45 | dont annulations : 17551.32 | export hors
      ↪ annulations : 600164.13
```

Lecture. Les 6 078 commandes du site se retrouvent dans la base, **à l'euro près** (écart maximal par commande nul). La différence de 17 551,32 € entre la base et le chiffre d'affaires « hors annulations » (600 164,13 €) n'est pas une erreur : c'est la valeur des 181 commandes annulées, que la base compte et que la plateforme exclut.

```
ca_canal = base.groupby("canal")["montant"].sum()
print("part du chiffre d'affaires 2025 de la base, par canal (%) :", (ca_canal / ca_canal.sum() *
      ↪ 100).round(1).to_dict())
```

```
part du chiffre d'affaires 2025 de la base, par canal (%) : {'Boutique': 42.3, 'Réseaux': 11.0,
      ↪ 'Site': 46.6}
```

Un contrôle de **couverture** : les deux exports ne couvrent que deux canaux sur trois. Le canal Réseaux, qui pèse environ un neuvième du chiffre d'affaires, n'est dans aucun fichier : on ne peut donc **pas** retrouver le chiffre d'affaires total de la comptable avec ces sources, et il faudra le dire.

Les deux sources **concordent commande par commande** une fois les doublons, les tests et l'unité traités. Reste une **décision**, pas un écart : la base compte les commandes annulées comme des ventes, la plateforme les marque « annulées ». Quel est le chiffre d'affaires de la gérante ? La réponse dépend de la définition, qu'il faut **écrire** (livre, 4.2) ; ici, une commande annulée n'est pas une vente.

P.7 Étape 6 : assembler la table unique

On met les deux canaux au **même grain** (une ligne par commande) et aux **mêmes colonnes** (livre, 2.2 et 2.3).

```

c_caisse = propre.groupby("id_commande").agg(date=("date", "first"), montant=("montant_net", "sum"),
↳ lignes=("article", "size"),
                                reconstitue=("montant_reconstitue", "max")).reset_index()
c_caisse["canal"], c_caisse["source"] = "Boutique", "caisse"
c_site = site[~site["annulee"]][["id_commande", "date", "total_num"]].rename(columns={"total_num":
↳ "montant"})
c_site["date"] = c_site["date"].dt.normalize()
c_site["canal"], c_site["source"], c_site["reconstitue"] = "Site", "site", False
ventes = pd.concat([c_caisse, c_site[c_caisse.columns.drop("lignes")]], ignore_index=True)
ventes["montant"] = ventes["montant"].round(2)
ventes["mois"] = ventes["date"].dt.to_period("M").astype(str)
print(ventes.groupby("canal").agg(commandes=("id_commande", "nunique"), ca=("montant",
↳ "sum")).round(2))
print("clés uniques :", ventes["id_commande"].is_unique, "| lignes :", len(ventes))

```

	commandes	ca
canal		
Boutique	5442	561154.69
Site	5897	600164.13
clés uniques :	True lignes : 11339	

Lecture. La table compte 11 339 commandes : 5 442 pour la Boutique (561 154,69 €) et 5 897 pour le Site (600 164,13 €), avec des clés uniques.

P.8 Étape 7 : les contrôles

Une table livrée sans contrôle est une table **qu'on espère juste**. On écrit des contrôles qui **échouent bruyamment** : pandera pour le schéma (type, plage, unicité), et des contrôles de totaux contre les sources (livre, 3.2 et 3.5).

```

import pandera.pandas as pa

schema = pa.DataFrameSchema({
    "id_commande": pa.Column(int, unique=True),
    "date": pa.Column("datetime64[us]", pa.Check.in_range(pd.Timestamp("2025-01-01"),
↳ pd.Timestamp("2025-12-31"))),
    "montant": pa.Column(float, pa.Check.in_range(0.01, 5000)),
    "canal": pa.Column(str, pa.Check.isin(["Boutique", "Site"])),
}, coerce=True)

try:
    schema.validate(ventes[["id_commande", "date", "montant", "canal"]], lazy=True)
    print("schéma pandera : OK")
except pa.errors.SchemaErrors as e:
    print("échecs de schéma :", len(e.failure_cases))

```

schéma pandera : OK

```

ca_attendu_site = base_site.sum() - annule
ca_attendu_boutique = base_boutique.sum()
verif = {"clés uniques": ventes["id_commande"].is_unique,
        "Site = base hors annulations (à 1 centime)": abs(ventes.loc[ventes["canal"] == "Site",
↳ "montant"].sum() - ca_attendu_site) < 0.01,
        "Boutique = total affiché par la caisse (à 0,1 %)": abs(ventes.loc[ventes["canal"] ==
↳ "Boutique", "montant"].sum() - controle["total_affiche"].sum()) < 0.001 *
↳ controle["total_affiche"].sum(),
        "aucune vente de plus de 5 000 €": bool((ventes["montant"] < 5000).all()),
        "douze mois présents": ventes["mois"].nunique() == 12}

for k, v in verif.items():
    print("OK " if v else "KO ", k)

```

OK clés uniques
 OK Site = base hors annulations (à 1 centime)
 OK Boutique = total affiché par la caisse (à 0,1 %)
 OK aucune vente de plus de 5 000 €
 OK douze mois présents

P.9 Étape 8 : le journal et le dictionnaire

Le journal est **déjà écrit** pendant le nettoyage ; on le complète et on le range en tableau. Le dictionnaire se **génère** à partir de la table, puis on y ajoute à la main ce que la machine ne sait pas (définitions, unités, règles) (livre, 4.1 et 4.2).

```
noter("export du site lu", pd.DataFrame({"montant_net": site["total_num"]}), "doublons, tests retirés  

↳ ; montants en euros (unité corrigée après le 15/09)")
noter("table des ventes", ventes.rename(columns={"montant": "montant_net"}), "caisse + site hors  

↳ annulations, une ligne par commande")
print(pd.DataFrame(journal).to_string(index=False))
```

	etape	lignes	somme	
↳ note				
	caisse lue	12678	564472.59	douze fichiers, montants vides reconstitués par
↳ quantité x prix				
doublons de scan retirés		12610	561154.69	68 lignes dont le montant égale exactement
↳ l'écart du ticket				
	export du site lu	6078	617715.45	doublons, tests retirés ; montants en euros (unité
↳ corrigée après le 15/09)				
	table des ventes	11339	1161318.82	caisse + site hors annulations, une
↳ ligne par commande				

```
descriptions = {"id_commande": "Numéro de commande (clé unique, commun à la caisse, au site et à la  

↳ base)", "date": "Date de la commande (jour)",  

               "montant": "Montant TTC en euros, remises déduites", "lignes": "Nombre de lignes  

↳ d'achat (caisse seulement)",  

               "reconstitue": "Vrai si un montant de ligne a été reconstitué par quantité x prix",  

↳ "canal": "Boutique ou Site", "source": "Fichier d'origine : caisse ou site",  

               "mois": "Mois de la commande (AAAA-MM)}"
dico = pd.DataFrame({"colonne": ventes.columns, "type": [str(t) for t in ventes.dtypes], "manquants":  

↳ ventes.isna().sum().values,  

               "distincts": ventes.nunique().values, "description": [descriptions[c] for c in  

↳ ventes.columns]})
print(dico.to_string(index=False))
```

	colonne	type	manquants	distincts	
↳ description					
id_commande		int64	0	11339	Numéro de commande (clé unique, commun à la caisse, au
↳ site et à la base)					
	date	datetime64[us]	0	365	Date de
↳ la commande (jour)					
	montant	float64	0	2606	Montant TTC en euros,
↳ remises déduites					
	lignes	float64	5897	8	Nombre de lignes d'achat
↳ (caisse seulement)					
reconstitue		bool	0	2	Vrai si un montant de ligne a été reconstitué
↳ par quantité x prix					
	canal	str	0	2	
↳ Boutique ou Site					
	source	str	0	2	Fichier d'origine
↳ : caisse ou site					
	mois	str	0	12	Mois de la
↳ commande (AAAA-MM)					

P.10 Étape 9 : juger, puis ouvrir la correction

On fixe les critères de livraison **avant** d'ouvrir les fichiers de vérité, puis on mesure ce qui reste (livre, 3.4).

```
vs = pd.read_csv("donnees/verite_site.csv"); vc = pd.read_csv("donnees/verite_caisse.csv")
vrai_site = vs[vs["default"].fillna("") == ""].drop_duplicates("order_ref")
ca_vrai_boutique = base_boutique.sum()
ca_boutique = ventes.loc[ventes["canal"] == "Boutique", "montant"].sum()
retirees = caisse.loc[caisse["doublet_scan"], ["fichier", "numero_ligne"]].merge(vc,
↳ left_on=["fichier", "numero_ligne"], right_on=["fichier", "numero_ligne_fichier"], how="left")
print("doublets de scan réels :", int(vc["est_doublet"].sum()), "| lignes retirées :", len(retirees),
↳ "| dont vraies doublets :", int(retirees["est_doublet"].sum()))
print("CA Boutique : table", round(ca_boutique, 2), "| vérité", round(ca_vrai_boutique, 2), "|
↳ écart", round(ca_boutique - ca_vrai_boutique, 2))
print("CA Site : table", round(ventes.loc[ventes["canal"] == "Site", "montant"].sum(), 2), "| vérité
↳ hors annulations", round(vrai_site["total_vrai"].sum(), 2))
```

```
doublets de scan réels : 67 | lignes retirées : 68 | dont vraies doublets : 67
CA Boutique : table 561154.69 | vérité 560973.91 | écart 180.78
CA Site : table 600164.13 | vérité hors annulations 600164.13
```

Lecture. Le chiffre d'affaires du Site est **exact** (écart nul). Celui de la Boutique est supérieur de 180,78 € (0,03 %) à la vérité. Deux causes, que l'on peut chiffrer : les 398 montants reconstitués par quantité × prix ignorent les remises de janvier à septembre (+241,68 € au total), et une ligne de 60,90 € (une « Jardinière design » du ticket 34537) a été retirée à tort, car elle ressemblait à un doublet (−60,90 €) : $241,68 - 60,90 = 180,78$. Sur 67 doublets de scan réels, la règle en a retiré 67 (et une de trop).

```
criteres = {"clés uniques": verif["clés uniques"], "Site égal à la base hors annulations": verif["Site
↳ = base hors annulations (à 1 centime)"],
            "écart Boutique inférieur à 0,1 % du chiffre d'affaires": abs(ca_boutique -
↳ ca_vrai_boutique) < 0.001 * ca_vrai_boutique,
            "au moins 95 % des lignes retirées sont de vrais doublets": retirees["est_doublet"].sum()
↳ >= 0.95 * len(retirees),
            "journal et dictionnaire produits": len(journal) >= 3 and len(dico) == len(ventes.columns)}
for k, v in criteres.items():
    print("OK " if v else "KO ", k)
print("\nDécision :", "LIVRER la table, avec le rapport d'écarts" if all(criteres.values()) else "NE
↳ PAS LIVRER : revoir le nettoyage")
```

```
OK clés uniques
OK Site égal à la base hors annulations
OK écart Boutique inférieur à 0,1 % du chiffre d'affaires
OK au moins 95 % des lignes retirées sont de vrais doublets
OK journal et dictionnaire produits
```

```
Décision : LIVRER la table, avec le rapport d'écarts
```

À retenir. Une table propre n'est pas une table « corrigée » : c'est une table dont on connaît **chaque transformation**, dont les **totaux** s'accordent avec une source de référence, et dont les **écarts résiduels** sont écrits. Le nettoyage est fini quand on peut **expliquer** les chiffres, pas quand ils « ont l'air bons ».

P.11 Les limites de l'étude

- **Une base de référence qui est elle-même une source.** Ici, la base concorde avec les deux exports une fois nettoyés ; dans la réalité, c'est la **réconciliation** qui décide laquelle des sources est la plus fiable.
- **Remises avant octobre.** Les fichiers de caisse de janvier à septembre ne contiennent pas la remise : un montant reconstitué par quantité × prix **ignore** une éventuelle remise.
- **Un canal absent.** Le canal Réseaux (environ 11 % du chiffre d'affaires de 2025 dans la base) n'est exporté par aucun des deux fichiers : la table unique ne couvre que deux canaux sur trois, et le chiffre d'affaires qu'elle donne **ne peut pas** être comparé tel quel à celui de la comptable. Un contrôle de couverture (quels canaux, quelle part du total de la base) fait partie de toute réconciliation.
- **Annulations.** La décision « une annulation n'est pas une vente » dépend de la définition comptable de la boutique, pas d'une règle de nettoyage.
- **Pas de rapprochement des clients.** La table est au grain de la commande ; relier les commandes au CRM suppose le rapprochement flou du chapitre 2 (section 2.5).
- **Données simulées.** Les erreurs injectées sont plus propres que celles de la réalité (typographies, fichiers corrompus, changements non documentés).

P.12 Variante : dédoublonner le CRM

La même démarche s'applique au CRM (donnees/crm_clients.csv, 140 lignes de test, 1 000 doublons) : **normaliser, rapprocher, juger** par précision et rappel. Version courte avec rapidfuzz (livre, 2.5).

```
from unidecode import unidecode

crm = pd.read_csv("donnees/crm_clients.csv", dtype=str)
crm = crm[crm["email"].fillna("") != "test@example.com"].copy()
simple = lambda s: re.sub(r"[^a-z]", "", unidecode(str(s)).lower())
valide = crm["email"].str.contains(r"^[^@\\s]+@[^@\\s]+\\. [a-z]+$", na=False, case=False)
cles = {"e-mail": crm["email"].str.strip().str.lower().where(valide),
        "nom + initiale + année": crm["nom"].map(simple) + "|" + crm["prenom"].map(lambda s:
        ↪ simple(s)[:1]) + "|" + crm["date_naissance"].fillna("").str[-4:]}
def paires(cle):
    g = crm.assign(k=cle).dropna(subset=[cle]).groupby(cle)[cle].apply(list)
    return {(a, b) for liste in g if len(liste) > 1 for i, a in enumerate(liste) for b in liste[i + 1:]}
vcrm = pd.read_csv("donnees/verite_crm.csv", dtype={"id_crm": str}); vcrm = vcrm[vcrm["id_client"] >=
    ↪ 0]
vrai = vcrm.set_index("id_crm")["id_client"]
n_vraies = int(vcrm.groupby("id_client").size().pipe(lambda s: (s * (s - 1) / 2).sum()))
resultats = {nom: paires(cle) for nom, cle in cles.items()}
resultats["union"] = resultats["e-mail"] | resultats["nom + initiale + année"]
for nom, p in resultats.items():
    bons = sum(vrai[a] == vrai[b] for a, b in p)
    print(f"{nom:24s} paires {len(p):4d} | précision {bons / len(p):.3f} | rappel {bons /
    ↪ n_vraies:.3f}")
```

e-mail	paires	698		précision	0.994		rappel	0.661
nom + initiale + année	paires	462		précision	0.916		rappel	0.403
union	paires	878		précision	0.951		rappel	0.795

Lecture. Il existe 1 050 paires réelles de doublons. L'**e-mail** est très précis (99,4 %) mais ne retrouve que 66 % des paires, car l'adresse est parfois absente ou modifiée ; la clé « nom + initiale + année » en retrouve 40 % avec 92 % de précision. Leur **union** gagne en rappel (79,5 %) en perdant un peu de précision (95,1 %). Pour aller plus loin (fautes de frappe, inversion du nom et du prénom), il faut une mesure de similarité et des seuils : c'est le sujet de la section 2.5.

Auto-évaluation

Mode d'emploi. Répondez à voix haute ou par écrit **avant** de lire le corrigé, en une ou deux phrases. Les sections à relire sont indiquées dans le corrigé. Trente-cinq bonnes réponses sur quarante signalent un volume bien assimilé ; les questions des sections facultatives (⊕ 1.5, 1.6, 2.4, 2.5, 3.4, 3.5, 4.3 et chapitre 5) comptent si vous les avez lues.

Nettoyage des données (chapitre 1)

1. Le revenu manque plus souvent chez les moins de 30 ans ; la satisfaction manque plus souvent quand elle est basse ; la dépense manque au hasard pour 5 % des clients. Nommez le mécanisme de chaque absence (MCAR, MAR, MNAR) et dites ce que cela change à une suppression des lignes incomplètes.
2. Un montant vaut 9999 dans trois lignes sur 6 000. Que représente probablement ce nombre, et quel est l'effet sur la moyenne de trois valeurs 20, 30 et 9999 ?
3. Pourquoi une valeur aberrante n'est-elle pas toujours une erreur ? Citez trois façons de la repérer.
4. Une caisse contient 154 lignes identiques à une autre ligne du même ticket. Peut-on les supprimer d'un `drop_duplicates` ? Que faire ?
5. Le montant moyen d'une commande du site passe de 103,30 € à 9 913,89 € au 15 septembre. Quelle est l'explication la plus probable, comment la confirmer et comment corriger ?
6. Une date s'écrit 03/04/2025. Est-ce le 3 avril ou le 4 mars ? Comment lever le doute ?
7. La colonne « ville » contient `Ville A`, `VILLE A`, `ville a`, `Vile A`, `Ville A.` et `المدينة أ`. Quelles écritures une normalisation simple (casse, espaces, point) corrige-t-elle, et lesquelles demandent une table de correspondance ?
8. Le texte `ÃotÃo` apparaît dans un fichier. D'où vient cette erreur et comment la corriger ?
9. On remplace tous les revenus manquants par la moyenne. Quel est l'effet sur la moyenne et sur la dispersion ? Cette méthode répare-t-elle une absence de type MNAR ?

Transformation et fusion (chapitre 2)

10. Joindre 12 678 lignes de caisse à un catalogue de produits **sur le nom** donne 25 356 lignes. Que s'est-il passé, comment le détecter avant de sommer, et comment corriger ?
11. Citez deux contrôles à faire **avant** et **après** une jointure.
12. Douze fichiers de caisse n'ont pas le même format. Comment les empiler sans erreur ?
13. Un article à 50 € TTC (TVA à 20 %) coûte 22 € à l'achat. Quelle est sa marge brute hors taxe, et le taux de marge ?
14. Quelle est la différence entre `cut` et `qcut` ?
15. On passe du grain « ligne de commande » au grain « commande ». Quelle vérification garantit que rien n'a été perdu ?
16. Dans le tableur de stocks, les cellules contiennent `ND`, `-`, `rupture` et `28` (avec un espace). Comment passer à un tableau propre (une ligne, un produit, un mois) ?
17. Le nom Dormar est saisi `Dorrmar`. Quelle est la distance de Levenshtein, et quel score de ressemblance (0 à 100) donne `fuzz.ratio` ? Pourquoi ne pas fusionner automatiquement au-dessus d'un seuil unique ?

Qualité et réconciliation (chapitre 3)

18. Donnez un indicateur chiffré pour chacune des quatre dimensions : exactitude, complétude, cohérence, actualité.
19. Qu'est-ce qu'un contrôle « qui renvoie ses échecs », et pourquoi est-ce plus utile qu'un simple oui/non ?
20. Expliquez la méthode de réconciliation en trois temps.
21. La base compte 181 commandes de plus que la plateforme (17 551,32 €). Est-ce une erreur ? Que faut-il décider ?
22. Une tolérance de 0,1 % s'applique à un total de 560 973,91 €. À partir de quel écart absolu un contrôle échoue-t-il ? Un écart de 180,78 € est-il toléré ?
23. Comment trie-t-on un rapport d'exceptions pour agir d'abord là où il y a le plus à gagner ?
24. Quand choisir pandera, Great Expectations ou de simples fonctions ?
25. Une colonne est « complète à 100 % » : toutes ses cellules sont remplies de N/A. Que montre cet exemple sur les indicateurs de qualité ?

Documentation (chapitre 4)

26. Que doit contenir la fiche d'un jeu de données ? Citez six éléments.
27. Un journal de nettoyage indique : 7 140 lignes lues, 140 tests retirés, 677 doublons retirés, 6 323 lignes restantes. Pourquoi y consigner les effectifs avant et après chaque étape ?
28. Le chiffre d'affaires du Site au quatrième trimestre donne 211 434 € d'un côté et 176 195 € de l'autre. Quelle est l'explication la plus simple, et que faut-il écrire pour éviter la confusion ?
29. Que contient un dictionnaire de données, et comment vérifie-t-on automatiquement qu'il n'est pas périmé ?
30. « Client actif » donne 2 654, 3 148 ou 3 875 selon la définition. Que faire ?
31. Une empreinte (hachage) de fichier prouve-t-elle que les données sont **correctes** ?
32. Que signifie « refaire depuis le brut », et pourquoi est-ce la meilleure documentation ?

Confidentialité (chapitre 5)

33. Classez en identifiant direct, quasi-identifiant ou donnée sensible : adresse électronique, année de naissance, ville, état de santé, numéro de client.
34. Citez trois principes communs aux cadres de protection des données.
35. Un hachage sans clé des adresses électroniques protège-t-il les personnes ? Que fait une clé secrète en plus ?
36. Une donnée pseudonymisée est-elle anonyme ?
37. Un tableau compte quatre groupes de 3, 2, 5 et 4 personnes sur les quasi-identifiants. Quel est son k-anonymat, et que faire pour atteindre $k = 5$?
38. Pourquoi 26 % de clients « uniques » sur ville, année de naissance, canal et carte posent-ils problème ?
39. Peut-on publier un tableau où l'on masque les petits effectifs, mais avec les totaux de ligne et de colonne ?
40. En confidentialité différentielle, avec un bruit de Laplace d'échelle $1/\epsilon$, quelle est l'erreur médiane pour $\epsilon = 0,1$ et pour $\epsilon = 1$?

Corrigés des questions

Pour les questions numériques, **calculons** plutôt que de nous fier à la mémoire.

```
import numpy as np, pandas as pd
from unidecode import unidecode
import ftfy
from rapidfuzz import fuzz
from rapidfuzz.distance import Levenshtein

print("Q2  moyenne de 20, 30, 9999 :", round(np.mean([20, 30, 9999]), 1), "| sans le placeholder :",
      ↪ np.mean([20, 30]))
print("Q8  réparation du mojibake :", ftfy.fix_text("Ã©tÃ©"), "|",
      ↪ "été".encode("utf-8").decode("cp1252"))
print("Q13 marge hors taxe :", round(50 / 1.2 - 22, 2), "| taux de marge :", round((50 / 1.2 - 22) /
      ↪ (50 / 1.2) * 100, 1), "%")
print("Q17 Levenshtein :", Levenshtein.distance("Dormar", "Dorrmar"), "| fuzz.ratio :",
      ↪ round(fuzz.ratio("Dormar", "Dorrmar"), 1))
print("Q22 seuil de tolérance :", round(0.001 * 560973.91, 2), "€ | 180,78 € toléré :", 180.78 < 0.001
      ↪ * 560973.91)
print("Q28 TTC -> HT :", round(211433.79 / 1.2, 2))
groupes = pd.Series([3, 2, 5, 4], index=list("ABCD"))
print("Q37 k-anonymat :", groupes.min(), "| groupes à supprimer ou fusionner pour k = 5 :",
      ↪ list(groupes[groupes < 5].index))
print("Q40 erreur médiane de Laplace : eps = 0,1 ->", round(np.log(2) / 0.1, 2), "| eps = 1 ->",
      ↪ round(np.log(2) / 1, 2))
```

```
Q2  moyenne de 20, 30, 9999 : 3349.7 | sans le placeholder : 25.0
Q8  réparation du mojibake : été | Ã©tÃ©
Q13 marge hors taxe : 19.67 | taux de marge : 47.2 %
Q17 Levenshtein : 1 | fuzz.ratio : 92.3
Q22 seuil de tolérance : 560.97 € | 180,78 € toléré : True
Q28 TTC -> HT : 176194.83
Q37 k-anonymat : 2 | groupes à supprimer ou fusionner pour k = 5 : ['A', 'B', 'D']
Q40 erreur médiane de Laplace : eps = 0,1 -> 6.93 | eps = 1 -> 0.69
```

1. Revenu manquant selon l'âge : **MAR** (l'absence dépend d'une variable observée). Satisfaction manquante quand elle est basse : **MNAR** (l'absence dépend de la valeur elle-même). Dépense manquante au hasard : **MCAR**. Supprimer les lignes incomplètes ne biaise que dans le cas MCAR ; en MAR et MNAR, la suppression **déforme** la population (par exemple moins de jeunes, moins de clients mécontents) (1.1.3, 1.1.4).

2. 9999 est un **code spécial** (un « placeholder » de saisie), pas un montant. La moyenne de 20, 30 et 9999 vaut 3 349,7 contre 25 sans le code : un seul placeholder ruine la moyenne. On le remplace par une valeur manquante avant tout calcul (1.1.2).

3. Une valeur extrême peut être **réelle** (un très gros achat) : l'erreur se juge par la **règle métier** (un montant doit valoir quantité × prix), pas seulement par la distance à la moyenne. Trois façons de la repérer : la règle des **1,5 écart interquartile**, le **score z robuste** (médiane et MAD), et une **règle métier** ou un contrôle croisé avec une table de référence (1.2.1 à 1.2.3).

4. Non. Parmi ces 154 lignes identiques, la vérité en désigne 67 comme de vrais doubles scans et 87 comme des répétitions **légitimes** (deux articles identiques achetés ensemble). Un `drop_duplicates` aveugle supprimerait 87 vraies ventes. On compare avec une source de référence (le total du ticket) et l'on ne retire que ce que l'on peut prouver (1.3.3).

5. Un montant moyen multiplié par 96 d'un jour à l'autre n'est pas un miracle commercial : c'est un **changement d'unité** (centimes). On le confirme par la **rupture temporelle** (le saut est net au 15 septembre) et par le contrôle contre une source de référence ; on corrige en divisant par cent à **partir de la date de rupture**, ce qui ramène la moyenne à 99,14 € (1.4.2).

6. Sans autre information, c'est ambigu. On lève le doute avec la **source** (la plateforme écrit-elle jour/mois ?), avec les **autres valeurs de la colonne** (un 13/04/2025 ne peut pas être un mois), et en documentant le format retenu. Le mois et le jour ne se devinent jamais ligne par ligne sans règle (1.4.3).
7. La normalisation (casse, espaces, point final) ramène VILLE A, ville a et Ville A. à Ville A. La **faute** (Vile A) et l'écriture **arabe** ne se corrigent pas par une règle générale : il faut une **table de correspondance** vérifiée, qui est aussi une décision à documenter (1.4.4, 1.5.6).
8. Le texte a été écrit en **UTF-8** puis lu comme du **cp1252** : chaque caractère accentué devient deux caractères (c'est le « mojibake »). On le répare en relisant avec le bon encodage, ou avec une bibliothèque comme `ftfy` (code ci-dessus : `ftfy.fix_text("Ã©tÃ©")` rend « été ») (1.5.2, 1.5.5).
9. La moyenne reste inchangée, mais la **dispersion diminue** (on ajoute des valeurs toutes égales) et les corrélations sont atténuées. Une imputation ne répare pas un MNAR : si l'absence dépend de la valeur, aucune information des autres colonnes ne la contient. Dans nos données, après imputation, la part de clients très insatisfaits reste à 2,55 % contre 4,08 % en vérité (1.6.1, 1.6.4).
10. Deux produits portent le même nom : la jointure sur le nom **multiplie** les lignes (12 678 devient 25 356). On le détecte **avant de sommer** par le contrôle des effectifs (lignes avant et après) et par `validate="m:1"` ; on corrige en ajoutant une clé qui distingue les homonymes (ici le prix), puis en vérifiant les effectifs (2.2.2, 2.2.5).
11. **Avant** : la clé est-elle unique dans la table de droite, et quelle est la cardinalité attendue ? **Après** : le nombre de lignes est-il celui que l'on attend (inchangé pour une jointure n-1), et le total d'une colonne numérique est-il conservé ? (2.2.2).
12. Avec une **fonction de lecture paramétrée** par fichier (encodage, séparateur, décimale, noms de colonnes, format de date), appliquée aux douze fichiers, puis un concat ; chaque fichier est contrôlé contre sa propre ligne de total (2.2.3).
13. Prix hors taxe = $50/1,2 \approx 41,67$ € ; marge brute = $41,67 - 22 = 19,67$ € ; taux de marge = $19,67/41,67 \approx 47,2$ % (code ci-dessus). On compare des montants hors taxe aux coûts hors taxe (2.1.2).
14. `cut` découpe selon des **bornes fixées** (classes de largeur choisie, par exemple des tranches d'âge) ; `qcut` découpe selon des **quantiles** (classes d'effectifs à peu près égaux) (2.1.4).
15. Une **somme de contrôle** : le total du montant au grain commande doit être égal au total au grain ligne, et le nombre de commandes distinctes doit rester le même (2.3.7).
16. On lit les cellules en texte, on convertit ND et – en valeurs **manquantes**, rupture en **zéro**, on retire les espaces avant de convertir, on supprime titres et sous-totaux, puis on **dépivote** les colonnes de mois en lignes (`melt`) (2.4.2).
17. La distance de Levenshtein vaut **1** (un caractère à ajouter) et `fuzz.ratio` donne 92,3 (code ci-dessus). Un seuil unique mélange deux erreurs de coût différent : fusionner deux personnes distinctes ne se rattrape pas, rater un doublon se corrige plus tard. On utilise donc **trois zones** : accepter, revoir à la main, rejeter (2.5.3, 2.5.5, 2.5.6).
18. Par exemple : **exactitude** : écart entre le total de la source et celui d'une référence ; **complétude** : part de cellules renseignées ; **cohérence** : part des lignes qui respectent une règle entre colonnes (inscription avant naissance) ; **actualité** : âge de la dernière donnée (jours depuis l'extraction) (3.1.2 à 3.1.6).
19. C'est une règle qui **renvoie les lignes en échec** (et non un simple booléen) : on voit **quoi** corriger et combien, on peut trier, compter et joindre les exceptions à un rapport (3.2.1, 3.2.6).

20. (1) **Comparer les effectifs** (mêmes lignes, mêmes clés ?) ; (2) **comparer les totaux** (même somme ?) ; (3) **expliquer l'écart ligne à ligne** jusqu'à zéro (doublons, unités, annulations, arrondis) (3.3.2).

21. Ce n'est pas une erreur de données : la base compte les commandes annulées comme des commandes, la plateforme les marque « annulées ». Il faut **décider** de la définition du chiffre d'affaires (commandé ou encaissé), puis **l'écrire** dans le dictionnaire (3.3.6).

22. Une tolérance de 0,1 % vaut 560,97 € ; **180,78 € est toléré** (code ci-dessus), mais l'écart reste consigné au rapport : une tolérance accepte un écart, elle ne l'efface pas (3.4.2).

23. On **trie par montant** (ou par effet sur le total) et l'on regroupe par **cause** : 5 exceptions qui représentent 80 % de l'écart passent avant 100 exceptions de quelques centimes (3.4.5).

24. Des **fonctions maison** suffisent pour une analyse ponctuelle ; **pandera** convient pour valider le schéma d'un DataFrame de façon déclarative dans le code ; **Great Expectations** vaut son poids pour un **pipeline récurrent** avec rapports partagés (3.5.5).

25. Un indicateur de complétude ne regarde que le **vide** : N/A est un texte rempli. Un indicateur ne remplace pas la **validité** et le regard sur les valeurs ; il se lit toujours avec les autres dimensions (3.1.8).

26. Par exemple : la **source** et le propriétaire, la **date d'extraction**, le **périmètre**, le **grain**, la **clé**, la **description des colonnes**, les **limites connues**, la **version** et la **licence ou les droits** (4.1.1).

27. Parce qu'un journal avec les effectifs permet de **vérifier chaque étape** (le nombre de lignes retirées est-il celui qu'on attend ?) et de **retrouver** une perte inexpiquée. Ici : $7\,140 - 140 - 677 = 6\,323$; on peut ensuite comparer à la vérité et voir que 674 des 677 retraits étaient de vrais doublons (4.1.3 à 4.1.5).

28. Le premier chiffre est **TTC**, le second **hors taxe** : $211\,433,79 / 1,2 \approx 176\,194,83$ (code ci-dessus). Il faut écrire dans le dictionnaire et sur le rapport la **définition** du montant (TTC ou HT, remises déduites ou non, période exacte) (4.2.6).

29. Pour chaque colonne : nom, libellé, type, unité, valeurs permises, caractère obligatoire, codage des manquants, exemple, règle de calcul, source et sensibilité. On le teste automatiquement en comparant ses colonnes, ses types et ses domaines à ceux du fichier réel : tout écart fait échouer le test (4.2.1, 4.2.5).

30. On ne choisit pas au hasard : on **écrit une définition unique** (par exemple « au moins une commande dans les 12 derniers mois ») dans le glossaire métier, on l'utilise partout, et l'on indique quelle définition correspond à quel chiffre publié (4.2.6).

31. Non : une empreinte prouve que le fichier est **identique** à celui que l'on a documenté, pas qu'il est **correct**. Elle protège contre les modifications silencieuses, pas contre les erreurs d'origine (4.3.4).

32. C'est disposer d'un script qui, à partir des **fichiers bruts** et d'une configuration, reproduit **sans retouche manuelle** les tables et les chiffres. Elle documente mieux que n'importe quelle description : si le script donne le même chiffre, la documentation est exacte (4.1.7, 4.3.8).

33. **Adresse électronique** : identifiant direct ; **numéro de client** : identifiant direct (indirect si la table de correspondance est séparée) ; **année de naissance** et **ville** : quasi-identifiants ; **état de santé** : donnée sensible (5.1.2).

34. Par exemple : la **finalité** (on collecte pour un usage déclaré), la **minimisation** (seulement ce qui sert), la **limitation de conservation**, la **sécurité**, le **consentement ou une autre base légale**, et les **droits des personnes** (5.1.3).

35. Non : les adresses se retrouvent par **dictionnaire** (on hache les adresses plausibles et l'on compare : 85,7 % retrouvées dans notre jeu). Une **clé secrète** (HMAC) conservée à part rend cette attaque impossible sans la clé, tout en gardant le même identifiant pour joindre des fichiers (5.2.2, 5.2.3).

36. Non : une donnée **pseudonymisée** reste personnelle puisque l'on peut en principe retrouver la personne (table de correspondance, recoupement). Seule une anonymisation irréversible sort du champ des données personnelles, et elle est difficile à garantir (5.2.5).

37. Le k-anonymat est la **plus petite taille de groupe : 2** (le groupe B). Pour atteindre $k = 5$, il faut **généraliser** (tranches d'âge, régions) ou **supprimer** les lignes des groupes A (3), B (2) et D (4), jusqu'à ce que chaque groupe compte au moins cinq personnes (code ci-dessus) (5.3.2, 5.3.3).

38. Une personne **unique** sur ces quatre colonnes est identifiable par quiconque connaît ces quatre informations (par recoupement avec une autre source). Un quart de clients uniques signifie que la table, malgré l'absence de nom, **n'est pas anonyme** (5.3.1, 5.3.5).

39. Non, pas sans précaution : si l'on publie aussi les totaux de lignes et de colonnes, on peut **retrouver les cases masquées par soustraction** (dans notre exemple, toutes les valeurs masquées sont retrouvées). Il faut un masquage complémentaire ou ne pas publier les totaux détaillés (5.4.2).

40. L'erreur médiane vaut $\ln 2/\varepsilon$: environ **6,93** pour $\varepsilon = 0,1$ et **0,69** pour $\varepsilon = 1$ (code ci-dessus) : plus la protection est forte (ε petit), plus le comptage est bruité (5.3.7).

Votre grille d'auto-évaluation

Pour chaque ligne : **je sais l'expliquer** / **je sais le faire** / **à revoir**.

Compétence	Où la retravailler
Repérer et traiter des valeurs manquantes selon leur mécanisme	1.1
Distinguer valeur aberrante et erreur, choisir une règle	1.2
Détecter et traiter doublons exacts et flous	1.3
Corriger formats, unités, dates, catégories, schémas	1.4
Nettoyer texte, encodage et dates (⊕)	1.5
Imputer et en mesurer l'impact (⊕)	1.6
Dériver des variables et les documenter	2.1
Joindre sans multiplier les lignes, empiler des fichiers	2.2
Agréger et changer de grain avec contrôle	2.3
Passer du large au long, lire un tableur désordonné (⊕)	2.4
Rapprocher des enregistrements approximativement (⊕)	2.5
Mesurer la qualité selon plusieurs dimensions	3.1
Écrire des contrôles de validation	3.2
Réconcilier deux sources et expliquer les écarts	3.3
Régler des tolérances et un rapport d'exceptions (⊕)	3.4

Compétence	Où la retravailler
Utiliser pandera ou Great Expectations (⊕)	3.5
Documenter un jeu de données et ses transformations	4.1
Construire et tester un dictionnaire de données	4.2
Tracer le lignage et rejouer un chiffre (⊕)	4.3
Pseudonymiser, anonymiser, mesurer un risque de réidentification	5.1 à 5.4
Mener un nettoyage et une réconciliation de bout en bout	Projet du volume